Iso-Memory Comparison of Weight Bit-Widths in Small Quantization-Aware Transformers

Daniel Yordanov

Abstract

At fixed weight storage, which number of bits per weight b minimises validation loss? I cross 7 bit-widths, ternary to bf16, with budgets of 256 Kbit, 828 Kbit and 4 Mbit, size each model to fill its budget, and train quantization-aware under one untuned learning-rate rule (5 seeds per cell; TinyStories, and FineWeb-Edu at two budgets).

On the preregistered metric `val_loss_best`, the lowest mean is at $b = 6$ at every budget. That result is decisive by my criteria (section 4) only at 256 Kbit, where changing b also changes depth and MLP ratio. At 828 Kbit the preregistered null criterion fires (gap 0.0123 nats, below the largest seed standard deviation, 0.0201; the ± 1 standard deviation intervals overlap), so I name no best bit-width. At 4 Mbit it does not fire (Welch $p = 0.0317$, uncorrected), but the gap of 0.0061 nats is below the power threshold (2.0 seed standard deviations), the intervals overlap, and the lowest mean depends on end-of-training averaging ($b = 4$ on averaged weights).

In the test-set read, the final-step lowest mean matches validation at every budget, but the 4 Mbit gap is above the power threshold (below it on `val_loss_best`), the averaged-weights lowest mean is $b = 4$, and at 828 Kbit averaged weights no longer give a null (on validation, $b = 3$ and $b = 6$ stay unresolved at 10 seeds). On FineWeb-Edu (not averaged) the 4 Mbit lowest mean is $b = 4$, which the preregistered rule reports as dependence on the text. On TinyStories, ternary, 2-bit and (where feasible) bf16 weights are worse than the lowest mean at every budget, on `val_loss_best` and on the final, averaged and test losses; their learning rate was not tuned.

Weight-only studies at fixed memory (19M parameters and above) place the optimum at 1–4 bits, or up to 6 bits at low training compute. On `val_loss_best` my lowest mean, $b = 6$ below 3M parameters, is above the 1–4-bit range and equals the low-compute 6-bit end; on averaged weights it is $b = 4$ at 4 Mbit, inside that range.

1 Introduction

When the weights of a language model must fit in a fixed amount of storage, precision and size are no longer independent: N weights at b bits each occupy about Nb bits, so at fixed storage the number of weights that fit scales as $1/b$, and halving b roughly doubles it. Headline results for low-bit language models, ternary especially, are often stated at equal parameter count (Ma et al., 2024), which gives the higher-precision model proportionally more storage; Kaushal et al. (2025) also compare ternary models at equal bit size. Comparisons at matched weight bits have also been made, for post-training quantization of released checkpoints from 19M to 176B parameters (Dettmers and Zettlemoyer, 2023) and for quantization-aware training (QAT) at 30M non-embedding parameters and above (Frantar et al., 2025; Panferov et al., 2025a; Dremov et al., 2026; Maskey et al., 2026). Kumar et al. (2025) enter here only through my own evaluation of their fitted weights-only law, which they do not report as a result. Throughout, a bit-width ranks above another when it gives the lower loss. Inside the 3–8-bit part of their 3-to-12-bit weight sweep the law ranks 3 bits above 4 above 6 (my derivation from their Eq. 3 and the constants of their Table 2; section 2). These studies put the memory-optimal weight precision at 1 to 4 bits, with one exception: up to 6 bits in

experiments (8 in the fitted law) at low training compute (Dremov et al., 2026), whose models are pretrained in full precision before QAT. The one comparison of Kumar et al. (2025) that holds the product of parameters and precision fixed has its minimum at FP6, but it also quantizes activations and attention and so is not a weight-only comparison. The studies differ in scale, quantizer, training compute and what they hold fixed, and none extends below 19M parameters (30M non-embedding parameters for QAT; section 2).

I ask: *at a fixed budget of B bits of weight storage, which number of bits per weight b gives the lowest validation loss?* I call a comparison at fixed B *iso-memory*. The stored weight bits W (eq. (2)) count the linear layers and the tied embedding at b bits per weight and the normalisation gains at 16 bits, and B is the target for W. My grid crosses 7 bit-widths, $b \in \{1.58, 2, 3, 4, 6, 8, 16\}$ (ternary to bf16), with 3 budgets, 256 Kbit, 828 Kbit and 4 Mbit. A *cell* is one (B, b) pair with its architecture; 20 cells are feasible, the 256 Kbit bf16 cell being impossible because its embedding table alone fills the budget at the minimum width. Each cell's architecture is searched to fill its budget (every model lies within 4.15% of B) and is trained under one untuned learning-rate rule, with quantization-aware training from step 0 at every $b < 16$ ($b = 16$ is an unquantized bf16 control). Since lowering b buys parameters, equal token budgets would leave low-precision cells the least trained. Instead, a gain model (the loss removed by training longer; section 3.7) fitted on off-grid calibration runs sets each cell's token budget, so that every cell is *predicted* to end 0.020 nats per token above its converged loss. Each primary cell has 5 seeds (the shape-robustness cells of section 5.4 have 3), and each mean is reported with its *seed standard deviation*, the sample standard deviation over seeds. Models are trained on TinyStories (Eldan and Li, 2023). The primary cells span 32 768 to 2 616 320 parameters, and all trained models (the shape-robustness cells included) span 32 768 to 2 621 440; I make no claim beyond that range (section 7). Predictions P1–P5 and a numerical null criterion were committed to my local git repository before the first sweep run, and were externally timestamped only after the sweep had finished. P6 and P7 were added after two budgets. At that time at least one 4 Mbit ternary sweep run existed (a second finished within about a minute of the commit), so P6 is post hoc. P7 concerns the 4 Mbit argmin over all bit-widths; apart from the ternary runs, only a $b = 6$ convergence-pilot run at 4 Mbit existed. P8–P10 were added before any run of the second corpus (section 4). To my knowledge, no prior study compares three or more uniform weight precisions, ternary included, at matched weight storage for transformer language models trained with QAT from the first step below about 30M non-embedding parameters (section 2).

On the primary metric, `val_loss_best`, the loss has an interior minimum over b at every budget, with the lowest mean at $b = 6$ in all three (fig. 1 and table 1), but the strength of the evidence for it differs by budget (table 2). At 256 Kbit it is decisive in the sense defined in section 4: $b = 6$ leads $b = 8$ by 0.0952 nats, 12.53 times the largest seed standard deviation of the three cells around the minimum (Welch 95% interval 0.0876 to 0.1027), though at 256 Kbit the tolerance on stored bits leaves little freedom of shape, so changing b also changes depth and MLP ratio (section 3.3).

At 828 Kbit the preregistered criterion returns null: $b = 6$ and $b = 8$ differ by 0.0123 nats, less than the largest seed standard deviation, 0.0201 ($p = 0.212$), and under a second architecture-search rule, declared in advance, the lowest mean moves to $b = 4$; I name no best bit-width there.

At 4 Mbit the criterion returns not null: $b = 6$ leads $b = 4$ by 0.0061 nats (95% Welch interval 0.0007 to 0.0114), the preregistered Welch test against it gives $p = 0.0317$, uncorrected as preregistered (exact permutation $p = 0.0317$), and the second rule also puts the lowest mean at $b = 6$. The gap is, however, below 0.0085 nats, which is 2.0 times the larger seed standard deviation of the pair. By the preregistration's estimate 5 seeds give roughly 80% power against a true gap of 2.0 seed standard deviations (77% in simulation); that is the power at the threshold gap, not at the observed gap. The lower end of the 95% interval is 0.0007 nats. A Holm correction over

the 6 adjacent tests, which was not preregistered, does not reject this comparison (section 5.1), and an exploratory equivalence test (two one-sided tests) does not show the two within 0.01 nats either. The 4 Mbit $b = 6$ cell is also a convergence-pilot cell, which the preregistration's pilot clause excludes and its run plan includes; read by the clause, the 4 Mbit minimum is at $b = 4$, not null and above the power threshold. I report both readings (section 5.1 and table 6).

Of the preregistered predictions (table 11), P1 is *partly correct* (an interior minimum, but not at the predicted 2–4 bits), P2 is *wrong* (the argmin was 6/6/6 at 256 Kbit/828 Kbit/4 Mbit, not the predicted 4/3/2), P3 is *not testable as preregistered* and holds only under a scoring rule written after the data existed, P4 is *correct* (every configuration beats a bigram baseline), and P5 is *wrong* (the null fired at 828 Kbit, not at the smallest budget, and the 4 Mbit gap is below the power threshold). The two predictions added after two budgets, when some 4 Mbit data existed, are scored *correct* with qualifications: P6's bound held on the primary sweep, while under the alternative shape rule the penalty exceeds it, and P6 is post hoc; P7 holds only with the pilot cell included. Except for the one read of the test split, losses are validation losses. The primary metric `val_loss_best` is the minimum over each run's validation evaluations on one fixed validation set; the averaged loss of Amendment A7 (below) is the validation loss of averaged weights. The minimum is biased downward; an exploratory estimate puts the bias generally larger at low b, whose evaluations are noisier (section 5.5). Under the final validation loss, which involves no selection, the lowest mean is at the same b at every budget.

The one read of the test split, made after all training had finished and every validation verdict was final (section 5.6), scored each primary run's final-step weights and its averaged weights (the full-precision latent weights averaged over a 400-step continuation at the final learning rate and then quantized; Amendment A7, below). On the final-step weights the test argmin is the validation argmin and the null criterion returns the validation verdict at every budget. At 4 Mbit, however, the gap of $b = 6$ to $b = 4$ (0.0112 nats, $p = 0.00393$) is above the power threshold, where on `val_loss_best` it is below. On the averaged weights the test argmin is $b = 4$ at 4 Mbit, not the `val_loss_best` argmin, and at 828 Kbit the null criterion does not fire, where on `val_loss_best` it does; in each case the test result matches the corresponding result on the averaged validation loss. The preregistered primary verdicts remain those on `val_loss_best`.

My lowest-mean bit-width on `val_loss_best`, $b = 6$ at every budget and named best only at 256 Kbit, is at higher precision than the 1–4-bit optima of the weight-only comparisons above. Inside the 3–8-bit range that Dettmers and Zettlemoyer (2023) and Kumar et al. (2025) both cover, those two rank 4 bits above 6 (for Kumar et al., by my evaluation of their fitted law), and on `val_loss_best`, with the pilot cells included, I rank 6 above 4 at every budget. That difference could come from scale, from the quantizer, or from my training recipe. After the sweep I recorded checks of the recipe as Amendment A7: end-of-training weight averaging, the learning rate, the training length and more seeds (section 5.8). It was written before its own runs but after the seed-0 averaging measurements that motivated it had been seen (section 4). Amendment A8 then extended the learning-rate search until the optimum of each argmin-group cell was bracketed. The argmin-group cells are the 10 of the 20 cells that are the averaged-loss argmin, one of its adjacent bit-widths, or within a preregistered margin of it; the other cells, including all ternary, 2-bit and bf16 cells, were not tuned. At 256 Kbit the argmin survives every check that applies: it is robust to averaging (same lowest mean, no reversal of an adjacent gap beyond the larger seed standard deviation of the pair), does not move when the learning rate of its group is tuned (1 seed per multiplier), and is not sensitive to training length (the spread of the half-length gains across the cells is below the gap to the nearest neighbour; also one seed per cell). At 828 Kbit, on the 5 primary seeds, $b = 6$ stays lowest after averaging and is not null against its grid neighbours, but $b = 3$ is 0.0059 nats above it ($p = 0.305$). With 10 seeds per cell, $b = 3$ minus $b = 6$ on the averaged

loss is -0.0009 nats ($p = 0.832$). The minimum detectable effect is 0.0164 nats at $p \leq 0.01$ and 80 % simulated power. By the rule of Amendment A7 (Welch $p > 0.01$ at the cap of ten seeds) the comparison is unresolved, and I name no best bit-width between $b = 3$ and $b = 6$. The range across cells of the mean half-length gain (the loss increase when training is halved), 0.0233 nats, also exceeds the gap between $b = 3$ and $b = 6$.

At 4 Mbit averaging moves the argmin from $b = 6$ to $b = 4$ (gap 0.0230 nats, $p < 10^{-4}$). The averaged-loss argmin $b = 4$ stays lowest at every learning-rate multiplier that all cells of its group ran, and with each cell at its own best multiplier. The ranking is not sensitive to training length. With each cell of the argmin group at its own best multiplier, tuning the learning rate does not move the averaged-loss argmin at any budget; at the single multiplier $\times 2$ and 828 Kbit the argmin is $b = 3$. At 256 Kbit none of the recipe checks I ran moves the argmin, so the difference from the 1–4-bit optima is not explained by those recipe choices there; scale, the quantizer and the parts of the recipe that Amendment A7 did not vary (for example weight decay, warmup and batch size) remain candidates. At 4 Mbit the argmin depends on end-of-training averaging, and at 828 Kbit the comparison of $b = 3$ and $b = 6$ is unresolved.

A second arm on FineWeb-Edu (Penedo et al., 2024), educational web text, registered after the TinyStories sweep had finished and before any of its own runs, tests at 256 Kbit and 4 Mbit whether the result is an artifact of simple text (section 6). The argmin is $b = 6$ at 256 Kbit, as on TinyStories, and $b = 4$ at 4 Mbit, 0.0125 nats below $b = 6$ ($p = 0.0019$), where TinyStories has $b = 6$ on the same metric; P8 ($b \leq 6$) therefore holds. The preregistered rule reports the 4 Mbit difference as dependence on the text, but on the averaged loss TinyStories also has $b = 4$ there, and the FineWeb-Edu runs were not averaged. At 256 Kbit the half-length check finds the FineWeb-Edu ranking not sensitive to training length; at 4 Mbit it finds it not sensitive.

My contributions are:

1. An iso-memory protocol: budget-filling architecture search reported at stored bits, quantization-aware training from step 0 with absmean-derived per-tensor scaling at every $b < 16$, and per-cell token budgets that equalise the predicted convergence residual rather than leave it correlated with b (section 3).

2. A measurement of the lowest-mean bit-width at fixed weight memory for models of under 3M parameters, where, to my knowledge, it had not been measured. With one bit-width for every weight matrix, per-tensor absmean-derived scaling and a straight-through estimator, QAT from step 0 and one untuned learning-rate rule, ternary and 2-bit weights are worse than the lowest-mean b at fixed weight memory at all three budgets, by gaps larger than the seed standard deviations of the cells compared (table 2). The lowest mean is at $b = 6$ at every budget in the primary analysis: decisive at 256 Kbit, null against $b = 8$ and dependent on the shape rule at 828 Kbit, and at 4 Mbit not null but below the power threshold, and dependent on the reading of the pilot clause and on end-of-training averaging (section 5 and table 6).

3. The difference between this lowest mean and the 1–4-bit optima reported at 19M parameters and above (up to 6 bits in experiments at low training compute), stated as a finding, with checks of whether it is an artifact of my recipe, recorded after the sweep as Amendment A7 (sections 2 and 5.8). The checks leave the argmin in place at 256 Kbit, leave $b = 3$ and $b = 6$ unresolved at 828 Kbit, and move it to $b = 4$ at 4 Mbit under end-of-training averaging.

4. A ternary penalty (defined in section 5.2) that falls monotonically from 0.5102 to 0.2108 to 0.0842 nats over a 16-fold range of budget while the lowest mean stays at $b = 6$ (seed standard deviations in table 1). This is the direction expected if ternary becomes competitive at much

larger scale (Ma et al., 2024; Kaushal et al., 2025), and also what scale-free effective-parameter laws imply with no crossover, by my deduction from the form of the laws (Panferov et al., 2025a; Kumar et al., 2025; Maskey et al., 2026); the law of Kumar et al. (2025) was fitted at 3 bits and above, so ternary lies outside it. Three budgets cannot distinguish the two. The bound recorded as P6 after two budgets held under the primary shape rule; it was recorded when at least one 4 Mbit ternary sweep run existed, so it is post hoc, not an out-of-sample test.

5. An observation, below 3M parameters with QAT from step 0 and 5 seeds, that the matched-parameter and matched-storage readings of the same runs rank precision differently: on `val_loss_best` at nearly matched parameter count (section 5.3 gives each pair's mismatch), higher precision wins 4 of 4 pairs (3 distinct higher-precision cells), but only by storing more bits. Such a difference is implicit in the fixed-bit analysis of Dettmers and Zettlemoyer (2023), and has been reported for ternary language models (Kaushal et al., 2025), for weights and activations at 1–4 bits (Panferov et al., 2025a), for binary translation Transformers (Zhang et al., 2023), and for CNNs (Moons et al., 2017); both readings are plotted for ResNets (Abdolrashidi et al., 2021) (section 5.3).

6. A self-registered, multi-seed study with its code: 142 completed runs (100 primary, 42 shape-robustness), every TinyStories prediction (P1–P7) scored, including those the results did not support, P8–P10 scored on the second corpus (section 6), and every result number in this paper computed by a script from the recorded results (that script is not released); the code and results are released at `https://github.com/milar111/iso-memory-scaling`.

2 Related Work

Precision at a fixed number of weight bits. Dettmers and Zettlemoyer (2023) compare post-training-quantized language models at fixed total model bits. From 19M to 176B parameters, at 3–8 bits with a 16-bit baseline, they find 4 bits optimal for mean zero-shot accuracy for almost every family and scale (the one stated exception is BLOOM-176B, where 3 bits is, in their words, slightly but not significantly better). With a stronger quantizer, 3-bit Llama 1 and Llama 2 models (7B to 70B) scale better than 4-bit ones in Wikitext2 perplexity (Tseng et al., 2024), so, for these models, the 4-bit optimum depends on the quantizer. For training in low precision, Kumar et al. (2025) fit a precision law at 30–220M non-embedding parameters and report 7–8 bits as the compute-optimal precision, where compute is the product of parameters, tokens and precision, and weights, activations and attention are quantized together. Their one comparison at fixed parameters times precision (five models of 220M to 1.76B parameters), quantized the same way, has its minimum at FP6. They report no weight-only optimum at fixed weight memory. Evaluated at fixed weight memory and fixed token count, their fitted weights-only law ranks 3-bit weights above 4-bit above 6-bit (a bit-width ranks above another when it gives the lower loss), inside the 3–8-bit part of their 3-to-12-bit weight sweep. This is my derivation from their Eq. 3 and the constants of their Table 2. Their Appendix K does not say which fit those constants come from, and the ranking is the same under each reading of the offset that Appendix K mentions. Frantar et al. (2025) train weight-only QAT models from scratch at roughly 0.2–1.6B non-embedding parameters (read off their figure). Per non-embedding bit they rank the precisions monotonically, 1 bit best, assuming effective-parameter multipliers per precision that do not depend on model size. QuEST finds 2-bit weights apparently Pareto-dominant among 1–4 bits when only weights are quantized (30–200M parameters), and about 4 bits when activations are quantized too (30–800M non-embedding parameters in the fitted law)

(Panferov et al., 2025a). The capacity law of Panferov et al. (2025b) agrees, in a single remark, that 2-bit weight-only quantization is Pareto optimal for its architectures, at 30–200M non-embedding parameters. Maskey et al. (2026) find fewer bits "generally preferable under a fixed weight memory budget" at 0.8–3.9B parameters, a ranking that follows from the saturating form they fit once embedding bits are not counted. Their appendix, which counts the 16-bit embedding, puts the optimum for uniform formats at 2 bits for an 8 GB budget and notes that it shifts upward at very small budgets. Dremov et al. (2026) study a closely related question at 86M–759M total parameters (the 2.2B model checks only their QAT-fraction prediction), with full-precision pretraining followed by QAT, with training FLOPs fixed as well as memory, and with the embedding quantized to at least 4 bits, not to the QAT width. At a fixed memory budget they find the optimal precision falling as training compute grows: up to 6 bits in their experiments (1, 2, 4 and 6 bits) at low compute, and up to 8 bits in their fitted law, which is the top bin of the colour scale of their figure. ParetoQ (Liu et al., 2025), with QAT mostly from full-precision checkpoints at 125M parameters and above, reports that ternary, 2- and 3-bit weights are tied in performance and often surpass 4-bit ones in accuracy against model size, with 2-bit and ternary on the Pareto frontier, while binary quantization substantially compromises accuracy.

The weight-only comparisons among these studies place the memory-optimal precision at 1 to 4 bits at 19M parameters and above, depending on scale, quantizer and recipe, and up to 6 bits in experiments at low training compute (Dremov et al., 2026). Below 3M parameters I find an interior lowest mean, at $b = 6$ on the primary metric at every budget, with ternary and 2-bit weights worse than the lowest-mean b at every budget (section 5). Inside the 3–8-bit range that Dettmers and Zettlemoyer (2023) and Kumar et al. (2025) both cover, those two rank 4 bits above 6 (for Kumar et al., by my evaluation of their fitted law), and on the primary metric I rank 6 above 4, decisively at 256 Kbit. This difference from prior work is the main contrast with the studies above, and it is decisive by my criteria only at 256 Kbit, where precision, depth and MLP ratio vary together (below). It can come from scale, since no study I found compares precisions at fixed weight memory with QAT between about 3M and 30M parameters; from the quantizer, since per-tensor absmean-derived scaling with a straight-through estimator differs from the block-wise, per-channel, rotated or learned-step quantizers above; or from my training recipe. Amendment A7 tests the recipe (section 5.8). It leaves the argmin in place at 256 Kbit under weight averaging, learning-rate tuning of its argmin group and half-length training (the last two on one seed per cell). At 828 Kbit it leaves $b = 3$ and $b = 6$ unresolved after 10 seeds each. At 4 Mbit, end-of-training averaging moves the argmin on the averaged loss from $b = 6$ to $b = 4$, which then survives learning-rate tuning and half-length training, though the argmin on `val_loss_best` stays at $b = 6$. Ternary and 2-bit weights stay worse than the lowest-mean b at every budget. Scale and quantizer remain untested, as do the parts of the recipe that Amendment A7 did not vary.

Low-bit and ternary language models trained from scratch. BitNet b1.58 (Ma et al., 2024), which extends the binary BitNet (Wang et al., 2023) with the absmean ternarizer I use, reports parity with FP16 from 3B parameters at matched parameter count and tokens, with the embedding at full precision. Spectra (Kaushal et al., 2025) finds ternary models ahead of quantized and floating-point ones of the same bit size above one billion parameters, with the embedding and head counted at half precision. At 3.9B parameters and equal parameter count it reports competitive average scores across six benchmarks for its ternary model against the FP16 model, with a higher validation perplexity. In small language models of 6M to 48M parameters, ternary models need hidden layers about twice as wide as 16-bit ones to reach comparable performance (compared at hidden sizes 32 to 128), which the authors read as about half the effective capacity

(Nielsen and Schneider-Kamp, 2024). A 16-bit phase before ternary training is reported to beat ternary training from scratch (Nielsen et al., 2025), and ternary weights are reported to need a higher learning rate (Zhu et al., 2024; Kaushal et al., 2025). Parity results at matched parameter count measure a different quantity from my iso-memory penalty. My ternary penalty falls with budget, the direction expected if ternary eventually becomes competitive. A penalty that falls while the optimum stays fixed is also what scale-free effective-parameter laws imply with no crossover (Panferov et al., 2025a; Kumar et al., 2025; Maskey et al., 2026); this is my deduction from the form of the laws, and the law of Kumar et al. was fitted at 3 bits and above, so ternary lies outside it. Three budgets cannot tell the two apart.

Small language models and memory-constrained models. TinyStories (Eldan and Li, 2023) was introduced to show that language models below 10M parameters can write fluent, consistent stories. On it, Atome LM (Atome LM contributors, 2026) compares a ternary model with a 60K-parameter FP32 model (15 KB against 238 KB) and with a 6K-parameter FP32 model (15 KB against 23 KB). The models differ in architecture, and the headline comparison (3,000 steps) uses one seed; a three-seed, 1,500-step repeat in the same repository shows the same direction. My endpoints have the same direction as its 60K-parameter comparison (ternary ahead of bf16 at 828 Kbit and 4 Mbit), whereas in its own 944K equal-parameter comparison the FP32 baseline is ahead; an intermediate precision beats both of my endpoints. Veldanda (2026) runs a version of the 60K-parameter comparison of Atome LM under one fixed recipe with three seeds, on its own corpus and not with Atome LM's exact setup, and audits the 944K comparison without repeating it. It shows that the shape of a full-precision baseline can account for part of a reported ternary-hybrid margin. For CNNs trained from scratch with quantized activations, 4-bit weights need the least storage at 12% error on CIFAR-10, about 0.5M weights (my conversion of 2 Mbit) (Moons et al., 2017). With weights and activations quantized, 4-bit ResNet-50 models with the first and last layers at 8 bits give the best compute cost–accuracy Pareto curve on ImageNet, with a similar pattern for memory cost (Abdolrashidi et al., 2021). For networks fine-tuned from a full-precision initialization, 2-bit ResNet-34 and ResNet-50 give the best accuracy at a fixed model size (Esser et al., 2020). Memory-limited mixed-precision networks start from 8-bit tensors and cut individual layers to 4 or 2 bits only as far as the memory constraint requires (Rusci et al., 2020), and MCUNet fits larger 4-bit ImageNet models into microcontroller memory without a bit-width sweep (Lin et al., 2020). I found no prior study that compares three or more uniform weight precisions, ternary included, at matched weight storage for transformer language models trained with QAT from the first step below about 30M non-embedding parameters; my search had no systematic forward-citation pass. Small-model QAT exists at matched parameter count: Thomassen (2026) trains from initialization at INT4 down to about 2.7M parameters, and at FP16, INT8 and INT6 from about 5M, in a study of learning-rate schedules, and Nielsen and Schneider-Kamp (2024) compare ternary with 16-bit models at 6M to 48M parameters. My models are counted as total parameters including the tied embedding (under 3M); the prior-work 30M is non-embedding.

Quantizer design. My quantizer is assembled from standard parts (section 3.5). The location of the optimum depends on such choices. QuEST, which combines Hadamard normalization with MSE-optimal scales and a trust gradient estimator, places the Pareto-optimal precision for quantized weights and activations at about 4 bits (Panferov et al., 2025a), against the 7 to 8 bits reported by Kumar et al. (2025); the two setups differ in more than the quantizer, and in QuEST's own fit its learned-scale baseline (LSQ) also has the highest parameter efficiency per bit, $\mathrm{eff}(P)/P$, at 4 bits (my computation from its Table 2). Chen et al. (2026) attribute the 8-bit result mainly to the

per-tensor activation quantization used by Kumar et al. Below 4 bits the choice of quantization function is reported to be highly sensitive and to alter scaling-law outcomes (Liu et al., 2025). In the check reported with LSQ, the learned step sizes of a 2-bit ResNet-18 did not minimise quantization error (Esser et al., 2020). Weights collect at quantization decision boundaries and oscillate between levels during QAT (shown for convolutional networks, most strongly at 3–4 bits in depth-wise layers (Nagel et al., 2022); for ternary weights in LLaMA-3.2 models, reported as accumulation at the deadzone boundary and argued to trap weights in oscillation (Huang et al., 2026)). Uneven allocation of a bit budget has been reported to beat uniform allocation at similar model size or resource cost for CNNs on ImageNet (Wang et al., 2019; Yang and Jin, 2021). In the model-size comparison of Wang et al. (2019) the gain in top-1 accuracy is 8.7 to 19.5 points at 2 bits for the two MobileNets (1.7 for ResNet-50) and at most 0.6 points at 4 bits. By tensor-wise Fisher allocation it improves the scaled KL divergence to the original model after direct-cast quantization across 8 of 11 pretrained language models and different formats (Orr et al., 2025). A per-group allocation learned at 58M parameters and frozen into a fixed recipe, applied at 131M parameters and 1.82 mean bits, beats a 55M FP16 model in perplexity on TinyStories with 3.8 times less storage (Ogilvy, 2026); that is a comparison with FP16, not with a uniform allocation at the same average bits. My argmin b^* (the bit-width with the lowest mean loss; section 4) is the lowest-mean *uniform* precision, not the lowest-mean allocation of bits. My budget counts fixed-width storage, not the description length of the weights (Hinton and van Camp, 1993) or the code length of model plus data (Delétang et al., 2024).

Methodology. I report seeds, and statistical tests that take the variance across seeds into account, as argued for deep reinforcement learning (Colas et al., 2018) and for machine-learning benchmarks (Bouthillier et al., 2021). I use Welch tests (Welch, 1947), and equivalence bounds (Lakens, 2017) rather than non-rejection as evidence of a tie. I keep confirmatory and exploratory analyses apart (Herrmann et al., 2024), and I preregister, as argued for predictive modelling (Hofman et al., 2023) and done in a language-model study (Thrush et al., 2025). Small models are reported to be more sensitive to hyperparameters than larger ones (Lourie et al., 2026; Porian et al., 2024), and a learning-rate rule fitted at 59M–1B parameters overestimates the optimal rate by a median factor of 4.0 (range 2.4–6.6) for single-seed TinyStories models of 0.25M to 2M non-embedding parameters (Romanyukov et al., 2026). A7 measures learning-rate sensitivity because my rule is applied below the scales over which Porian et al. (2024) swept the learning rate (appendix B). Three budgets over a 16-fold range support local slopes, not a fitted scaling law. The survey of Li et al. (2025) says that fitting a scaling law needs models across multiple orders of magnitude, and reports that the papers that state their number of data points per fit use from 4 to several hundred, most fewer than 50. A power law fitted over a narrow range can differ from one fitted over the full range (Pearce and Song, 2024).

3 Experimental Design

I mark each choice below as *fixed*, *measured* or *assumed*. A choice marked fixed was set by the experiment specification, which predates all training (the version of 2026-09-27 is released in `timestamps/files/2026-09-27-f00fcd5/docs/SPEC.md`), or was recorded in the preregistration before the sweep's first run (in its body, in one of its declared deviations D1–D8, or in Amendments A1–A3; section 4 and appendices B and F). A measured choice was adopted only after the measurement cited. An assumed choice was not tested. A property marked *verified* was checked afterwards, on the finished runs or on the code, or by a computation of `scripts/paper_numbers.py`, whose source

is given beside each macro in `paper/generated/numbers.tex`.

3.1 Iso-memory constraint and bit accounting

Iso-memory comparison (section 1) requires every model at a budget B to store B weight bits, to within a fixed tolerance, whatever its b. A model with L blocks of width d, MLP ratio r and vocabulary $V = 512$ has

$$P = L\,(4 + 2r)\,d^2 + Vd \tag{1}$$

quantized parameters. Each block contributes its fused query–key–value, attention-output, MLP-up and MLP-down matrices, and the tied embedding–output matrix is counted once. The model stores

$$W = \lceil bP \rceil + 16\,(2L + 1)\,d \tag{2}$$

weight bits (fixed). The second term counts the RMSNorm scale parameters, which I call the norm gains, at 16 bits for every b: they are not quantized, but they are storage. They are held and applied in fp32 in training and in every reported evaluation (Deviation D6), so the model evaluated is not exactly the model counted. The weights have a second departure of the same kind (section 3.5). I measured the difference for the norm gains. Rounding every norm gain to bf16 changes the final-step validation loss by at most 4.1×10^{-4} nats over the final-step checkpoints of every complete run of the primary and shape-robustness sweeps (142 checkpoints; the A7, A8 and FineWeb-Edu runs were not re-evaluated; measured; `scripts/gain_precision_check.py`, `results/gain_bf16_check.json`). As a scale for seed noise I use the smallest seed standard deviation of `val_loss_best`, 0.0027 nats over the primary cells and 0.0014 nats over the shape-robustness cells. It belongs to a different quantity from the final-step loss compared here, and serves only as a scale.

The re-evaluation ran on device `cuda`, the device type that trained each run, and there the fp32-gain loss reproduces each run's logged final validation loss, the largest difference being 0 nats. The preregistered metric, `val_loss_best`, cannot be re-evaluated this way, because checkpoints hold the final-step weights.

Counted instead at the 32 bits in which the norm gains are held, 2 cells would leave the tolerance, 256 Kbit, $b = 1.58$ (TIGHTEST, +7.48%) and 828 Kbit, $b = 4$ (ASPECT, +5.07%), and the largest error would be +7.48% rather than 4.15%. The excess is $16(2L + 1)d$ bits, so it grows with depth times width and falls mostly on the deep low-b cells (table 14).

Computed from the shapes in table 14, counting the gains at 32 bits would add more bits to the grid neighbour just below b^* than to b^* itself at every budget, and fewer bits to the neighbour just above b^* at 256 Kbit and 828 Kbit. I did not measure its effect on the ranking.

Ternary weights are counted at the grid's 1.58 bits (fixed; Deviation D3). Recounted at the 1.6 bits of five-trits-per-byte packing, every ternary cell of both sweeps stays within tolerance, the largest error being +1.84% (verified; limitation L7). Every count is fixed-width storage, not an entropy-coded size. Measured on the final checkpoints of the 90 quantized primary runs (the empirical entropy of each matrix's integer codes, weighted by matrix size and averaged over runs; `scripts/clip_share.py`), the ternary codes carry 1.570 bits per weight against the 1.58 bits charged ($\log_2 3 = 1.585$), and the codes at $b = 2$, 3, 4, 6 and 8 carry 1.761, 2.694, 3.561, 5.227 and 6.927.

An order-0 entropy coder could store a cell's codes in H bits per weight, the empirical entropy of those codes, rather than the b bits charged, so at a fixed budget it would admit about b/H times as many quantized weights (the norm gains stay at 16 bits, and a per-matrix code table is not counted). What compares bit-widths is this ratio, not the difference $b - H$: the parameter increase $b/H - 1$ is 0.619% for ternary and 13.6%, 11.4%, 12.3%, 14.8% and 15.5% at $b = 2$, 3, 4, 6 and 8, smallest for

ternary and largest for $b = 8$. Fixed-width accounting therefore favours ternary relative to every other quantized bit-width: averaged over runs, an entropy coder would admit at least 11.4% more parameters for each of them but only 0.619% more for ternary. The $b = 16$ control has no integer codes and was not measured. I did not repeat the search at matched entropy.

Not counted are the per-matrix quantization scales, which at 32 bits each would add at most 0.84% to W (verified). Also not counted are the key–value cache (logged per run; section 7), activations, and optimizer state. A configuration is admissible only if $|W - B|/B \leq 5\%$ (fixed). This is asserted before training, and a miss raises an error. Every storage axis and table uses W, never B, except the power law of prediction P6 and the exploratory budget-equivalent ratio of the ternary cell (section 5.2), which are computed in B (the P6 slope is also given on W; table 14 and fig. 4). The largest deviation from B is 2.15% among primary cells and 4.15% over all runs.

3.2 Grid

The budgets are 256 Kbit, 828 Kbit and 4 Mbit, which are 262 144, 847 872 and 4 194 304 bits (Deviation D1). The bit-widths are $b \in \{1.58, 2, 3, 4, 6, 8, 16\}$, where 1.58 is ternary $\{-1, 0, +1\}$ and 16 is an unquantized bf16 control (both fixed). A *run* is one seed of a cell. Of the 7×3 cells, 20 are feasible.

At 256 Kbit and $b = 16$, the tied embedding alone, at the minimum width $d = 32$, costs $512 \times 32 \times 16 = 262\,144$ bits. That is 100.0% of the budget. Vocabulary and head dimension are held fixed, and the head dimension sets the minimum width of 32. I report this cell as the limit of the embedding-bound regime, in which the embedding holds more than half of W. No feasible cell enters that regime; the largest embedding share is 49.5%. Each feasible cell has 5 seeds (100 runs; fixed). The preregistration states that a Welch test then has roughly 80% power against a gap of 2.0 seed standard deviations (sections 3.9 and 4).

3.3 Architecture search and shape rules

The search is exhaustive over L from 2 to 24, d from 32 to 1024 in steps of 32, and $r \in \{2, 2.5, 3, 3.5, 4\}$, with heads of dimension 32 (fixed). Minimizing $|W - B|$, as the specification says, picks one configuration per cell except where exact ties occur, which they do in 1 of the 20 cells. It picks on budget accuracy alone, among the admissible configurations of different depth, width and MLP ratio, of which a cell has between 1 and 31. Which shape wins therefore depends on rounding, not on model quality, and the winning shape can vary with b, the factor under study. The rule is therefore named and fixed in advance (Deviation D8). The rule TIGHTEST minimizes $|W - B|$ and breaks the rare exact ties by fewest layers, then narrowest width, then smallest r. It defines the primary sweep. The rule ASPECT considers every configuration within tolerance, picks the one whose aspect ratio d/L is nearest a target of 10, fixed from the search space alone, and breaks ties by the smallest $|W - B|$.

ASPECT defines the preregistered shape-robustness sweep, which was declared secondary and cannot override the primary argmin (section 4). It covers every bit-width at 828 Kbit and 4 Mbit with 3 seeds (42 runs), and at those budgets it changes the shape of 5 of 7 and 6 of 7 cells (table 4). At 256 Kbit the rules select different shapes in 1 of 6 cells, because the tolerance leaves little freedom of shape there. It fixes the aspect ratio outright at $b = 6$ and $b = 8$ (10.7 and 16), and allows only the narrow ranges 4 to 5.33 at $b = 3$ and 5.33 to 8 at $b = 4$, which overlap only at 5.33. At $b = 2$ the admissible aspect ratios span 2.46 to 32, but the configuration nearest the target is also the tightest. At $b = 1.58$ they span 1.88 to 21.3 and the rules differ, but 256 Kbit was excluded from that sweep by design (preregistration §5.2). The sweep therefore tests shape at 828 Kbit and 4 Mbit

only. At 256 Kbit, including its ternary cell, the effect of shape is untested (section 7).

3.4 Model

The model is a decoder-only, pre-norm transformer in the style of GPT-2 (Radford et al., 2019), with context 256. Each block has two residual sublayers. The first is RMSNorm (Zhang and Sennrich, 2019) followed by causal self-attention with rotary embeddings of base 10 000 (Su et al., 2024). The second is RMSNorm followed by a two-matrix MLP with a GELU nonlinearity. A final RMSNorm precedes the output head, which is tied to the embedding. There are no biases and no dropout (fixed). The quantized weights are initialized from $\mathcal{N}(0,\, 0.02^2)$, except that the attention-output and MLP-down matrices use a standard deviation of $0.02/\sqrt{2L}$, which scales the residual-path weights by the inverse square root of the number of residual layers as described for GPT-2 (Radford et al., 2019). Each run begins by asserting that the model holds exactly P quantized parameters, $(2L+1)d$ norm gains, and nothing else.

3.5 Quantizer

Every attention and MLP matrix, and the tied embedding, keeps fp32 latent weights (Courbariaux et al., 2015). These are quantized in the forward pass at every step (Jacob et al., 2018), from step 0 (fixed), with a per-tensor scale derived from $\mu = \text{mean}\,|w|$ at every bit-width. This extends the mean-absolute-value scale of XNOR-Net, which is per filter at 1 bit (Rastegari et al., 2016); ternary weight networks set their ternary threshold from the same statistic, at about $0.7\,\mu$ (Li et al., 2016). Ternary follows the absmean quantizer of BitNet b1.58 (Ma et al., 2024), without the small constant they add to the denominator and with the scale μ applied to give the dequantized weight: $Q(w) = \mu\ \text{clip}(\text{round}(w/\mu), -1, 1)$. With its step at μ, this rule has a higher mean squared error on a Gaussian source than the L2-optimal ternary quantizer, which has the same form with a larger step (on a unit Gaussian, the optimal step is $1.534\,\mu$ and the step at μ has 38.5% more error; verified numerically). I argue that the difference is a choice of step and not of capacity: the latent weights can take any values under QAT, so both steps reach the same set of nonzero ternary tensors (the all-zero tensor is reached by neither). To see this, take a ternary tensor with $k \geq 1$ nonzero entries among n, and latents of one magnitude a on those k positions, with the required signs, and zero elsewhere. Then $\mu = ka/n$, and each nonzero latent equals $n/(ks)$ steps for a step of $s\mu$, which is at least $1/s > 1/2$ for $s = 1$ and for $s = 1.534$, so it rounds to ± 1 under either step. This is an argument, not a test, and I did not train with the optimal step. For $b \in \{2, 3, 4, 6, 8\}$,

$$Q(w) = \Delta\ \text{clip}(\text{round}(w/\Delta),\, -2^{b-1},\, 2^{b-1} - 1), \qquad \Delta = c_{2^b}\sqrt{\pi/2}\ \mu, \tag{3}$$

where $\sqrt{\pi/2}\,\mu$ estimates a Gaussian's standard deviation. c_N is the MSE-optimal step of an N-level symmetric uniform quantizer of a unit Gaussian, computed by the criterion of Max (1960), not fitted to my data. The grid of (3) is symmetric signed in the terminology of Nagel et al. (2021): it has no zero-point and takes the two's-complement range, so it has a level at zero and one more negative level than positive ones; at $b = 2$ its levels are $\{-2, -1, 0, 1\}\Delta$, where the symmetric 4-level quantizer of Max has none at zero. Whether this difference between the two alphabets helps or hurts under QAT is not known, and I did not test the symmetric alternative. The constants c_N are optimal for Max's symmetric quantizer and not for the grid of (3): on that grid, c_N gives a Gaussian MSE at most 0.61% above the grid's own minimum, at $b = 2$ (computed for a unit Gaussian; verified). The scale is constant in the backward pass, and rounding is bypassed with the straight-through estimator (Bengio et al., 2013). At $b = 16$ the latents are simply cast to bf16. Only weights are quantized; activations stay in bf16 (fixed).

Because the activations are bf16, the dequantized weight $q\Delta$, formed in fp32, is rounded to bf16 where it enters each matrix product and the embedding lookup, in training and in every evaluation, so the weight applied is $\mathrm{bf16}(q\Delta)$. A bf16 number has an 8-bit significand, so this moves a weight by at most $|q|/256$ of a step, which is at most 0.125 of a step at $b = 6$ and less than 0.5 at $b = 8$. It therefore never changes a weight's integer code and keeps its 2^b levels distinct, and a stored code and scale still determine the applied weight.

I measured this on the 7 final checkpoints held off the training host (seed 0, 828 Kbit $b \in \{1.58, 2, 3, 4, 6, 8\}$ and 4 Mbit $b = 1.58$; evaluated on CPU; `scripts/bf16_cast_check.py`). The largest displacement was 0.107 of a step at $b = 6$ and 0.414 at $b = 8$, and no code changed. The squared error of the unclipped weights against their latents, with each tensor's error in units of its own step, rose by +0.12% at $b = 6$ and +4.0% at $b = 8$; pooled in absolute units the change is −0.38% at $b = 6$ and +4.5% at $b = 8$. Applying the exact dequantized weights in the matrix products instead (the embedding lookup still rounds each row to bf16 as an activation in every arm of the check) changed the final-step validation loss of the 7 checkpoints by at most 3.2×10^{-4} nats, on CPU. For comparison, the loss logged at the end of training on the training host differs from the CPU loss of the same weights by up to 2.9×10^{-4} nats, which is of the same size, so I do not separate the effect of the cast on the loss from numerical noise. Both are smaller than the seed-noise scale of 0.0027 nats defined above. I did not repeat the check on the other runs.

The specification prescribes absmean scaling only for ternary. I use it at every width because of a unit test with one 100× outlier (measured; Deviation D4). In that test absmax zeroed almost every weight at $b = 1.58$, 2 and 4, while absmean did not, so absmax could degrade differently along the axis under study (assumed: the test uses a planted outlier, and no run used absmax). A check every 100 steps aborts a run if any quantized tensor has 5% or fewer nonzero entries; it aborted 0 runs.

3.6 Training protocol

All runs use AdamW (Loshchilov and Hutter, 2019) with $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices only (Deviation D7), and gradient clipping at 1.0 (fixed). The peak learning rate is one function of width, the same for every sweep run and never tuned (the sensitivity runs of Amendments A7 and A8 apply a multiplier to it and are separate from the sweeps; section 5.8):

$$\eta(d) = 0.003 \cdot (64/d)^{1/2} \tag{4}$$

(fixed). It warms up linearly over 2% of the steps, then decays by cosine to 10% of the peak, a single cycle of the schedule of Loshchilov and Hutter (2017) without restarts. Each step uses 32 768 tokens, in 128 windows of 256 tokens sampled uniformly from the training stream. Every completed run used this batch without gradient accumulation (verified from the `micro_batch` and `grad_accum` columns of `results.csv`). The seed sets both the initialization and the data order.

Early stopping as specified never fired in the 3 pilot runs, one per budget, each a single seed of a grid cell that was later retrained under the final protocol and included in the sweep (section 4). Their best validation loss came at 96% to 100% of training, and truncating a cosine schedule would leave the learning rate high (Amendment A3). Every run therefore completes its schedule, and the schedule length is set per cell. A run counts as undertrained if it received fewer than 99.9% of its token budget, recomputed from logged token counts (the budget is rounded down to whole optimizer steps, so a run falls short of it by less than one step); 0 of the 142 complete runs are undertrained. The `undertrained` column of `results.csv` is not used (appendix C). This definition replaced the preregistered one after the sweep began (appendix F).

3.7 Token budgets

A lower b buys more parameters, so a common token budget would give low-b cells the fewest tokens per parameter, and the comparison would measure training speed rather than capacity. Amendment A3, made before any sweep run, instead sets each cell's token budget T so that every cell is predicted to finish the same distance from convergence.

I calibrated on off-grid shapes only. Each shape was trained at successive doublings of T, each run a complete schedule, and the *calibration gain* $g(N, T)$ is the drop in best validation loss from $T/2$ to T at $N = P$ parameters. At matched tokens per parameter (98 to 394), the ratio ρ of successive calibration gains was 0.713, 0.620 and 0.653, from the smallest calibration size to the largest; I use $\rho = 0.65$, the largest size's value rounded (Amendment A3). The calibration gains fit $g = CN^{\alpha_N}T^{\alpha_T}$ with $\alpha_N = 0.179$ and $\alpha_T = -0.793$ (8 calibration gains, 88 064 to 3 047 424 parameters, mean absolute log residual 0.106), so the calibration gain depends mainly on tokens and only weakly on parameter count in this fit (see the confound below).

The fit rests on 12 calibration runs at 3 model sizes, each trained at a single bit-width ($b = 3$, 2 and 1.58 from the smallest size to the largest), so in the fit model size and precision are confounded, and a dependence of the token exponent on b cannot be separated from one on size. If ρ persists, the improvement left after T is $R = g\rho/(1-\rho)$. The fit's token exponent alone implies $\rho = 2^{\alpha_T} = 0.577$, below the directly measured 0.65. Using it would lower every cell's predicted residual by the same factor, to 0.0147 nats, so the predicted residuals would remain equal across cells (algebra from the definition of R; I did not compute whether the validation-loss ranking would change). Setting R to a target $R^\star$ gives

$$T(N) = \min\left\{ \left[\frac{R^\star(1-\rho)}{\rho C N^{\alpha_N}} \right]^{1/\alpha_T},\ 4\,U \right\}, \tag{5}$$

where $U = 805\,873\,616$ is the number of unique training tokens, and C is pinned to the largest calibration point ($N = 3\,047\,424$, $T = 1\,200\,000\,000$, $g = 0.0242$).

I also measured equal tokens per parameter and rejected it. At matched ratios it left a predicted residual far larger for the smallest calibration model than for the largest (0.547, 0.105 and 0.045 nats, each from that model's own ρ and unrounded gains; the preregistration records the last as 0.046), which on the grid would bias the comparison against the small, high-precision cells.

$R^\star = 0.020$ nats is the tightest residual that every cell is predicted to reach within 4 epochs of repeated data. The cap is assumed from the data-constrained scaling literature, which reports that up to four epochs of repetition are nearly free, at 2.8B to 8.7B parameters, and fits its scaling law on models of 7M to 9B parameters (Muennighoff et al., 2023); it is not measured at my sizes. Here an epoch is the number of training tokens seen divided by the number of unique training tokens, with window starts drawn uniformly with replacement, so a token is covered a random number of times, not once per epoch. The largest primary cell (4 Mbit, $b = 1.58$, 2 616 320 parameters) trains for 3.99 epochs (3 218 440 192 tokens). The smallest (256 Kbit, $b = 8$, 32 768 parameters) trains for 1.49 epochs (1 197 441 024 tokens; table 14). No cell is capped, and a tighter target would cap the largest cells but not the smallest. The comparison is therefore iso-memory and, by prediction, iso-convergence, but not iso-compute. Counted as 6×parameters×tokens (a FLOP proxy that omits the attention-score cost), a run of the most expensive cell at a budget costs 16.8 times one of the cheapest at 4 Mbit ($b = 1.58$ against $b = 16$), 16.5 times at 828 Kbit and 6.8 times at 256 Kbit ($b = 1.58$ against $b = 8$; table 14). This compute difference gives the ternary cell the most compute at each budget, and it nonetheless loses to b^* at every budget (section 5.2).

The calibration gains, ρ, the fit and the seed noise are measured. The seed noise is 0.0075 nats on an off-grid model of 88 064 parameters. That the grid cells share one residual is a *prediction*. The predicted residual is logged per run, but the realized residual was not re-measured per cell on

this corpus.

The prediction assumes that the fit transfers to grid shapes, and that ρ holds at the grid's 1 230 to 36 543 tokens per parameter, well above the matched calibration range. By their token budgets T, 19 of the 20 primary cells lie beyond the range in which the model was calibrated. The one calibration pair at higher ratios (1 136 to 4 542) decayed faster, with $\rho = 0.370$. If ρ falls as tokens per parameter rise, the high-b cells of each budget, which have the most tokens per parameter, finish closer to convergence than predicted. That would bias the comparison toward high b by at most $R^\star = 0.020$ nats, provided that the residual of the lower-b cells is not larger than predicted; the one calibration pair beyond the matched range, whose ρ is lower, is consistent with that proviso, and no other point tests it. Such a bias favours the argmin, $b = 6$ at every budget, over its lower neighbour $b = 4$, and works against it relative to $b = 8$. The bound exceeds the gap to $b = 4$ at 4 Mbit (0.0061) but not at 828 Kbit (0.1101) or 256 Kbit (0.1174). It is a bound, not a measured bias. If the prediction holds, every loss sits 0.020 nats above convergence, and the offset cancels within each budget (section 7). Amendment A7 measures the spread of residuals on TinyStories, through the gain model, by retraining the cells around each argmin at half their token budget (section 5.8), and Amendment A6 preregisters the same check for the second corpus (section 6).

3.8 Data

The corpus is TinyStories (Eldan and Li, 2023), whose train split (`roneneldan/TinyStories`) serves as the document pool. A seeded permutation (seed 1 337) divides the documents 98/1/1 into training, validation and test sets. A byte-level BPE tokenizer (Sennrich et al., 2016; Radford et al., 2019) with 512 tokens is trained on the training split only, which yields 805 873 616 training tokens. The SHA-256 hashes of the tokenizer and of every split are committed, and the hash of each token stream is asserted at each load (fixed).

3.9 Evaluation

Validation loss is the mean next-token cross-entropy, in nats per token, of the quantized model. It is measured on the same 50 batches of 128 windows of 256 tokens, every 250 steps and at the last step, identically for every run. A run reports `val_loss_best`, the minimum over its evaluations, which is the preregistered primary metric (section 4). A cell reports the mean of `val_loss_best` over its seeds, with its seed standard deviation. Because it is a minimum over many noisy evaluations scored on one fixed validation set, it is biased downward, and more so at low b, whose evaluation-to-evaluation fluctuations are larger (section 5.5). The final evaluation involves no selection; under it, every budget's argmin is unchanged (appendix E).

A bigram model with add-k smoothing ($k = 0.1$), counted on the training split, reaches 3.1648 nats on validation (perplexity 23.68). Any run that does not beat this floor is flagged (fixed); 142 of the 142 complete runs beat it. The test split is sealed behind one guarded loader, and a gating test asserts that training never reads it. It was read exactly once, after all training had finished and every validation verdict in this paper was final (`scripts/test_read.py`, log `results/test_read.json`, whose record of the test stream's hash equals the committed one). Because checkpoints hold the final-step weights, the read measures final-step loss, not `val_loss_best`. The analysis was fixed in a dated note in the preregistration (its §14, 2026-09-27, commit `d74768d`) before the test split was opened. It comprises, per cell, the mean and seed standard deviation of the final-step test loss; per budget, the null criterion of section 4 on those values; and Welch tests of the test-loss argmin against each adjacent b, with the power rule. Amendment A7 (§16) extended the same read, with the same analysis, to each primary run's averaged weights (variant (c) of Amendment A7, whose

loss is val_c; section 5.8). The validation verdicts remain the preregistered primary result; the test results are reported beside them (section 5.6).

4 Preregistration and Statistical Protocol

The predictions, the null criterion, the run plan and the list of planned analyses were written into a preregistration document (`PREREGISTRATION.md`, released with the code) and committed to the project's git repository on 2026-09-13, when, as the document records, no model at a grid configuration had been trained beyond a short smoke test. That repository was local: the preregistration and its amendments A1–A6 were not pushed to a public remote or deposited elsewhere when they were made, so their only timestamps are git commit dates, which I control. The first external timestamps (OpenTimestamps, which anchor a file hash in the Bitcoin blockchain) were made on 2026-09-27 at 04:54 UTC, for Amendment A7 and the correction to A4, and at 06:24 UTC, for the note of preregistration §17 that fixes two details of A7's first measurement, the end-of-training averaging check M1 (defined under *Amendments* below), before it ran. Each stamp covers a hash of the whole preregistration at the stamped commit, so the first also covers P1–P5 as committed. Further stamps followed on 2026-10-02 (21:52 UTC), for the note fixing A7's remaining choices, and on 2026-10-09 (15:55 and 17:01 UTC), for Amendment A8 and its note, all after the TinyStories sweep had finished. The preregistration was also not reviewed by a third party before the results, unlike the proposals at the NeurIPS 2020 Preregistration Workshop, which were reviewed before any experiment was run (Bertinetto et al., 2021); it is self-registered.

Every results row records the commit hash of the code that completed its run. 40 of the 142 complete rows carry a `-dirty` suffix, because their working tree held uncommitted changes. The code provenance, including runs trained under more than one code state, is in appendix C. Later changes to the document are dated amendments (appendix B). Decisions taken after the sweep began that the document does not contain are listed in appendix F; the two that bear on the results, the treatment of the pilot cells and the standard deviation used by the power threshold, are described below. Predictions P1–P5 and the null criterion read as they did before the first run; P6 and P7 were added later by Amendment A4, and P8–P10 by Amendment A6.

The document separates planned from exploratory analyses (preregistration §6 and §7) and requires anything not on the planned list to be labelled exploratory. In this paper the following are exploratory and labelled so where they appear: the ternary-penalty trend beyond the scoring of P6 and its budget-equivalent form (section 5.2); the selection-bias analysis (section 5.5 and appendix E); the comparison of shape effects with the gaps between bit-widths and the storage-adjusted gaps (section 5.4); the Welch tests against non-adjacent bit-widths, the Welch intervals, the equivalence tests, the permutation tests, the Holm correction and the seed bootstrap (table 2); and every analysis variant in table 6 other than the two readings of the pilot clause. The document also declares the shape-robustness sweep under the ASPECT rule secondary (section 3): that sweep can qualify the primary argmin but never replace it.

Predictions. Five predictions were recorded before any result (table 11). P1: validation loss is U-shaped in b, with its minimum at intermediate precision, $2 \le b \le 4$. P2: the optimum moves to lower b as the budget grows, to $b^* = 4$, 3 and 2 at 256 Kbit, 828 Kbit and 4 Mbit. P3: the argmin at matched parameter count differs from the argmin at matched bits. P4: every feasible cell beats the bigram floor of 3.1648 nats. P5: the seed standard deviation is small relative to adjacent-b gaps at 4 Mbit but may not be at 256 Kbit, where the null criterion is most at risk of firing. Amendment A4 added two predictions on 2026-09-16, after the sweep had begun; they are not part of the original

preregistration, and the data that existed when they were recorded are given under *Amendments* below. P6: the ternary penalty at 4 Mbit will be about 0.063 nats (recorded to three decimals; the unrounded value is 0.0632), the value of the power law in B through the first two budgets (exponent -0.753), and in any case below 0.12. P7: the argmin stays at $b = 6$ at 4 Mbit, contrary to P2. The second corpus has its own predictions, P8–P10 (section 6). All verdicts are given in section 5.

Null criterion. Fix a budget. For each bit-width b, let $m(b)$ be the mean over seeds of the best validation loss, let $s(b)$ be its seed standard deviation, and let $b^* = \arg\min_b m(b)$. Let $\mathcal{N}$ be the one or two bit-widths adjacent to b^* in the grid. The preregistered criterion is

$$\text{gap} = \min_{b \in \mathcal{N}} |m(b) - m(b^*)|, \qquad s_{\max} = \max_{b \in \{b^*\} \cup \mathcal{N}} s(b), \qquad \text{null} \iff \text{gap} \le s_{\max}. \tag{6}$$

A null budget is reported as null in the abstract as well as the body, and its argmin is not called the best bit-width anywhere in the paper. Every claimed gap can be read against the seed standard deviations of the cells compared, which table 1 gives for each cell. The preregistration also requires the abstract to state when the ± 1 seed standard deviation intervals of the compared cells overlap. I call a budget's argmin *decisive* if the null criterion does not fire, the argmin differs from both adjacent bit-widths in the Welch test at $\alpha = 0.05$ (defined below), and its gap to the nearest one is above the power threshold defined below. The term is mine, not the preregistration's.

Welch tests and power. As a secondary test, the preregistration specifies a two-sided Welch t-test between b^* and each adjacent bit-width at $\alpha = 0.05$. The p-values are reported whatever their size (values below 10^{-4} are printed as $p < 10^{-4}$, as described below), and an argmin that is not significant against a neighbour is not described as a finding. Unless stated otherwise, "significant" means $p < \alpha$ at this level; Amendment A7 uses its own cutoff, $p > 0.01$, for its seed-extension rule (check M4, below). No correction for multiple comparisons was preregistered, so I report each adjacent test uncorrected first. The claim that b^* is better than both of its neighbours is an intersection–union test: it holds at level α only if each adjacent test rejects at α, so its level is controlled without a correction for having two components, with b^* treated as fixed; b^* is in fact the observed argmin, so the nominal level is not guaranteed after that selection. A claim about several budgets at once would need a correction. As a sensitivity analysis that was not preregistered, I apply a Holm correction to the family of the 6 adjacent tests across the three budgets; it rejects 4 (section 5.1). Table 2 also tests b^* against the non-adjacent bit-widths; those tests are exploratory. With 5 seeds per cell the smallest two-sided p an exact permutation test can reach is $2/252 = 0.0079$, so a Welch p far below that is a t-distribution approximation that rests on normality, not on the exact permutation distribution; I print such values as $p < 10^{-4}$ and give the exact permutation p beside each adjacent test. The effect-size statement for each budget is the Welch 95% interval on the gap between b^* and its nearest neighbour (table 2). A test that does not reject is not evidence that two cells are equal (Lakens, 2017): where I speak of a tie I give the two one-sided tests' 90% interval and its bound (exploratory), and where neither a difference nor an equivalence is shown I call the comparison unresolved at $n = 5$ and give its minimum detectable effect, with the significance level that it assumes. For the ten-seed comparison of Amendment A7 (check M4, defined under *Amendments* below), "unresolved" keeps the preregistered meaning, Welch $p > 0.01$ at the cap of ten seeds; the two one-sided tests reported beside it are exploratory, use a margin chosen after those data existed, and place the difference within ± 0.01 nats (section 5.8).

The preregistration states that with 5 seeds a Welch test has roughly 80% power against a true gap of about 2.0 standard deviations. That figure was stated, not measured. A simulation run afterwards (20 000 draws of two normal samples of 5 with a true gap of 2.0 standard deviations,

tested with the analysis's Welch test) gives a power of 77%. The preregistration commits to calling smaller gaps unresolved rather than reporting them as trends. The analysis applies this as a descriptive threshold: a gap is *below the power threshold* if it is smaller than 2.0 times the larger seed standard deviation of the two cells compared. Such a gap can still reach $p < 0.05$; it is then reported as significant but below the power threshold. The preregistration's §4 does not say which standard deviation to use. Its §14 note, written after the sweep, says the larger one, for the test analysis only. The rule of taking the larger one was first written into the analysis code on 2026-09-16, after the 256 Kbit results and some of the 828 Kbit results had been committed and after the first 4 Mbit ternary run had finished (appendix F).

The one permitted adjustment, and two readings of the pilot clause. The preregistration let exactly one parameter move after commitment (its §5.4): if a convergence pilot on held-out cells showed that early stopping did not trigger reliably before the per-budget token caps, the caps could be changed once, before the main sweep. The pilot trained one seed of each of three cells, $b = 3$ at 256 Kbit and at 828 Kbit and $b = 6$ at 4 Mbit, and early stopping never fired. The adjustment was used once, as Amendment A3, which did more than change the caps: it removed early stopping, and with it the preregistered definition of an undertrained run. The replacement definition was fixed after the sweep began (appendix F); under it, 0 of the 142 complete runs are undertrained.

The same clause says that "the pilot cell is excluded from the headline curve", and Amendment A2 extended that to all three pilot cells. The run plan (preregistration §5.1) trains every feasible cell with 5 seeds, the pilot cells among them. The preregistration is therefore internally inconsistent: each reading contradicts one of its clauses. The pilot runs themselves, trained under the superseded protocol, contribute to no table or figure. The three cells were retrained under the final protocol with all 5 seeds, and I report the analysis both with and without them. The choice matters only at 4 Mbit, where the pilot cell is the argmin: with it, the lowest mean is at $b = 6$ (1.3286); without it, at $b = 4$ (1.3346), where the null criterion returns not null, with the gap above the power threshold. With $b = 6$ absent, the neighbours of $b = 4$ are $b = 3$ and $b = 8$, and the nearest is $b = 8$ (section 5.1 and tables 2 and 6). Tables and figures show the cells with the pilot cells included.

Amendments. Eight amendments were made, each dated in the document with its reason; appendix B gives each one's commit, the data that existed when it was made, and its content.

A1 (2026-09-13) removed the compute ceiling from the run plan, before the pilot and before any sweep run.

A2 (2026-09-13) widened the convergence pilot from one held-out cell to one per budget.

A3 (2026-09-14) replaced early stopping with complete cosine schedules and set each cell's token budget from a gain model fitted on off-grid runs (section 3). It was triggered by the pilot and made before any sweep run but after the three pilot runs.

A4 (2026-09-16) recorded P6 and P7 after two budgets had been scored. When it was committed, at least one 4 Mbit ternary sweep run had finished on the training host, several hours earlier. A second finished within about a minute of the commit, and I cannot say whether it finished before the commit. The pilot run of $b = 6$ at 4 Mbit, which P7 predicts to be the argmin, had also reached a best validation loss of 1.3670 nats. A4 itself states that no 4 Mbit run had started, which the sweep log contradicts (appendix B). A dated correction in the preregistration (its §15, 2026-09-27) records that this statement was false and that P6, which concerns exactly the 4 Mbit ternary cell, is therefore post hoc. P7 concerns the 4 Mbit argmin over all bit-widths;

apart from the ternary runs above and the pilot run, no 4 Mbit run had started when A4 was committed.

A5 (2026-09-16) added limitations L9 and L10 after results at two budgets existed. It changes no prediction, criterion or analysis (section 7).

A6 (2026-09-26) added the FineWeb-Edu arm and P8–P10 after the TinyStories sweep had finished and before any run of the arm (section 6).

A7 (2026-09-27) added checks of the training recipe after the TinyStories sweep had finished and before any of its own runs: an end-of-training averaging continuation of every run (M1), and, for the cells around each argmin, learning-rate sensitivity (M2), a half-budget residual check (M3) and extra seeds (M4), with falsifiers fixed in advance (section 5.8). Two dated notes (its §17 and §18) fixed details of A7 before the corresponding runs. A separate dated note, recorded just before A7 (its §14), fixed what the single test-set read measures (the final-step weights) and how it is analysed, before the test split was opened, and A7 (its §16) extended that read to each primary run's averaged weights.

A8 (2026-10-09) extended A7's learning-rate search upward until each cell's optimum was bracketed, with a cap, and re-applied the M2 falsifier unchanged. It was recorded after most M2 runs had been read and before any of its own runs, and its note (§20) fixed its open choices before its first run.

A7 and A8 change no prediction, criterion or primary verdict; they add sensitivity analyses reported beside them. A6, A7 and A8 preceded their own data but not the results that motivated them.

5 Results

Every run planned for the primary sweep (100 runs: 20 cells, 5 seeds each) and for the shape-robustness sweep (42 runs) completed. 0 of them are undertrained (section 3.6), and 1 primary run, 828 Kbit, $b = 3$, seed 0, failed once and was rerun (appendix C). Unless stated otherwise, a loss in this section is the preregistered primary metric `val_loss_best` (section 3.9) in nats per token, averaged over the seeds of a cell and reported with the seed standard deviation beside it. Analyses that are not on the preregistration's list of planned analyses are labelled exploratory. Gaps and differences are computed from unrounded means, so they can differ in the last digit from the difference of the printed means. All numbers come from the validation split, except those of section 5.6, which reports the one read of the test split (section 4).

5.1 The iso-memory curve

Figure 1 and table 1 show validation loss against b at each budget. At every budget the lowest mean lies at an interior bit-width, $b^* = 6/6/6$ at 256 Kbit/828 Kbit/4 Mbit. The grid is coarse: $b = 5$ and $b = 7$ were not trained, so $b^* = 6$ locates the minimum of the grid only to between $b = 4$ and $b = 8$. The lowest cell mean falls with budget, from 2.3315 to 1.8580 to 1.3286. Both ends of the grid lose. At 828 Kbit and 4 Mbit the bf16 control is the worst feasible cell (2.1294 and 1.5250), and ternary is worse than b^* at every budget (section 5.2). The curves are not monotone on the low-precision side. At 828 Kbit, $b = 3$ (1.9292) lies below $b = 4$ (1.9680), and at 4 Mbit, $b = 1.58$ (1.4127) lies below $b = 2$ (1.4705). At 256 Kbit, $b = 1.58$ and $b = 2$ are within one seed standard deviation of each other. The 256 Kbit, $b = 16$ cell does not exist, because the tied embedding alone takes 100.0% of that budget. The largest embedding share among feasible cells is 49.5%, at 256 Kbit

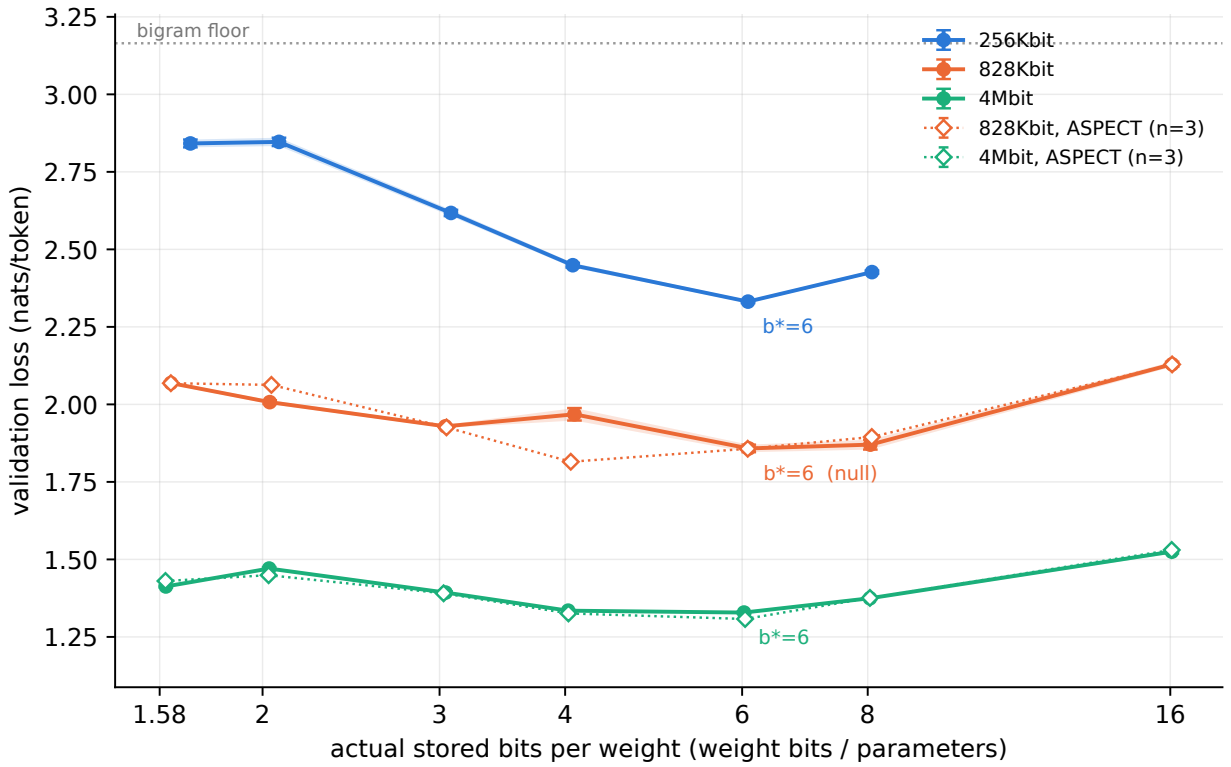


Figure 1: **The lowest mean is at an interior bit-width at every budget, and the preregistered criterion classifies it differently at each.** Validation loss (`val_loss_best`, nats per token) against actual stored bits per weight (stored weight bits divided by parameters), one line per weight-storage budget. Ticks mark the grid values of b; each point sits at its cell's actual value, which is slightly larger (table 14). Filled points are primary-sweep cell means over $n = 5$ seeds; shaded bands and error bars are ± 1 seed standard deviation and are mostly narrower than the markers. Hollow diamonds on dotted lines are the ASPECT shape-robustness cells at 828 Kbit and 4 Mbit ($n = 3$), which show how far a change of depth and width moves each cell (section 5.4). The lowest mean is at $b^* = 6/6/6$ (256 Kbit/828 Kbit/4 Mbit). This is decisive at 256 Kbit and null at 828 Kbit (label "null", so the lowest mean is not claimed as the optimum there). At 4 Mbit it is not null, but the gap is below the power threshold, and the $b = 6$ cell there is a convergence-pilot cell, without which the minimum is at $b = 4$ (section 5.1). The grey horizontal line is the bigram floor (3.1648). No cell exceeds the preregistered embedding-bound threshold of 50% of its stored bits; the largest share is 49.5% (256 Kbit, $b = 8$). The 256 Kbit, $b = 16$ cell is infeasible. Every cell's stored weight bits are within 5% of its budget (fig. 4 and table 14).

and $b = 8$, below the preregistered embedding-bound threshold of 50%. The number of feasible cells in the embedding-bound regime (embedding share above that threshold) is 0; only the infeasible cell reaches it.

Table 2 applies the preregistered null criterion (section 4) to each budget. It compares the gap between b^* and its nearest adjacent bit-width with $s_{\max}$, the largest seed standard deviation over b^* and its neighbours. The table also gives the power threshold (section 4) and two-sided Welch p-values of b^* against every other bit-width. Only the tests against adjacent bit-widths were preregistered. The others, quoted below and in section 5.2, are exploratory and uncorrected for

Table 1: Validation loss (`val_loss_best`, nats per token) at each weight-storage budget and bit-width b: mean over $n = 5$ seeds, with ±1 sample standard deviation beneath. Every cell's stored weight bits are within the search tolerance of its budget (table 14). Bold marks the lowest mean at each budget; ‡ marks a budget where the preregistered null criterion fires (gap to the nearest adjacent $b \leq$ the largest seed sd over the triple), so its lowest mean is not claimed as the optimum. † Infeasible: at 256 Kbit and $b = 16$ the tied embedding alone fills the budget.

	bits per weight b						
Budget	1.58	2	3	4	6	8	16
256 Kbit	2.8416 ±0.0127	2.8467 ±0.0135	2.6175 ±0.0101	2.4489 ±0.0076	**2.3315** ±0.0048	2.4267 ±0.0055	infeasible†
828 Kbit‡	2.0688 ±0.0051	2.0073 ±0.0062	1.9292 ±0.0050	1.9680 ±0.0201	**1.8580** ±0.0122	1.8702 ±0.0160	2.1294 ±0.0081
4 Mbit	1.4127 ±0.0034	1.4705 ±0.0047	1.3931 ±0.0041	1.3346 ±0.0027	**1.3286** ±0.0043	1.3747 ±0.0043	1.5250 ±0.0060

multiple comparisons. The three budgets receive three different verdicts.

At 256 Kbit the nearest adjacent bit-width is $b = 8$. The gap is 0.0952, 12.53 times $s_{\max} = 0.0076$ and above the power threshold of 0.0110. The Welch 95% interval on the gap is 0.0876 to 0.1027, and the Welch test gives $p < 10^{-4}$; the exact permutation test gives $p = 0.0079$, the smallest value five seeds allow. Every other bit-width is separated from b^* at $p < 10^{-4}$ (exploratory, apart from the adjacent $b = 4$ and $b = 8$). The criterion returns not null, with the gap above the power threshold. This is the only budget at which the argmin is decisive (section 4; the choice of which seed standard deviation enters the power threshold was made after the 256 Kbit results, some of the 828 Kbit results and the first 4 Mbit ternary run existed, appendix F). In the seed bootstrap, $b = 6$ has the lowest mean in 100.0% of draws. At this budget every cell has width 32 and depth falls as b rises, so the decisive result concerns precision and depth varied together (section 7).

At 828 Kbit the nearest adjacent bit-width is $b = 8$. The gap of 0.0123 is smaller than $s_{\max} = 0.0201$ (ratio 0.61), so the criterion returns **null**. The gap is also below the power threshold of 0.0320, its Welch 95% interval, -0.0087 to 0.0332, includes zero, the ±1 seed-sd intervals of the two cells overlap, and the Welch test gives $p = 0.212$ (permutation $p = 0.2698$). As preregistered, I do not describe $b = 6$ as the optimum at this budget. In the seed bootstrap $b = 6$ has the lowest mean in 94.6% of draws; when each draw also picks either shape rule's runs for every cell, that falls to 48.8%, against 49.8% for $b = 4$ (exploratory; table 2). The data do not distinguish $b = 6$ from $b = 8$, and under the alternative shape rule the lowest mean moves to $b = 4$ (section 5.4). Under the TIGHTEST shapes, Welch tests separate $b = 6$ from every bit-width other than $b = 8$ at $p < 10^{-4}$ on `val_loss_best`. Only the tests against $b = 4$ and $b = 8$ were preregistered, and none is corrected for multiple comparisons. For example, $b = 6$ is lower than $b = 3$ by 0.0712 nats. On the averaged metric of Amendment A7, $b = 3$ is not separated from $b = 6$ even at 10 seeds per cell (Welch $p = 0.832$; section 5.8). Under ASPECT, $b = 4$ has a lower mean than $b = 6$ (section 5.4), so the finding that $b = 6$ is lowest depends on the shape rule. Here $s_{\max}$ is set by the $b = 4$ cell (0.0201), whose shape, 17×32, is the deepest at this budget. The criterion would fire without that cell as well, because the seed standard deviation of $b = 8$ (0.0160) also exceeds the gap. The comparison of $b = 6$ with $b = 8$ is unresolved at $n = 5$, not a demonstrated tie. At the observed seed standard deviations the Welch test has a power of about 21% at the observed gap, and its minimum detectable effect at 80% power (Welch test at $p \leq 0.05$) is 0.0299 nats, more than twice the gap (both simulated at the

observed standard deviations; exploratory).

At 4 Mbit the nearest adjacent bit-width is $b = 4$. The gap of 0.0061 exceeds $s_{\max} = 0.0043$ (ratio 1.41), so the criterion returns not null. The preregistered Welch test against $b = 4$, uncorrected as preregistered, gives $p = 0.0317$ (exact permutation $p = 0.0317$), significant at $\alpha = 0.05$, and the test against the other neighbour, $b = 8$, gives $p < 10^{-4}$. The intersection–union claim that $b = 6$ is better than both of its neighbours therefore holds at that level without a correction for two components, with $b = 6$ treated as fixed although it is the observed argmin (section 4). The gap is, however, below the power threshold of 0.0085. The ±1 seed-sd intervals of the two cells overlap, and the Welch 95% interval on the gap, 0.0007 to 0.0114, has its lower end close to zero. The 90% interval of the two one-sided tests, 0.0018 to 0.0103, excludes zero, but its upper end lies above the margin of 0.01 nats, so equivalence within that margin is not established (two one-sided tests, TOST, $p = 0.0625$; exploratory; the margin was chosen for this analysis and is not preregistered). As a sensitivity analysis that was not preregistered, a Holm correction over the family of the 6 adjacent tests across the three budgets gives $p = 0.0634$ for this comparison and rejects 4 of the 6, not including this one. The preregistration states that 5 seeds have roughly 80% power against gaps of about 2.0 seed standard deviations, and less against smaller gaps (77% in my later simulation; section 4), and it commits me not to report smaller gaps as trends. I therefore state only that, in the primary analysis, $b = 6$ has the lowest mean at 4 Mbit, that the null criterion does not fire and the preregistered tests against both neighbours reject, and that the size of its lead over $b = 4$ is not resolved. With the cell included, every bit-width other than $b = 4$ is separated from b^* at $p < 10^{-4}$ (exploratory, apart from the adjacent $b = 8$).

The $b = 6$ cell at this budget is one of the three convergence-pilot cells, which the preregistration's pilot clause excludes and its run plan includes (section 4). The preregistration supports both readings, and at 4 Mbit they disagree. Without the pilot cell, the lowest mean is at $b = 4$ (1.3346). Its neighbours in the remaining cells are $b = 3$ and $b = 8$, and the null criterion returns not null, with a gap of 0.0400 to $b = 8$ (95% interval 0.0346 to 0.0455) against $s_{\max} = 0.0043$, above the power threshold (Welch $p < 10^{-4}$; row 'no pilot' of table 2). At 256 Kbit and 828 Kbit the pilot cell is $b = 3$, which is not adjacent to b^*, and dropping it changes nothing.

In the primary analysis, $b = 6$ has the lowest mean at all three budgets. That result is decisive at 256 Kbit, null at 828 Kbit, and at 4 Mbit not null but below the power threshold. Read by the pilot clause, the 4 Mbit minimum is instead at $b = 4$, not null and above the threshold (section 5.7). Amendment A7 qualifies two of these results (section 5.8). On the averaged metric val_c the 4 Mbit argmin is $b = 4$, so it depends on end-of-training averaging (not robust under the preregistered rule), and at 828 Kbit the comparison of $b = 3$ with $b = 6$ on that metric is unresolved at 10 seeds per cell, whereas on `val_loss_best` $b = 6$ is lower than $b = 3$ by 0.0712 nats. On the test split, read once, the final-step argmin is $b = 6/6/6$, which is the validation argmin at every budget, and the null criterion returns the validation verdict at every budget. The qualifications at 4 Mbit and for the averaged weights are in section 5.6.

5.2 The ternary penalty shrinks with budget

I define the *ternary penalty* at a budget as the mean loss of the $b = 1.58$ cell minus the lowest cell mean at that budget. It is 0.5102 at 256 Kbit, 0.2108 at 828 Kbit and 0.0842 at 4 Mbit. The Welch tests against b^* give $p < 10^{-4}$, $p < 10^{-4}$ and $p < 10^{-4}$; they are exploratory, because $b = 1.58$ is not adjacent to b^* at any budget. Ternary loses at every budget even though it holds the most parameters in the grid at each one. At 4 Mbit, for example, it holds 2 616 320 parameters, against 696 320 for $b = 6$. Two scope conditions apply to every penalty below. First, the ternary cell quantizes its tied embedding at $b = 1.58$ like every other matrix, whereas large low-bit models are

Table 2: The preregistered null criterion and the secondary Welch tests, per budget. (a) b^* is the lowest-mean bit-width; the gap is to the nearest adjacent bit-width in the grid; $s_{\max}$ is the largest seed standard deviation over b^* and its neighbours. The result is null if gap $\leq s_{\max}$. The 95% interval is Welch's on the gap, and it is the effect-size statement; perm. p is the exact two-sided permutation p over all $\binom{10}{5}$ relabellings of the two cells' seeds, whose smallest attainable value is 2/252. Power: whether the gap reaches 2.0× the larger seed sd of the pair, the gap at which five seeds give ∼80% power as stated in the preregistration (simulated: 77%); it is a descriptive rule. The row marked 'no pilot' drops the convergence-pilot cell, as §5.4 of the preregistration reads literally; it is shown only where that changes the pair. (b) Two-sided Welch p-values of b^* against every other bit-width; * marks a gap below the power threshold. Values below 10^{-4} are tail probabilities of Student's t that rest on the seed losses being normal, and are printed as $< 10^{-4}$. Only the tests against the bit-widths adjacent to b^* were preregistered; the others are exploratory and uncorrected for multiple comparisons. (c) Exploratory seed bootstrap: the share of draws in which each b has the lowest mean, resampling the seeds of every cell; in the second block each draw also picks, per cell, the TIGHTEST or the ASPECT runs with equal probability.

(a) Null criterion

Budget	b^*	mean	nearest	gap	95% interval	$s_{\max}$	gap/$s_{\max}$	perm. p	power	verdict
256 Kbit	6	2.3315	8	0.0952	[0.0876, 0.1027]	0.0076	12.53	0.0079	above	not null
828 Kbit	6	1.8580	8	0.0123	[-0.0087, 0.0332]	0.0201	0.61	0.2698	below	**null**
4 Mbit	6	1.3286	4	0.0061	[0.0007, 0.0114]	0.0043	1.41	0.0317	below	not null
4 Mbit, no pilot	4	1.3346	8	0.0400	[0.0346, 0.0455]	0.0043	9.28	0.0079	above	not null

(b) Welch p-value, b^* vs. b

Budget	1.58	2	3	4	6	8	16
256 Kbit	$<10^{-4}$	$<10^{-4}$	$<10^{-4}$	$<10^{-4}$	b^*	$<10^{-4}$	—
828 Kbit	$<10^{-4}$	$<10^{-4}$	$<10^{-4}$	$<10^{-4}$	b^*	0.212^*	$<10^{-4}$
4 Mbit	$<10^{-4}$	$<10^{-4}$	$<10^{-4}$	0.0317^*	b^*	$<10^{-4}$	$<10^{-4}$

(c) Bootstrap: share of draws in which b has the lowest mean

256 Kbit, seeds	0	0	0	0	100.0%	0	—
828 Kbit, seeds	0	0	0	0	94.6%	5.4%	0
4 Mbit, seeds	0	0	0	0.1%	99.9%	0	0
828 Kbit, seeds+shapes	0	0	0	49.8%	48.8%	1.4%	0
4 Mbit, seeds+shapes	0	0	0	23.4%	76.6%	0	0

reported with the embedding (Ma et al., 2024), or the embedding and the output head (Kaushal et al., 2025; Maskey et al., 2026), at higher precision. The embedding holds 9.8%, 6.1% and 3.1% of the ternary cell's stored bits at the three budgets, a share that falls as the penalty falls, so part of the penalty's dependence on budget may be an embedding effect; I did not measure it. Second, training is quantization-aware from step 0, and ternary training preceded by a full-precision phase is reported to do better than ternary training from scratch (Nielsen et al., 2025); the penalty is that of ternary trained from scratch under my recipe.

The local log-log slope of the penalty against budget is -0.753 from 256 Kbit to 828 Kbit and -0.574 from 828 Kbit to 4 Mbit. Computed on the mean stored bits of the two cells that define each penalty instead of the nominal budgets, the slopes are -0.761 and -0.575, so the two axes give nearly the same slopes. Against the non-embedding bits of the argmin cell, a non-embedding count of the kind on which Kumar et al. (2025) and Frantar et al. (2025) compare precisions, the two local slopes are -0.594 and -0.575, and are not distinguishable ($z = +0.51$ for their difference in a seed bootstrap). The penalty falls across the measured range. The local slopes and the comparison on stored bits are exploratory. The only prediction about the penalty is P6, which Amendment A4 recorded after two budgets had been scored, so it is not part of the original preregistration (see below).

Because Amendment A4 (appendix B) recorded P6 after two budgets had been scored, P6 is post hoc and not an out-of-sample test of this extrapolation. When A4 was committed, the 4 Mbit ternary cell that P6 concerns already had a finished sweep run (a second finished within about a minute of the commit, so I cannot say whether that second run finished before or after the commit), and the pilot run of $b = 6$ existed (section 4); whether I had seen those results is not recorded. None of the other 4 Mbit sweep cells had started. A4 took the power law through the first two points (exponent -0.753). It recorded P6: a penalty of about 0.063 at 4 Mbit (recorded to three decimals; the unrounded value is 0.0632), and in any case below 0.12. The measured penalty is 0.0842. The bound holds, but the point estimate is off by $+0.0210$ (from the unrounded value), more than the seed standard deviation of either cell that defines the 4 Mbit penalty (0.0034 and 0.0043). P6 is scored on the primary sweep, as recorded. Under the preregistered ASPECT rule the 4 Mbit penalty is 0.1225 (3 seeds per cell), above the bound, so the bound holds under one shape rule and not the other.

A penalty in nats that falls with budget does not by itself mean that ternary is catching up. If loss follows $L = E + AN^{-\alpha}$ (irreducible loss E, coefficient A, effective parameter count N, exponent α) and a ternary weight is worth a fixed fraction of a b^* weight, the nat gap falls as a power of the budget with no change in relative efficiency.

A scale-free measure is the ratio B_{eq}/B, where B is the budget of the ternary cell and B_{eq} is the budget at which the lowest cell mean, interpolated log-linearly in B between budgets, equals that ternary cell's mean (exploratory). It is 0.59 at 828 Kbit (seed-bootstrap 95% interval 0.58–0.60) and 0.78 at 4 Mbit (0.77–0.79); at 256 Kbit it needs extrapolation below the smallest budget and is 0.28 (0.27–0.29). On this measure the ratio is higher at 4 Mbit than at 828 Kbit (the seed-bootstrap intervals do not overlap), and it is below 1 at both; a ratio of 1 would mean that ternary matches the lowest cell at the same budget. Across the three budgets the penalty is positive (the Welch tests above, which are uncorrected and exploratory) and decreases in nats over a 16-fold range of budget. The local slope is smaller in magnitude on the second interval (-0.574, seed-bootstrap 95% interval -0.621 to -0.532) than on the first (-0.753, -0.794 to -0.705; $z = +4.29$ for the difference), and the P6 point estimate missed in the same direction. Against the non-embedding bits of the argmin cell the two slopes, -0.594 (-0.627 to -0.556) and -0.575 (-0.622 to -0.533), are not distinguishable ($z = +0.51$). The apparent deceleration therefore depends on the axis against which the budget is measured, and I do not report it as a property of the penalty.

The slope bootstraps resample seeds within the ternary and argmin cells with b^* held fixed, and the budget-equivalent bootstrap resamples every cell and recomputes the lowest mean; both reflect seed noise only, not the interpolation (exploratory). The three budgets do not establish a functional form: three points give two local slopes. The data also do not establish whether the penalty reaches zero, or where. Amendment A4 records that a break-even budget was extrapolated and deliberately not relied on. I make no claim beyond 4 Mbit (section 7).

Two further qualifications apply. First, the penalties are measured on TIGHTEST shapes. Under ASPECT, the lowest mean at 828 Kbit falls from 1.8580 to 1.8150, while the ternary cell has the same shape under both rules (table 4), so the middle point depends on the shape rule. Defined within the ASPECT sweep, the penalty is 0.2533 at 828 Kbit and 0.1225 at 4 Mbit, a local slope of −0.454, against −0.574 under TIGHTEST. Second, the penalty would be larger at every budget under the final validation loss (section 5.5), and it is smaller at every budget on the averaged metric of Amendment A7, 0.3821, 0.1150 and 0.0487 nats, where it still falls with budget (section 5.8).

5.3 Matched parameters versus matched memory

Headline results for low-bit language models are often stated at matched parameter count (Ma et al., 2024), and Kaushal et al. (2025) state them at both matched parameters and matched bits. P3 predicted that the argmin at matched parameters differs from the argmin at matched bits, and the preregistration's test of it (its §6, item 2) is whether the argmin moves between the two panels of fig. 2. That test cannot be computed on this grid: no set of cells at one parameter count spans b, so the matched-parameter argmin is undefined. P3 is therefore not testable as preregistered. The analysis instead pairs cells from different budgets whose parameter counts agree within 12%, a tolerance set in the analysis script after data existed (table 3, whose pairs are lettered A to D in the order of its rows). The higher-precision member wins 4 of 4 pairs, which involve 3 distinct higher-precision cells (pairs A and B share one), and it stores 3.22–4.97 times as many weight bits as its partner. Every higher-precision member comes from the larger budget of its pair, and every cell at a larger budget has a lower mean than every cell at the next smaller one (table 1), so this outcome follows from the storage difference alone. The analysis scores P3 as holding under this post-hoc rule, together with the interior matched-memory argmin. Only pair C has a counterpart at equal storage that points the other way: at matched parameters $b = 16$ at 4 Mbit (1.5250) beats $b = 3$ at 828 Kbit (1.9292), while at equal storage $b = 3$ beats $b = 16$ at both budgets (1.9292 against 2.1294 at 828 Kbit, 1.3931 against 1.5250 at 4 Mbit). That precision can rank differently at fixed total bits than at fixed parameter count is implicit in the fixed-bit analysis of Dettmers and Zettlemoyer (2023) and is shown with both readings for ternary models in Kaushal et al. (2025). A related reversal between parameter count and storage at fixed accuracy has been reported for CNNs: lower-precision networks need more weights but, at 12% error, 4-bit networks need the least storage (Moons et al., 2017), and Abdolrashidi et al. (2021) plot both readings. Pair C shows the same kind of reversal, at 262 144 and 278 528 parameters, with QAT from step 0. Section 8 discusses what each comparison answers.

5.4 Sensitivity to the architecture shape rule

At a fixed budget and bit-width, usually more than one shape (layers × width) falls within the search tolerance. The rule that picks among them is a design choice that can interact with b (section 3.3). The primary sweep uses the TIGHTEST rule. The preregistered secondary sweep reruns 828 Kbit and 4 Mbit under the ASPECT rule with 3 seeds per cell (table 4). I define the *shape effect* of a cell as its ASPECT mean minus its TIGHTEST mean, for cells whose shape differs between the two rules.

Table 3: Matched-parameter pairs (P3): cells from different budgets whose parameter counts agree within 12%. Loss is mean ± seed sd over five seeds. Matching parameters across budgets cannot match memory: the storage column is the ratio of stored weight bits, higher-precision member over lower; bold marks the lower loss. The letters name the pairs in the text, in order of the lower-precision member's parameter count. Every pair is won by the higher-precision member, which holds 3.22–4.97× the storage, so each pair confounds a change of storage with a change of precision.

	lower precision			higher precision			Δparams	storage
pair	params	cell	loss	params	cell	loss		ratio
A	126 976	256 Kbit, $b = 2$	2.8467 ±0.0135	139 264	828 Kbit, $b = 6$	**1.8580 ±0.0122**	8.8%	3.22×
B	155 648	256 Kbit, $b = 1.58$	2.8416 ±0.0127	139 264	828 Kbit, $b = 6$	**1.8580 ±0.0122**	10.5%	3.22×
C	278 528	828 Kbit, $b = 3$	1.9292 ±0.0050	262 144	4 Mbit, $b = 16$	**1.5250 ±0.0060**	5.9%	4.97×
D	524 288	828 Kbit, $b = 1.58$	2.0688 ±0.0051	524 288	4 Mbit, $b = 8$	**1.3747 ±0.0043**	0.0%	4.96×

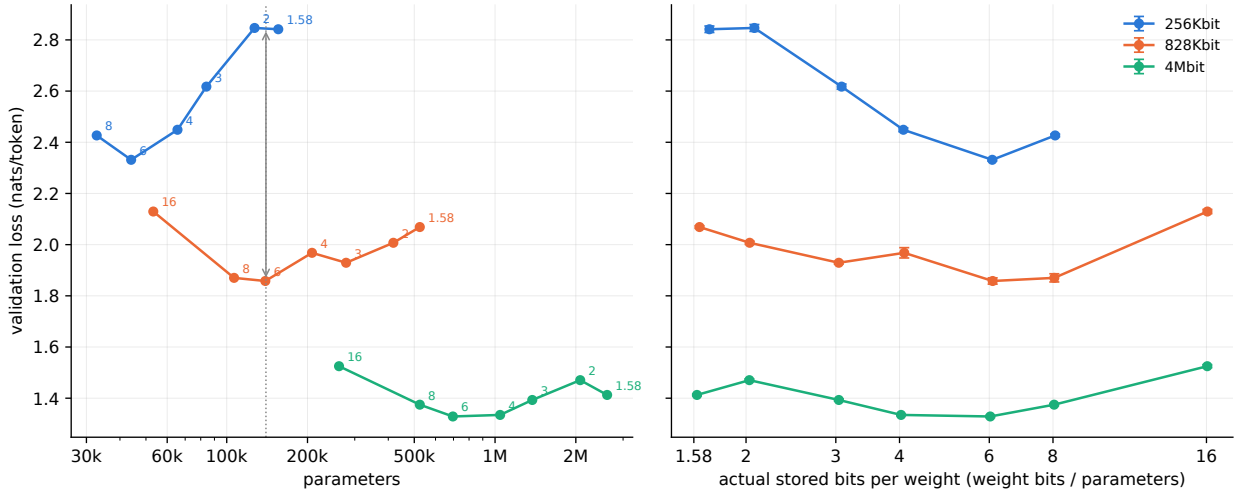


Figure 2: **Cell means against parameter count and against stored bits per weight.** *Left:* cell mean loss (nats per token) against parameter count, one line per budget, with labels giving b. Only means over $n = 5$ seeds are shown; seed standard deviations are in tables 1 and 3. The dotted vertical line marks pair A of table 3; its higher-precision member stores 3.22× the bits of its partner. *Right:* the same cells against actual stored bits per weight, with error bars of ±1 seed standard deviation; ticks mark the grid values of b.

The preregistered question is whether the argmin moves; the comparisons of shape effects with the gaps between bit-widths below are exploratory. The rules pick the same shape for 2 of the 7 cells at 828 Kbit and 1 of the 7 at 4 Mbit. For those cells both sweeps train the same architecture, and the secondary runs repeat the primary runs of seeds 0–2, so the difference of means comes only from averaging three seeds instead of five, not from shape.

At 4 Mbit the argmin does not move. It is $b = 6$ under both rules, and under ASPECT the null criterion returns not null (gap 0.0180 to $b = 4$, $s_{\max} = 0.0057$), with 3 seeds per cell, whose standard deviations the preregistration itself regards as weak (its §5.1). The largest shape effects in magnitude are -0.0214 ($b = 2$) and -0.0207 ($b = 6$). Every shape effect at this budget is smaller in magnitude than the primary-sweep gap between b^* and any bit-width other than its nearest neighbour $b = 4$; the smallest of those gaps is 0.0461, for $b = 8$. The two largest shape effects are, however, larger than the gap between b^* and its nearest neighbour (0.0061), so at 4 Mbit a change of shape moves the loss by more than the step from $b = 4$ to $b = 6$. The argmin survives because $b = 6$ improves under ASPECT by more than $b = 4$ does (-0.0207 against -0.0088), which widens the gap between them to 0.0180.

At 828 Kbit the argmin moves. Under ASPECT the lowest mean is at $b = 4$ (1.8150) rather than $b = 6$ (1.8580), and under ASPECT the null criterion returns not null (gap 0.0420 to $b = 6$, $s_{\max} = 0.0102$), again with 3 seeds per cell. The move comes from one cell. TIGHTEST builds $b = 4$ as 17×32 and ASPECT builds it as 5×64, a shape effect of -0.1531. That effect is larger in magnitude than the primary-sweep gap between $b = 6$ and every other bit-width except $b = 1.58$ (0.2108) and $b = 16$ (0.2714). At this budget, therefore, one change of shape at fixed precision moves the loss by more than changing precision from $b = 6$ to any of $b = 2$, 3, 4 or 8. The other shape effects at 828 Kbit are $+0.0556$ at $b = 2$ and $+0.0246$ at $b = 8$, both in favour of TIGHTEST. Of the remaining four cells, 2 ($b = 1.58$ and $b = 16$) have the same shape under both rules, so their differences are seed noise, and $b = 3$ has the largest effect of the four (-0.0034).

At both budgets the lowest ASPECT mean is below the lowest TIGHTEST mean (1.8150 against 1.8580, and 1.3078 against 1.3286). Neither rule finds the best shape for every cell, so the curves in fig. 1 are conditional on the shape rule. As preregistered, the secondary sweep cannot override the primary argmin, and the move at 828 Kbit is reported as a limitation: at that budget the ranking of bit-widths depends on the depth/width choice, in addition to being null. At 256 Kbit the question cannot be asked with this design. Only 1 of its 6 cells picks a different shape under ASPECT, so the budget was excluded from the secondary sweep by design (preregistration §5.2). Its reachable aspect ratios also differ strongly with bit-width (section 3.3): $b = 6$ and $b = 8$ each admit a single aspect ratio, and $b = 3$ and $b = 4$ only narrow ranges that overlap only at 5.33, so at those bit-widths precision and shape cannot be separated at this budget. Only $b = 1.58$ and $b = 2$ admit a spread of aspect ratios, from 1.88 to 21.3 and from 2.46 to 32 respectively, with gaps; neither admits any aspect ratio from 4 to 10.7, which contains all of those of $b = 3$, 4 and 6 (computed from the search space).

The search tolerance also lets cells at one budget store different amounts. Under ASPECT at 828 Kbit the neighbour $b = 6$ stores -3.8% relative to the argmin $b = 4$; in the primary sweep the nearest neighbour stores $+1.0\%$ relative to b^* at 828 Kbit and $+0.3\%$ at 4 Mbit. To price these differences I take the fall of the lowest cell mean across budgets, 0.360 nats per e-fold of the budget in a least-squares fit over the three budgets (0.403 and 0.331 on the two intervals), and charge each cell for its storage beyond the budget (exploratory). The storage-adjusted gap to the nearest neighbour is 0.0159 at 828 Kbit (against 0.0123) and 0.0070 at 4 Mbit (against 0.0061) in the primary sweep, and 0.0281 (against 0.0420) and 0.0162 (against 0.0180) under ASPECT. No argmin changes under the adjustment. The adjustment assumes that the budget-to-budget slope applies within a budget.

Table 4: Shape robustness (preregistered secondary sweep). The primary sweep picks each architecture with the TIGHTEST search rule ($n = 5$ seeds); the secondary reruns the same cells under the ASPECT rule ($n = 3$ seeds). Shapes are layers × width. Effect is the ASPECT mean over 3 seeds minus the TIGHTEST mean over 5; where the two rules pick the same shape, both sweeps train the same architecture on seeds of the primary sweep, so the difference comes only from averaging 3 seeds instead of 5, not from shape. Bold marks the lowest mean under each rule. 256 Kbit is excluded by design (`PREREGISTRATION.md` §5.2). The secondary sweep can never override the primary argmin.

		TIGHTEST (primary)		ASPECT (secondary)		
Budget	b	shape	loss	shape	loss	effect
828 Kbit	1.58	10×64	2.0688 ±0.0051	10×64	2.0683 ±0.0057	−0.0005
	2	4×96	2.0073 ±0.0062	8×64	2.0629 ±0.0075	+0.0556
	3	5×64	1.9292 ±0.0050	6×64	1.9258 ±0.0017	−0.0034
	4	17×32	1.9680 ±0.0201	5×64	**1.8150 ±0.0032**	−0.1531
	6	12×32	**1.8580 ±0.0122**	10×32	1.8570 ±0.0102	−0.0010
	8	2×64	1.8702 ±0.0160	7×32	1.8948 ±0.0027	+0.0246
	16	3×32	2.1294 ±0.0081	3×32	2.1289 ±0.0111	−0.0005
argmin: TIGHTEST $b = 6$, ASPECT $b = 4$ (moved; ASPECT null criterion: not null)						
4 Mbit	1.58	11×160	1.4127 ±0.0034	13×128	1.4304 ±0.0048	+0.0176
	2	20×96	1.4705 ±0.0047	13×128	1.4491 ±0.0014	−0.0214
	3	18×96	1.3931 ±0.0041	12×96	1.3906 ±0.0027	−0.0024
	4	9×96	1.3346 ±0.0027	10×96	1.3258 ±0.0021	−0.0088
	6	3×160	**1.3286 ±0.0043**	9×96	**1.3078 ±0.0042**	−0.0207
	8	10×64	1.3747 ±0.0043	10×64	1.3758 ±0.0057	+0.0011
	16	7×64	1.5250 ±0.0060	6×64	1.5304 ±0.0020	+0.0054
argmin: TIGHTEST $b = 6$, ASPECT $b = 6$ (stable; ASPECT null criterion: not null)						

5.5 Selection bias of the primary metric

The primary metric is a minimum over validation evaluations taken every 250 steps and at the final step, all scored on the same fixed validation batches. I define the *selection gain* of a run as its final validation loss minus its `val_loss_best`. This analysis, like the final-loss check below, is not among the preregistered analyses and is exploratory. Figure 3 shows it per cell, and table 17 (appendix E) lists it.

The selection gain falls as b rises. At 256 Kbit it falls at every step up in b, from 0.0474 ± 0.0245 at $b = 1.58$ to 0.0018 ± 0.0017 at $b = 8$; the means fall at every step, but each individual step is within the larger seed standard deviation of the two cells it joins (5 of 5 steps), so no single step is resolved against seed noise. At the other two budgets it is not strictly monotone. The only rises are $b = 3 \to 4$ (+0.0006, within seed sd 0.0064) at 828 Kbit and $b = 6 \to 8$ (+0.00003, within seed sd 0.0008) at 4 Mbit, both inside seed noise. At every budget the gain at $b = 1.58$ exceeds the gain at the highest feasible b by more than the seed standard deviation of either. At $b = 1.58$ it is 0.0474, 0.0197 and 0.0075 at the three budgets.

A positive selection gain does not show that validation loss rises late in training. The minimum of many noisy evaluations of a curve that is flat or still falling lies below its last value, by more when the evaluations are noisier and more numerous. I tested this against a no-rise null. For each run the null is the run's own non-increasing trend plus noise. The noise level is the standard deviation of successive differences over the second half of the run, divided by $\sqrt{2}$, which estimates the run's evaluation-to-evaluation fluctuation if the noise is independent across evaluations (calibrated below).

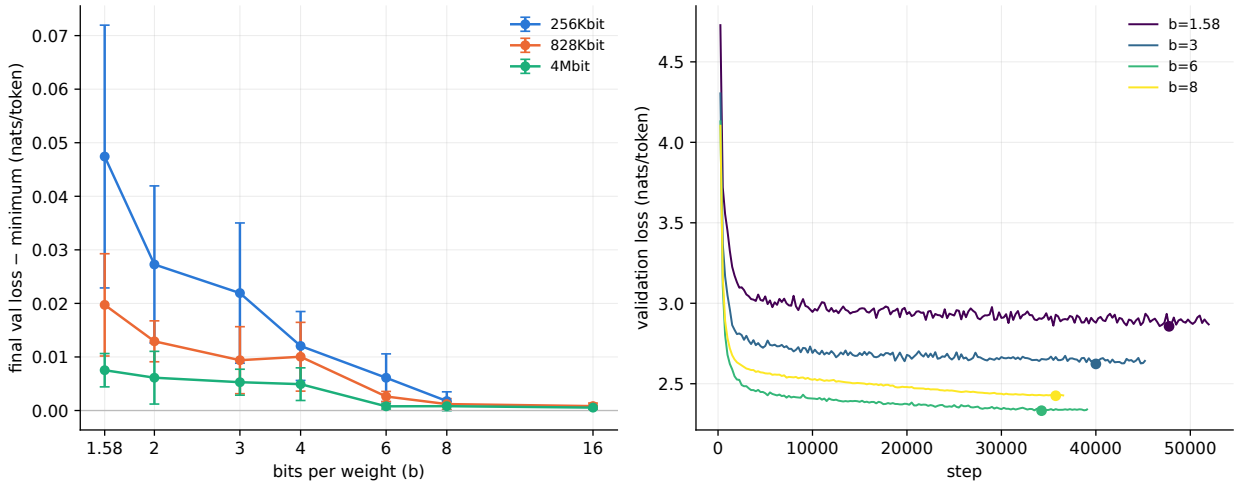


Figure 3: **The selection gain falls with b, as the noise of the evaluations does.** *Left:* selection gain (final validation loss minus `val_loss_best`) against b. Points are means over $n = 5$ seeds per cell, and error bars are ± 1 seed standard deviation. The gain falls with b at 256 Kbit and, apart from rises inside seed noise, at the other two budgets (table 17). A no-rise null, the minimum over noisy evaluations of each run's own non-increasing trend, accounts for these gains (section 5.5), so I read them as selection bias of the minimum, not as a late rise of validation loss. *Right:* for illustration only, validation curves of seed 0 of four cells at 256 Kbit, with dots marking each run's `val_loss_best`. Seed 0 is used for every cell. Seed 0 has the smallest ternary gain of the five seeds (0.0122 against a cell mean of 0.0474 nats), so this panel understates the ternary gain. These curves have no error bars; the cell means are in the left panel.

The null's minimum is taken over the same evaluation steps. Summed over the 20 primary cells, the null predicts 1.05 times the observed gain, and after a Holm correction over the cells no cell rejects the no-rise null at $\alpha = 0.05$: the largest excess of observed over null gain, 0.0116 nats at 256 Kbit, $b = 1.58$, has adjusted $p = 1$, and the Stouffer combination over cells (the sum of the per-cell normal scores, rescaled to a single z) gives $z = -1.45$, where a positive z would indicate a rise.

The null depends on how the fluctuation is estimated. Estimated instead from the residuals about each run's non-increasing trend over the whole run, it predicts 0.86 times the observed gain, $z = +2.51$, and one cell (4 Mbit, $b = 4$) rises at adjusted $p = 0.02$. On 400 synthetic no-rise curves with independent evaluation noise, that estimator puts the fluctuation at 0.76 times its true value and rejects in 14.2% of curves at $\alpha = 0.05$; the estimator used above gives 1.05 and 2.8%. I do not read the single rejection or the positive z of the residual-based estimator as evidence of a rise. The evaluation-to-evaluation fluctuation falls with b overall, though not at every step (at 256 Kbit from 0.0205 at $b = 1.58$ to 0.0020 at $b = 8$), and the selection gain falls with it. I therefore read the selection gain as selection bias of the metric, larger where evaluations are noisier, and not as evidence of a late rise in validation loss. As a further check on overfitting as a cause of a late rise, in 15 of the 20 primary cells the mean final training loss, that of the last step's own minibatch, is at or above the final validation loss (62 of 100 runs), so the runs do not typically end with training loss below validation loss. The right panel of fig. 3 shows single runs at 256 Kbit for illustration only; the claim rests on the seed means in the left panel and on the null.

Because the bias is larger at lower b, it lowers low-b losses more than high-b losses. Against the lower neighbours of b^* it works against b^*, so removing it could only widen those gaps. Against its

upper neighbour $b = 8$, which sets the gap at 256 Kbit and 828 Kbit, it works in favour of b^*: the gain of b^* is 0.0061 against 0.0018 for $b = 8$ at 256 Kbit, and 0.0026 against 0.0012 at 828 Kbit. The direct check is the final validation loss, which involves no selection. Under it, the lowest mean at every budget is at the same bit-width as under `val_loss_best` ($b = 6/6/6$ at 256 Kbit/828 Kbit/4 Mbit), with means of 2.3376, 1.8606 and 1.3294. The location of the lowest mean does not depend on the choice between these two metrics; at 4 Mbit it does depend on end-of-training averaging (section 5.8). Applied to the final loss, the null criterion returns not null, null and not null at the three budgets, against not null, null and not null under `val_loss_best`. At 828 Kbit the final-loss gap to $b = 8$ is 0.0108, against $s_{\max} = 0.0223$ and a `val_loss_best` gap of 0.0123. At 4 Mbit the final-loss gap to $b = 4$ is 0.0102, against 0.0061 under `val_loss_best`, and it is above the power threshold (Welch $p = 0.00489$). The preregistered verdicts are those on `val_loss_best`.

The bias is also a caveat on absolute losses. Each `val_loss_best` is a minimum selected on the split on which it is reported, so it is biased downward. Low-b cells train on more tokens (table 14) and so take more evaluations per run. I estimated the bias from the stored histories without further training (exploratory): each run's minimum over its odd-numbered evaluations is compared with the loss at the evaluation next to it (the following one, or the preceding one if the minimum is at the last evaluation), and the same is done for the even-numbered evaluations, and the two are averaged (table 17, panel b). The shift is largest at low b, up to 0.0486 nats. Between b^* and its adjacent bit-widths it differs by at most 0.0076, 0.0112 and 0.0014 at the three budgets, and the difference exceeds the pair's gap in 0 of the 6 adjacent pairs. Adding each cell's shift to its mean leaves the argmin at $b = 6/6/6$ and changes the nearest gap to 0.0923, 0.0099 and 0.0075 (from 0.0952, 0.0123 and 0.0061). Apart from any real change of the curve between adjacent evaluations, the estimate is the selection effect of a minimum over half of the evaluations. I expect the bias of the minimum over all of them to be larger (argued from the fall of an expected minimum with the number of draws, not measured), and I read this estimate as an order-of-magnitude check. The absolute losses of the low-precision cells, and so the ternary penalties of section 5.2, depend on this choice. Under the final loss, every ternary penalty would be larger, because the ternary cell's selection gain exceeds that of b^* at each budget: 0.0474 against 0.0061 at 256 Kbit, 0.0197 against 0.0026 at 828 Kbit, and 0.0075 against 0.0008 at 4 Mbit. Obtaining the reported loss of a low-b model in practice also requires validation-based checkpoint selection. The test split, read once, provides the unselected estimate (section 5.6): on the final-step weights of each run, scored on data never used for any choice, the argmin is the `val_loss_best` argmin at every budget, and the ternary cell's test loss is 2.8928, 2.0961 and 1.4306 at the three budgets, against 2.3437, 1.8689 and 1.3390 for $b = 6$.

5.6 The test split, read once

The test split was read once, after all training had finished and every validation verdict in this paper was final, with `scripts/test_read.py`. What the read measures and how it is analysed were fixed in the preregistration before the split was opened (its §14, extended by §16 of Amendment A7; section 3.9). Checkpoints hold the final-step weights, so the read measures the final-step loss of each of the 100 primary runs, not `val_loss_best`; as §16 requires, it also measures the averaged weights (variant (c) of A7, whose loss is val_c) for each of them (100 runs). Before the split was opened, every checkpoint was re-evaluated on validation; the re-evaluated losses equalled the recorded ones exactly: the largest difference was 0 nats for the final-step weights and 0 for the averaged ones. Per budget the analysis is the one applied to validation: cell means and seed standard deviations, the §4 null criterion, and Welch tests of the test argmin against each adjacent bit-width with the power rule (table 5). The test losses are the unselected estimate, reported beside the validation verdicts, which remain the preregistered primary result.

Final-step weights. At 256 Kbit the test argmin is $b = 6$ (it agrees with the validation argmin), 0.0909 below $b = 8$ against $s_{\max} = 0.0077$, so the criterion returns not null; Welch $p < 10^{-4}$, above the power threshold of 0.0118. This is the validation verdict. At 828 Kbit the test argmin is $b = 6$ (agrees), 0.0101 below $b = 8$ against $s_{\max} = 0.0223$, so the criterion returns **null**, as on validation; Welch $p = 0.288$, below the power threshold of 0.0314. At 4 Mbit the test argmin is $b = 6$ (agrees), 0.0112 below $b = 4$ against $s_{\max} = 0.0047$, so the criterion returns not null, with Welch $p = 0.00393$ against $b = 4$ and $p < 10^{-4}$ against $b = 8$. Here the test analysis differs from the validation verdict in one respect: the gap to $b = 4$ is above the power threshold of 0.0094, whereas on `val_loss_best` it is below (0.0061 against 0.0085). The final validation loss, which like the test read involves no selection, is also above the threshold there ($p = 0.00489$; section 5.5), so the difference lies between the selected minimum and the final step, not between the two splits. As preregistered, the primary verdict at 4 Mbit is unchanged: on `val_loss_best` the size of the lead of $b = 6$ over $b = 4$ is not resolved. On the unselected test estimate of the final step, that lead is above the power threshold.

Averaged weights. On the (c) weights the test argmin is $b = 6$, $b = 6$ and $b = 4$ at the three budgets, the val_c argmin at each (comparison with it: agrees, agrees and agrees). At 4 Mbit it therefore disagrees with the `val_loss_best` argmin: $b = 4$ is 0.0228 below $b = 6$ against $s_{\max} = 0.0044$ (not null; Welch $p < 10^{-4}$, above the power threshold). The dependence of the 4 Mbit argmin on end-of-training averaging (section 5.8) thus holds on held-out data. At 828 Kbit the criterion, which fires on `val_loss_best`, does not fire on the (c) weights, as on val_c: $b = 6$ is 0.0339 below $b = 8$ against $s_{\max} = 0.0206$ (not null; Welch $p = 0.00514$, above the power threshold). The non-adjacent $b = 3$, which the criterion does not test, is 0.0058 behind $b = 6$ (Welch $p = 0.307$ at 5 seeds; exploratory), so the test read does not resolve the comparison that check M4 of Amendment A7 left unresolved (section 5.8). At 256 Kbit $b = 6$ is 0.0554 below $b = 4$ against $s_{\max} = 0.0058$ (not null).

5.7 The analysis choices together

Four choices bear on the headline: the shape rule, the loss metric, whether the pilot cells are kept, and, for the power threshold, which seed standard deviation is used. Table 6 crosses the first three (8 combinations). At 256 Kbit, where only TIGHTEST was run, every combination gives $b = 6$, not null and above the power threshold. At 828 Kbit the argmin is $b = 6$ and null under TIGHTEST and $b = 4$ under ASPECT, whatever the metric in the table and the pilot choice. At 4 Mbit the argmin is $b = 6$ with the pilot cell and $b = 4$ without it, under either rule and either metric; it is below the power threshold only in the preregistered primary analysis. The table does not include the averaged metric val_c of Amendment A7, under which the 4 Mbit argmin is $b = 4$ with the pilot cell kept, and under which $b = 3$ and $b = 6$ are unresolved by the rule of Amendment A7 at 828 Kbit (section 5.8). The primary analysis and the literal reading of the pilot clause are the two preregistered readings; the other rows are exploratory.

5.8 Is the optimum an artifact of the training recipe?

The optimum found above, $b = 6$ on the primary metric, lies above the 1–4 bits at which weight-only studies at fixed memory place it, at 19M parameters and above. It also lies at the upper end of the range, up to 6 bits, reported for low training compute (section 2). Before that difference can be attributed to scale, three choices of my recipe have to be excluded, each of which the literature or my own checkpoints suggested could affect the comparison between bit-widths at 828 Kbit and 4 Mbit. The first is weight noise at the end of training. Under QAT, latent weights collect at quantization decision boundaries and oscillate between levels; this is shown for convolutional networks, most

Table 5: Test loss (PREREGISTRATION.md §14, §16), nats per token, every primary run evaluated once on the test split. *final*: the final-step checkpoint; *(c)*: the A7 averaged weights (section 5.8). Mean over seeds, ±1 seed sd beneath, bold the argmin. The §4 null criterion and the Welch tests as for validation. The preregistered primary result remains `val_loss_best`.

Budget	1.58	2	3	4	6	8	16
256 Kbit							
final	2.8928	2.8784	2.6451	2.4665	**2.3437**	2.4346	—
	±0.0217	±0.0187	±0.0159	±0.0077	±0.0056	±0.0059	
	$b^* = 6$: gap 0.0909 vs $s_{\max}$ 0.0077, not null; Welch $p < 10^{-4}$ vs $b = 8$, above the power threshold; agrees with the `val_loss_best` argmin						
(c)	2.6720	2.6842	2.4833	2.3457	**2.2903**	2.4142	—
	±0.0085	±0.0130	±0.0058	±0.0058	±0.0043	±0.0045	
	$b^* = 6$: gap 0.0554 vs $s_{\max}$ 0.0058, not null; Welch $p < 10^{-4}$ vs $b = 4$, above the power threshold; agrees with the `val_loss_best` argmin; agrees with the val_c argmin						
828 Kbit							
final	2.0961	2.0283	1.9472	1.9859	**1.8689**	1.8790	2.1365
	±0.0071	±0.0100	±0.0054	±0.0223	±0.0119	±0.0157	±0.0086
	$b^* = 6$: gap 0.0101 vs $s_{\max}$ 0.0223, **null**; Welch $p = 0.288$ vs $b = 8$, below the power threshold; agrees with the `val_loss_best` argmin						
(c)	1.9402	1.8913	1.8319	1.8836	**1.8261**	1.8600	2.1144
	±0.0033	±0.0053	±0.0030	±0.0206	±0.0109	±0.0157	±0.0077
	$b^* = 6$: gap 0.0339 vs $s_{\max}$ 0.0206, not null; Welch $p = 0.00514$ vs $b = 8$, above the power threshold; agrees with the `val_loss_best` argmin; agrees with the val_c argmin						
4 Mbit							
final	1.4306	1.4874	1.4091	1.3502	**1.3390**	1.3857	1.5354
	±0.0043	±0.0091	±0.0032	±0.0018	±0.0047	±0.0042	±0.0060
	$b^* = 6$: gap 0.0112 vs $s_{\max}$ 0.0047, not null; Welch $p = 0.00393$ vs $b = 4$, above the power threshold; agrees with the `val_loss_best` argmin						
(c)	1.3443	1.3921	1.3335	**1.2952**	1.3180	1.3613	1.5134
	±0.0016	±0.0031	±0.0029	±0.0026	±0.0044	±0.0040	±0.0057
	$b^* = 4$: gap 0.0228 vs $s_{\max}$ 0.0044, not null; Welch $p < 10^{-4}$ vs $b = 6$, above the power threshold; **disagrees** with the `val_loss_best` argmin; agrees with the val_c argmin						

Table 6: Every combination of the analysis choices that bear on the headline: the shape rule, the loss metric, and whether the convergence-pilot cells ($b = 3$ at 256 Kbit and 828 Kbit, $b = 6$ at 4 Mbit) are kept. Each entry gives the lowest-mean b, the preregistered null criterion applied to it, and whether the gap to its nearest neighbour reaches the power threshold. The first row is the preregistered primary analysis; the row below it is the literal reading of the preregistration's pilot clause (§5.4), which conflicts with its run plan (§5.1) and has equal standing. Every other row is exploratory. ASPECT rows have three seeds per cell and no 256 Kbit cells.

Shape rule	Metric	Pilot cells	256 Kbit	828 Kbit	4 Mbit
TIGHTEST	`val_loss_best`	kept	$b = 6$, not null, above	$b = 6$, **null**, below	$b = 6$, not null, below
TIGHTEST	`val_loss_best`	dropped	$b = 6$, not null, above	$b = 6$, **null**, below	$b = 4$, not null, above
TIGHTEST	final loss	kept	$b = 6$, not null, above	$b = 6$, **null**, below	$b = 6$, not null, above
TIGHTEST	final loss	dropped	$b = 6$, not null, above	$b = 6$, **null**, below	$b = 4$, not null, above
ASPECT	`val_loss_best`	kept	—	$b = 4$, not null, above	$b = 6$, not null, above
ASPECT	`val_loss_best`	dropped	—	$b = 4$, not null, above	$b = 4$, not null, above
ASPECT	final loss	kept	—	$b = 4$, not null, above	$b = 6$, not null, above
ASPECT	final loss	dropped	—	$b = 4$, not null, above	$b = 4$, not null, above

strongly in efficient architectures with depth-wise separable layers at 4 bits or fewer (Nagel et al., 2022). For ternary weights in LLaMA-3.2 models, weights are reported to accumulate at the deadzone boundary, which is argued to trap them in oscillation (Huang et al., 2026). I expect the same in my transformers, most at low b, so the last iterate may understate what a low-b cell has learnt. The second is the single untuned learning-rate rule. Small models are reported to be more sensitive to hyperparameters jointly, with the learning rate among those searched (Lourie et al., 2026). Tuning the learning rate, batch size and AdamW β_2 is reported to lower the loss at low compute, by up to 0.5 nats per token in that study (Porian et al., 2024), and ternary weights are reported to need a higher learning rate (Zhu et al., 2024; Kaushal et al., 2025). My rule gives wider cells a lower rate (at 4 Mbit the widest cells are $b = 1.58$ and $b = 6$, at 828 Kbit the widest is $b = 2$, and at 256 Kbit all cells have the same width). The third is the token-budget model, whose equal residual across cells is assumed, not measured (section 3.7). Amendment A7 (appendix B) specified four checks, with their falsifiers, after the TinyStories sweep and before any of its own runs. It changes no preregistered verdict; its results are reported beside the primary ones.

M1: end-of-training averaging. Every complete run, primary and shape-robustness, was continued for 400 optimizer steps from its final checkpoint, with model and optimizer state restored, at the schedule's final learning rate and on its own batch stream. The averaged metric val_c (variant (c) of the amendment) is the loss of the run's own quantizer applied to the fp32 latent weights averaged over those steps; it is defined at every b, so $b = 16$ is the control. M1 also records the mean A of the last iterate over 16 evaluations, a baseline with no selection; for $b < 16$ the rounded time-average of each weight's integer code; a separate continuation annealed linearly to zero; and the final checkpoint evaluated on 50 fresh validation batches drawn with another seed, which measures the offset of the fixed validation batches directly. Two falsifiers were fixed in advance. The averaging effect is generic rather than specific to low precision at a budget if its gain at $b = 16$ is at least half its gain at $b = 1.58$. The argmin is robust to averaging at a budget if the val_c argmin equals the `val_loss_best` argmin and no adjacent gap reverses by more than the larger seed standard deviation of the pair; otherwise I state in the abstract that the argmin at that budget depends on end-of-training averaging. Table 7 gives val_c beside `val_loss_best` for every primary cell. Averaging lowers the loss most at low b: the gain over the last-iterate mean is 0.2162 nats for ternary and 0.0518 for $b = 6$ at 256 Kbit, and 0.0866 against 0.0217 at 4 Mbit. At 828 Kbit the gain at $b = 16$ is 0.0224, 0.138 times the ternary gain of 0.1627, and at 4 Mbit it is 0.0223, 0.257 times 0.0866. The generic reading requires a ratio of at least one half, so it is not supported at either budget, and by this criterion the averaging effect is specific to low precision rather than generic. At 4 Mbit the extra gain is confined to $b \leq 4$: the gains at $b = 6$, 8 and 16 are 0.0217, 0.0245 and 0.0223. At 828 Kbit the gain at $b = 8$ (0.0186) is below that of the $b = 16$ control, and the gain at $b = 6$ (0.0443) is above it. At 256 Kbit there is no $b = 16$ cell, so the test does not apply.

On val_c the argmin is $b = 6$ at 256 Kbit (gap 0.0555 against $s_{\max} = 0.0060$, not null) and $b = 6$ at 828 Kbit, where the null criterion that fired on `val_loss_best` no longer fires: the nearest adjacent cell, $b = 8$, is 0.0352 behind against $s_{\max} = 0.0195$, with Welch $p = 0.00427$, and $b = 4$ is 0.0580 behind ($p = 9.71 \times 10^{-4}$). The non-adjacent $b = 3$, which the preregistered criterion does not test, is only 0.0059 behind ($p = 0.305$); M4 takes it up. The 256 Kbit argmin is robust to averaging and the 828 Kbit argmin is robust, under the preregistered rule: the argmin is unchanged and no adjacent gap reverses by more than the larger seed standard deviation of its pair. At 4 Mbit the val_c argmin is $b = 4$ (1.2851), 0.0230 below $b = 6$ (1.3080) against $s_{\max} = 0.0043$, not null, Welch $p < 10^{-4}$. The $b = 4 \to 6$ gap changes sign, from $b = 6$ ahead by 0.0061 on `val_loss_best` to $b = 4$ ahead by 0.0230, more than the larger seed standard deviation of the pair (0.0043). The 4 Mbit argmin

is therefore not robust to averaging, and, as preregistered, the abstract says that the argmin at this budget depends on end-of-training averaging. The reversal comes from averaging the weights, not from removing the selection of the minimum: on the last-iterate mean A, which involves no selection, the argmin is still $b = 6$ (gap 0.0082, $p = 0.0111$). The shape-robustness sweep gives the same verdicts: at 828 Kbit its argmin is $b = 4$ on both metrics (robust), and at 4 Mbit it moves from $b = 6$ to $b = 4$ (not robust).

Evaluated on 50 fresh validation batches, the final checkpoint scores worse than on the fixed batches by 0.0058–0.0094 nats per cell at 256 Kbit, 0.0091–0.0109 at 828 Kbit and 0.0106–0.0119 at 4 Mbit. This offset of the fixed batches varies little with b at a budget, and the ranges just given are each smaller than the gap between the argmin and its nearest neighbour there, so it cannot change the comparison of the argmin with its nearest neighbour at any budget. I did not test its effect on gaps smaller than that range. It is a different quantity from the selection of the minimum estimated in section 5.5, which is largest at low b, up to 0.0486 nats; the final checkpoint involves no such selection. On val_c the ternary penalty is 0.3821, 0.1150 and 0.0487 nats at the three budgets, against 0.5102, 0.2108 and 0.0842 on `val_loss_best`; at 4 Mbit it is measured against $b = 4$. Averaging narrows the penalty at every budget, and the penalty still falls with budget.

Argmin groups. M2–M4 use, at each budget, the *argmin group*: the val_c argmin, its grid neighbours, and every b whose val_c mean is within $\max(0.03, 2s)$ nats of it, with s the larger seed standard deviation of the two cells. The 0.03-nat margin is an assumption fixed in advance, not a measurement. Where every other member is more than 0.05 nats from the argmin, M2 and M3 use seed 0 only, and otherwise seeds 0–2. At 256 Kbit the group is $b \in \{4, 6, 8\}$, the argmin and its two neighbours; both are more than 0.05 nats behind (0.0555 and 0.1237), so M2 and M3 use 1 seed. At 828 Kbit it is $b \in \{3, 4, 6, 8\}$: $b = 3$ joins on the margin, 0.0059 nats behind, and M2 and M3 use 3 seeds. At 4 Mbit, where the val_c argmin is $b = 4$, it is $b \in \{3, 4, 6\}$, with 3 seeds; $b = 6$ is both a neighbour and within the margin (0.0230 behind).

M2: learning rate. Each group cell was retrained with the rule's learning rate multiplied by 0.5 and by 2, and extended one step further, to 0.25 or 4, where the best multiplier by val_c lay at an edge. These runs form a separate sensitivity arm and never enter the primary analysis, whose single learning-rate rule is unchanged. Falsifier: with every cell at its own best multiplier, the argmin of val_c changes or an adjacent gap reverses by more than the larger seed standard deviation of the pair; then the argmin at that budget is reported as dependent on learning-rate tuning. Table 8 gives the results. In no group cell is the rule's learning rate the best tested; the best multiplier lay at the upper edge of the M2 range in 2, 4 and 2 group cells at the three budgets, every cell except 256 Kbit $b = 4$ and 4 Mbit $b = 6$, so M2 alone did not bracket the optimum. With every cell at its best M2 multiplier, the argmin is $b = 6$, $b = 6$ and $b = 4$, unchanged, with no adjacent gap reversed, and the falsifier does not fire at any budget.

Amendment A8 (appendix B), recorded after most M2 runs had been read and before any of its own runs, extended those cells to $\times 8$ (8 cells) and then to $\times 16$ where $\times 8$ was better (3 cells), with a cap at $\times 16$. The optimum is then bracketed in every group cell, with none at the cap. The best multipliers lie between $\times 2$ and $\times 8$; at 828 Kbit, $b = 3$, 6 and 8 are best at $\times 8$, $\times 8$ and $\times 8$. Re-applied over the full range, the falsifier again does not fire: with every cell at its best multiplier, the argmin is $b = 6$, $b = 6$ and $b = 4$. The argmin is the same at every multiplier that every group cell ran, $b = 6$ at 256 Kbit and $b = 4$ at 4 Mbit; at 828 Kbit it alternates between $b = 6$ and $b = 3$, the pair that M4 could not separate. Tuning the learning rate lowers the loss of every group cell. Over the multipliers each cell ran ($\times 0.5$ to $\times 4$, $\times 8$ or $\times 16$; table 8) and with each cell at its own

Table 7: A7 M1: the averaging continuation of every primary run. `val_loss_best` is the preregistered metric; `val_c` quantizes the time-averaged latent weights of a 400-step continuation at the final learning rate (the corrected metric of Amendment A7); gain is $A - $ `val_c`, where A is the mean of 16 last-iterate evaluations over the continuation; offset is `val_fresh` $-$ `val_ckpt`, the offset of the fixed validation batches against 50 fresh ones, not the selection of the minimum. Mean over five seeds, ±1 seed sd beneath; bold marks each metric's argmin. Robust to averaging: same argmin on both metrics and no adjacent gap reversed by more than the larger `val_c` seed sd of the pair. Generic: gain(b=16) $\geq 0.5\times$ gain(b=1.58).

Budget	b	`val_loss_best`	`val_c`	gain	offset
256 Kbit	1.58	2.8416±0.0127	2.6663±0.0090	0.2162±0.0101	0.0058±0.0024
	2	2.8467±0.0135	2.6786±0.0140	0.2029±0.0066	0.0058±0.0015
	3	2.6175±0.0101	2.4773±0.0062	0.1616±0.0066	0.0073±0.0008
	4	2.4489±0.0076	2.3397±0.0060	0.1237±0.0033	0.0077±0.0016
	6	**2.3315±0.0048**	**2.2842±0.0042**	0.0518±0.0018	0.0093±0.0006
	8	2.4267±0.0055	2.4079±0.0048	0.0205±0.0009	0.0094±0.0009
	argmin $b = 6 \to 6$; §4 on `val_c`: gap 0.0555 vs $s_{\max}$ 0.0060, not null; robust to averaging				
828 Kbit	1.58	2.0688±0.0051	1.9323±0.0040	0.1627±0.0039	0.0107±0.0004
	2	2.0073±0.0062	1.8832±0.0052	0.1412±0.0029	0.0109±0.0009
	3	1.9292±0.0050	1.8232±0.0030	0.1175±0.0034	0.0108±0.0013
	4	1.9680±0.0201	1.8753±0.0195	0.1033±0.0014	0.0102±0.0008
	6	**1.8580±0.0122**	**1.8173±0.0111**	0.0443±0.0019	0.0102±0.0008
	8	1.8702±0.0160	1.8525±0.0157	0.0186±0.0002	0.0091±0.0006
	16	2.1294±0.0081	2.1079±0.0077	0.0224±0.0004	0.0092±0.0009
	argmin $b = 6 \to 6$; §4 on `val_c`: gap 0.0352 vs $s_{\max}$ 0.0195, not null; robust to averaging; gain ratio b=16/ternary 0.138: low-precision specific				
4 Mbit	1.58	1.4127±0.0034	1.3338±0.0017	0.0866±0.0016	0.0113±0.0011
	2	1.4705±0.0047	1.3816±0.0028	0.0981±0.0009	0.0118±0.0008
	3	1.3931±0.0041	1.3234±0.0026	0.0745±0.0011	0.0119±0.0007
	4	1.3346±0.0027	**1.2851±0.0027**	0.0528±0.0008	0.0111±0.0007
	6	**1.3286±0.0043**	1.3080±0.0043	0.0217±0.0004	0.0106±0.0004
	8	1.3747±0.0043	1.3513±0.0042	0.0245±0.0004	0.0109±0.0010
	16	1.5250±0.0060	1.5035±0.0056	0.0223±0.0002	0.0114±0.0005
	argmin $b = 6 \to 4$; §4 on `val_c`: gap 0.0230 vs $s_{\max}$ 0.0043, not null; **not robust** to averaging; gain ratio b=16/ternary 0.257: low-precision specific				

Table 8: A7 M2 and A8: mean `val_c` of each argmin-group cell at each learning-rate multiplier on the rule's value (×1 is the primary run of the same seeds), ±1 seed sd beneath; bold marks the cell's best multiplier over everything it was run at. M2 ran ×0.5 and ×2 and extended one step (×0.25 or ×4); A8 ran ×8 for every cell best at ×4 under M2, then ×16 where ×8 beat ×4, and no further. A best multiplier at the (high or low) edge of the tested range means the optimum was not bracketed; at ×16, not bracketed at the cap. The falsifier (M2, and re-applied by A8 over the full range): with every cell at its own best multiplier, the argmin changes or an adjacent gap reverses by more than the larger seed sd of the pair (one-seed budgets: the M1 five-seed sd). M2: 72 of 72 planned runs recorded; A8: 29 of 29 runs recorded.

Budget	b	×0.5	×1	×2	×4	×8	×16	best	bracketed
256 Kbit	4	2.3511 (n=1)	2.3347 (n=1)	**2.3192** (n=1)	2.3215 (n=1)	—	—	×2	yes
	6	2.3043 (n=1)	2.2854 (n=1)	2.2679 (n=1)	**2.2600** (n=1)	2.2640 (n=1)	—	×4	yes
	8	2.4285 (n=1)	2.4072 (n=1)	2.3899 (n=1)	**2.3824** (n=1)	2.3946 (n=1)	—	×4	yes
	seed 0; argmin at ×1 $b = 6$; M2, each cell at its best $b = 6$; no adjacent reversal beyond sd: argmin does not depend on learning-rate tuning; A8, each cell at its best over ×0.5–×8: $b = 6$; no reversal beyond sd: argmin does not depend on learning-rate tuning								
828 Kbit	3	1.8503 ±0.0046	1.8238 ±0.0041	1.8063 ±0.0043	1.7964 ±0.0027	**1.7937** ±0.0072	1.8145 ±0.0222	×8	yes
	4	1.8991 ±0.0151	1.8687 ±0.0147	1.8537 ±0.0058	**1.8329** ±0.0053	1.8412 ±0.0035	—	×4	yes
	6	1.8360 ±0.0087	1.8219 ±0.0045	1.8141 ±0.0076	1.7865 ±0.0080	**1.7809** ±0.0082	1.8362 ±0.0050	×8	yes
	8	1.8886 ±0.0124	1.8590 ±0.0151	1.8403 ±0.0125	1.8235 ±0.0212	**1.8163** ±0.0147	1.8172 ±0.0041	×8	yes
	seeds 0, 1, 2; argmin at ×1 $b = 6$; M2, each cell at its best $b = 6$; no adjacent reversal beyond sd: argmin does not depend on learning-rate tuning; A8, each cell at its best over ×0.5–×16: $b = 6$; no reversal beyond sd: argmin does not depend on learning-rate tuning								
4 Mbit	3	1.3375 ±0.0011	1.3218 ±0.0019	1.3144 ±0.0021	**1.3136** ±0.0039	1.3215 ±0.0020	—	×4	yes
	4	1.2977 ±0.0005	1.2845 ±0.0023	1.2709 ±0.0026	**1.2702** ±0.0005	1.2905 ±0.0047	—	×4	yes
	6	1.3203 ±0.0016	1.3088 ±0.0051	**1.3025** ±0.0024	1.3038 ±0.0006	—	—	×2	yes
	seeds 0, 1, 2; argmin at ×1 $b = 4$; M2, each cell at its best $b = 4$; no adjacent reversal beyond sd: argmin does not depend on learning-rate tuning; A8, each cell at its best over ×0.5–×8: $b = 4$; no reversal beyond sd: argmin does not depend on learning-rate tuning								

best multiplier, it does not move the argmin of val_c at any budget. A cell's best multiplier is chosen on the seeds on which its tuned loss is reported, so tuned means are biased downward, and a cell that ran more multipliers is selected from more of them; the falsifier compares cells that were all selected this way. Only the 10 argmin-group cells, of the 20 primary cells, were varied; the ternary, 2-bit and bf16 cells and the other cells were not. At 256 Kbit this rests on one seed per multiplier, judged against the five-seed standard deviations of M1.

M3: training length. Each group cell was retrained at half its token budget, with a complete cosine schedule over the shorter length, and its gain $\mathrm{val}_c(T/2) - \mathrm{val}_c(T)$ paired by seed. If the spread of gains across the group exceeds the gap between the argmin and its nearest neighbour, the ranking

Table 9: A7 M3: every argmin-group cell retrained at half its token budget with a full cosine schedule over that length. Gain = `val_c`$(T/2)$ − `val_c`(T), paired by seed; mean ±1 seed sd beneath. A budget's ranking is sensitive to training length if the spread of the cells' mean gains exceeds the gap between b_c^* and its nearest grid neighbour on `val_c`.

Budget	b	seeds	`val_c`$(T/2)$	`val_c`(T)	gain
256 Kbit	4	1	2.3564 (n=1)	2.3347 (n=1)	0.0218 (n=1)
	6	1	2.3015 (n=1)	2.2854 (n=1)	0.0161 (n=1)
	8	1	2.4437 (n=1)	2.4072 (n=1)	0.0365 (n=1)
	spread of gains 0.0203 vs nearest-neighbour gap of $b_c^* = 6$ 0.0555: not sensitive to training length (closest group member $b = 4$ at 0.0555)				
828 Kbit	3	3	1.8383±0.0060	1.8238±0.0041	0.0146±0.0046
	4	3	1.9066±0.0085	1.8687±0.0147	0.0379±0.0087
	6	3	1.8492±0.0059	1.8219±0.0045	0.0273±0.0022
	8	3	1.8886±0.0048	1.8590±0.0151	0.0297±0.0136
	spread of gains 0.0233 vs nearest-neighbour gap of $b_c^* = 6$ 0.0352: not sensitive to training length (closest group member $b = 3$ at 0.0059)				
4 Mbit	3	3	1.3415±0.0052	1.3218±0.0019	0.0197±0.0039
	4	3	1.2982±0.0025	1.2845±0.0023	0.0137±0.0029
	6	3	1.3269±0.0048	1.3088±0.0051	0.0180±0.0009
	spread of gains 0.0060 vs nearest-neighbour gap of $b_c^* = 4$ 0.0230: not sensitive to training length (closest group member $b = 6$ at 0.0230)				

at that budget is reported as sensitive to training length. This is the check that Amendment A6 specifies for the second corpus (section 6), applied to TinyStories, where it tests, through the gain model, the equal residual that the token-budget model assumes. Under the gain model, a check that passes bounds the spread of residuals by 1.86 times the gap, not by the gap itself (section 6). Table 9 gives the gains. Halving the training length raises val_c in every group cell. The spread of the gains across the group is 0.0203, 0.0233 and 0.0060 nats at the three budgets, against nearest-neighbour gaps of 0.0555, 0.0352 and 0.0230, so by the preregistered rule the ranking is sensitive to training length at none of the three budgets (not sensitive, not sensitive, not sensitive). The rule compares the spread with the nearest grid neighbour only. At 828 Kbit the closest group member is the non-adjacent $b = 3$, 0.0059 behind, and the spread exceeds that gap, so M3 does not protect the order of $b = 3$ and $b = 6$ against a change in training length. At 256 Kbit the check rests on 1 seed per cell.

M4: seeds. Where, on val_c, the null criterion fires at a budget or the Welch p between the argmin and a group member exceeds 0.01, both cells received new seeds until the number simulated for 80% power at the observed val_c gap and standard deviations was reached, capped at 10 seeds per cell. A comparison still unresolved at the cap is reported as unresolved at that n, with its minimum detectable effect. The new seeds are kept apart from the preregistered five-seed analysis, which they do not alter. One comparison met the trigger: 828 Kbit $b = 6$ against $b = 3$, with five-seed Welch $p = 0.305$ (table 10). The simulated power at the cap (Welch test at $p \leq 0.01$) was only 11.9%, so the comparison could not be powered within the cap, and both cells were taken to the cap of 10 seeds. At that n the gap of $b = 3$ over $b = 6$, the mean of $b = 3$ minus that of $b = 6$, is −0.0009 nats, so $b = 3$ has the nominally lower mean, by an amount that the data do not distinguish from zero (Welch $t = -0.218$, df 10.3, $p = 0.832$). The Welch 95% interval of the gap is −0.0097 to 0.0080 nats, so true differences below −0.0097 nats and above 0.0080 nats are excluded at that level, and

Table 10: A7 M4: comparisons that the preregistered trigger sent to more seeds (Welch $p > 0.01$ between b_c^* and a group member, or a null §4 verdict on `val_c`), capped at 10 seeds per cell. M1 columns: the five-seed gap, Welch p, and the simulated power at $n = 10$ (probability of $p \leq 0.01$ at the observed gap and sds). At the new n: the gap, Welch p, and the minimum detectable effect (the smallest true gap with simulated 80% power at $p \leq 0.01$, at the observed sds). Status: complete.

Budget	pair	M1 gap	M1 p	power$_{10}$	n	gap	p	MDE	verdict
828 Kbit	$b = 6$ vs $b = 3$	0.0059	0.305	11.9%	10/10	-0.0009	0.832	0.0164	**unresolved**

differences inside that interval are not. The minimum detectable effect, the true difference that a Welch test at $p \leq 0.01$ with these standard deviations detects with 80% power, is 0.0164 nats; it states what the test could have detected, not what the data exclude. By the rule of Amendment A7 the comparison is unresolved at $n = 10$ (Welch $p > 0.01$), and I name no best bit-width between $b = 3$ and $b = 6$ at 828 Kbit. In a two one-sided tests analysis that was not preregistered, with a margin of 0.01 nats chosen after these data existed, the 90% interval of the gap, -0.0081 to 0.0063, lies inside the margin (TOST $p = 0.0223$; exploratory): on the averaged loss the two are equivalent within 0.01 nats at 10 seeds, and not at 5(TOST $p = 0.233$ with five seeds). On `val_loss_best` the two are not equivalent: $b = 6$ is lower by 0.0712 nats ($p < 10^{-4}$, exploratory). The two comparisons that are unresolved or below the power threshold on `val_loss_best` did not meet the trigger on val$_c$: 828 Kbit $b = 6$ against $b = 8$ has $p = 0.00427$, and at 4 Mbit the val$_c$ argmin $b = 4$ against $b = 6$ has $p < 10^{-4}$.

What A7 decides. At a budget where the argmin survives M1–M4, the disagreement with the larger-scale optima cannot be attributed to these three recipe choices, and the remaining candidates include scale and the quantizer, which A7 does not test, and the embedding treatment, the vocabulary and the data, which I held fixed. At a budget where it does not survive, the location of the optimum there depends on the recipe, and I report it as such. At 256 Kbit the argmin of the averaged loss, $b = 6$, survives M1, M2 and M3 (M2 and M3 with one seed per cell; M4 was not triggered there), so in the checks run none of these three recipe choices (end-of-training weight noise, the learning-rate rule for the cells that were tuned, the token-budget model) changed the argmin, and the disagreement with the larger-scale optima is not explained by them. At 828 Kbit $b = 6$ survives M1–M3 against its grid neighbours, but M4 leaves its comparison with $b = 3$ unresolved at 10 seeds per cell; both have lower val$_c$ means than every other bit-width, and since one of the two lies inside the 1–4-bit range of the larger-scale studies, A7 neither confirms nor removes the disagreement at this budget. At 4 Mbit the argmin depends on end-of-training averaging: it is $b = 6$ on `val_loss_best` and $b = 4$ on val$_c$, and the val$_c$ argmin $b = 4$ then survives M2 and M3 (M4 was not triggered there), so the location of the optimum at this budget is a property of the recipe as well as of the bit-width.

5.9 Scorecard of predictions

Table 11 scores the preregistered predictions P1–P5 and the two predictions, P6 and P7, recorded in Amendment A4.

P1 is *Partly correct*. The analysis scored "U-shaped" as an interior argmin, and the argmin is interior at every budget. However, the curves are not monotone on the low-precision side (section 5.1), and the predicted location is wrong: $b^* = 6$ at 256 Kbit and 4 Mbit, outside $[2, 4]$. At 828 Kbit the evidence is mixed. The null criterion fires between $b = 6$ and $b = 8$, and under ASPECT the argmin is

$b = 4$, inside the predicted range, but under TIGHTEST $b = 3$ and $b = 4$ are behind $b = 6$ by 0.0712 and 0.1101 nats ($p < 10^{-4}$ and $p < 10^{-4}$; exploratory). The verdict rests on the other two budgets.

P2 is *Wrong.* It predicted $b^* = 4/3/2$ at 256 Kbit/828 Kbit/4 Mbit. The observed values are 6/6/6, where the 828 Kbit value is null, so the lowest mean did not move across a 16-fold range of budget. The verdict does not depend on 828 Kbit: 256 Kbit and 4 Mbit alone miss the prediction. Under the other reading of the preregistration, with the pilot cells excluded (section 4), the lowest means are at $b = 6/6/4$: the lowest mean moves to lower b at 4 Mbit, to $b = 4$ rather than the predicted $b = 2$, and none of the three predicted values is hit, so P2 is wrong under both readings.

P3 is *Not testable as preregistered; correct under a post-hoc rule.* Its preregistered test, whether the argmin moves between the matched-parameter and matched-memory panels, cannot be computed on this grid, because the matched-parameter argmin is undefined (section 5.3). The analysis scores it by a rule written after the data existed: higher precision wins every matched-parameter pair (pairs within 12%), while the matched-memory argmin is interior. By that rule P3 holds, but the rule is met by the storage difference alone. The reasoning recorded with P3 relied on the P1 location, which failed.

P4 is *Correct.* All 142 of 142 complete runs beat the bigram floor of 3.1648. The worst cell mean is 2.8467, at 256 Kbit and $b = 2$. The smallest model, which P4 singled out as at risk (256 Kbit, $b = 8$, 32 768 parameters), reached 2.4267.

P5 is *Wrong.* It expected the null criterion to be most at risk at 256 Kbit and seed noise to be small relative to adjacent gaps at 4 Mbit. Instead, the null criterion fired at 828 Kbit. At 256 Kbit the separation was the largest of the three budgets, at 12.53 times $s_{\max}$. At 4 Mbit the null criterion did not fire, but the gap was only 1.41 times $s_{\max}$ and below the power threshold, so seed noise was not small relative to the gap, as P5 had expected it to be. The prediction fails because the null fired at 828 Kbit and because 256 Kbit, which it expected to be most at risk, was the best separated.

P6 and P7 were recorded after two budgets had been scored, when at least one 4 Mbit ternary sweep run (a second finished within about a minute of the commit) and the pilot run of $b = 6$ existed, and they are not part of the original preregistration. P6 concerns exactly the ternary cell and is post hoc; P7 concerns the 4 Mbit argmin over all bit-widths, and apart from the ternary runs and the pilot run no 4 Mbit run had started. P6 is *Correct* on its bound under TIGHTEST (0.0842) but missed its point estimate by +0.0210; under ASPECT the penalty is 0.1225, above the bound (3 seeds; section 5.2). P7 is *Correct with the pilot cell, wrong without (recorded when only the pilot run of $b = 6$ existed among its cells)*: with the pilot cell the argmin at 4 Mbit is $b = 6$, with a gap to $b = 4$ below the power threshold; without it, the argmin is $b = 4$ (section 5.1).

Of the five preregistered predictions, P4 was correct, P1 was partly correct, P2 was wrong and P5 was wrong, and P3 was not testable as preregistered and holds only under a scoring rule the analysis supplied. The two recorded after some 4 Mbit data existed both hold with qualifications: P6 only on its bound and only under TIGHTEST, and P7 only with the pilot cell included. Predictions P8–P10 for the second corpus are in section 6.

6 A Second Corpus: FineWeb-Edu

One objection to the TinyStories result is that TinyStories is simple text. Simple text may need less capacity, which favours spending bits on precision rather than on parameters and could by itself produce the argmin at $b = 6$ that TinyStories shows at every budget on `val_loss_best`. At 828 Kbit the null criterion fires, so that argmin is not established, and at 4 Mbit it is not robust to end-of-training averaging (sections 5 and 5.8). A second arm asks whether the argmin survives a change to text that is harder by a measured criterion, the bigram floor. Because the corpora

Table 11: Predictions, scored. P1–P5 were preregistered before any sweep run; P6 and P7 were recorded in Amendment A4, after two budgets had been scored and when at least one 4 Mbit ternary sweep run (a second finished within about a minute of the commit) and the 4 Mbit pilot run of $b = 6$ existed (section 4); neither is part of the original preregistration. P6 concerns the cell of that ternary run and is post hoc; P7 concerns cells none of which, apart from the pilot run, had started. ‡ The null criterion fires at that budget, so its lowest mean is not claimed as the optimum. Wording paraphrased from `PREREGISTRATION.md`.

	Prediction (recorded)	Measured	Verdict
P1	The curve is U-shaped in validation loss, with the optimum at intermediate precision, $b \in [2, 4]$. (2026-09-13)	Interior optimum at every budget; $b^* = 6/6$ at 256 Kbit and 4 Mbit, outside $[2, 4]$; at 828 Kbit null, and $b^* = 4$ under ASPECT. The grid locates a minimum only between the neighbouring grid values.	Partly correct
P2	The optimum moves to lower b as the budget grows: $b^* = 4/3/2$ at 256 Kbit/828 Kbit/4 Mbit. (2026-09-13)	$b^* = 6/6^{\ddagger}/6$: no movement; 0 of 3 predicted values hit. Without the pilot cells, $b^* = 6/6/4$.	Wrong
P3	The matched-parameter and matched-bits argmins differ. (2026-09-13)	No set of cells at one parameter count spans b, so the matched-parameter argmin of §6.2 is undefined. Post-hoc rule: higher precision wins 4/4 parameter-matched pairs (3 distinct higher-precision cells), at 3.22–4.97× the storage; at matched bits the optimum is interior at every budget.	Not testable as preregistered; correct under a post-hoc rule
P4	Every feasible configuration beats the bigram floor. (2026-09-13)	142/142 complete runs beat it; the worst cell mean is 2.8467.	Correct
P5	Seed sd is small relative to adjacent gaps at 4 Mbit; the null is most at risk at 256 Kbit. (2026-09-13)	The null fired at 828 Kbit, not at 256 Kbit; at 4 Mbit gap/$s_{\max} = 1.41$.	Wrong
P6	Ternary penalty at 4 Mbit ≈0.063 nats, and in any case below 0.12 (Amendment A4). (2026-09-16)	TIGHTEST: 0.0842, bound holds; point estimate off by +0.0210. ASPECT (3 seeds): 0.1225, above the bound.	Correct (post hoc; a 4 Mbit ternary sweep run was already finished)
P7	The optimum stays at $b = 6$ at 4 Mbit (Amendment A4). (2026-09-16)	$b^* = 6$ with the pilot cell; $b^* = 4$ without it.	Correct with the pilot cell, wrong without (recorded when only the pilot run of $b = 6$ existed among its cells)

also differ in domain and style, a change would show that the optimum depends on the text, not that it depends on difficulty specifically. The arm was added as Amendment A6 (appendix B) after the primary sweep had finished, so its design uses the TinyStories results; its predictions and interpretation rules were committed to my local git repository before any of its runs (section 7). It is secondary and cannot override the TinyStories result. Its design is given first and its results at the end of the section.

Only the corpus changes. The text is educational web text from FineWeb-Edu (Penedo et al., 2024). The pool is a prefix of one pinned shard of `HuggingFaceFW/fineweb-edu` (revision `87f09149ef47`, licence ODC-By): the first 359 595 documents, in stored order, of the file `sample/10BT/000_00000.parquet`. It is split with the same seed and fractions as TinyStories, each split is hashed and asserted at load, and its own byte-level BPE tokenizer with vocabulary 512 is trained on its train split alone. Architectures, optimizer, learning-rate rule, schedule and batch are identical, and each cell's token budget is computed by the rule of section 3.7 with TinyStories' corpus size, so every cell trains on exactly as many tokens as its TinyStories counterpart. The pool was sized to hold at least TinyStories' training tokens; its train split holds 830 708 553 tokens, 3.1% more than TinyStories' training split, so no cell sees more epochs than before. A corpus smaller than TinyStories would have required more epochs, and so changed the repetition along with the text. Nats per token are not comparable across corpora because the tokenizers differ, so every cross-corpus quantity is converted to bits per byte (Gao et al., 2020), $L\,\tau/\ln 2$, with L the loss in nats per token and τ the corpus's measured validation tokens per byte under its own tokenizer (0.432179 and 0.494814 tokens per byte). Measured before training in these units, the bigram floor (section 3.9), computed under each corpus's own tokenizer, is 1.973 bits per byte on TinyStories and 2.739 on FineWeb-Edu.

Budgets. The arm runs at 256 Kbit and 4 Mbit with 5 seeds per cell, the TIGHTEST shape rule and all seven bit-widths (256 Kbit at $b = 16$ remains infeasible). 828 Kbit is excluded because its TinyStories argmin is not established: the null criterion returns null between $b = 6$ and $b = 8$, and under the ASPECT rule the argmin moves to $b = 4$ (table 4), so a change there could not be interpreted. At the two budgets kept, the null criterion does not fire on TinyStories. At 4 Mbit the TinyStories argmin is the same under both shape rules. The Welch test against $b = 4$ rejects at $\alpha = 0.05$ ($p = 0.0317$, uncorrected, as preregistered), but the gap (0.0061 nats) is below the power threshold of 0.0085 (section 4 and table 2), and the argmin depends on including the convergence-pilot cell at $b = 6$ (section 4).

Predictions and interpretation rules. Within a corpus, cells are compared exactly as in the primary sweep (`val_loss_best`, null criterion and Welch tests; section 4). **P8**: at each budget the FineWeb-Edu argmin is at $b \leq 6$; the objection predicts $b < 6$, while $b = 6$ would mean the TinyStories optimum is not an artifact of easy text. **P9**: at each budget the ternary penalty (section 5.2), in bits per byte, is smaller on FineWeb-Edu than on TinyStories, where it is 0.5102 and 0.0842 nats per token at 256 Kbit and 4 Mbit (0.3181 and 0.0525 bits per byte). **P10**: every feasible cell beats the FineWeb-Edu bigram floor of 3.8371 nats per token; a cell that fails is flagged and excluded, not averaged in. Where the null criterion does not fire, an argmin equal to TinyStories' means the result holds on harder text at that budget, and a different one is reported as *the optimum depends on the text*, with its direction and size in bits per byte; this narrows the generality of the TinyStories claim without changing its measured values. Where the null criterion fires, no FineWeb-Edu argmin is claimed.

Half-budget residual check. The token budgets are set so that, under a gain model fitted to off-grid TinyStories calibration runs, every cell is left with the same predicted residual of 0.020 nats, which would then cancel in the ranking (section 3.7). The per-cell residual itself was not measured on TinyStories either; Amendment A7 measured half-budget gains for the cells around each TinyStories argmin (section 5.8). On FineWeb-Edu even the model is untested, and unequal residuals larger than the gap between two bit-widths would bias the ranking. After the arm completed, I retrained seed 0 of the argmin and of each adjacent bit-width (two cells if the argmin is at an end of the grid) at half its token budget, as a complete run whose cosine schedule spans the shorter length and which never replaces an arm result, and paired each gain $\mathrm{val}(T/2) - \mathrm{val}(T)$ against the arm's own seed-0 run. If the gains differ by more than the 5-seed mean gap between the argmin and its nearest neighbour, the FineWeb-Edu ranking at that budget is reported as sensitive to training length. The check halves the token budget rather than doubling it, because doubling would take the 4 Mbit cells past the 4-epoch cap and confound the check with repetition.

In the gain model, every gain from $T/2$ to T is $1/\rho = 1.54$ times the corresponding gain from T to $2T$ at the decay per doubling $\rho = 0.65$ used for the token budgets (section 3.7), and $2^{-\alpha_T} = 1.73$ times at the fitted token exponent alone ($\alpha_T = -0.793$, negative, in $g \propto T^{\alpha_T}$). The spread of gains grows by a factor between 1.54 and 1.73 while the gap does not, so the check is stricter than a doubling check by that factor. At $\rho = 0.65$ the model puts the residual left after T at $\rho/(1-\rho) = 1.86$ times the gain from $T/2$ to T, so a check that passes bounds the spread of residuals by 1.86 times the gap, not by the gap itself. These factors come from a model fitted on TinyStories; on FineWeb-Edu they are assumed. Each gain is also a single-seed difference, and the noise of this check has not been measured. Where the gap is comparable to the seed standard deviation, as at 4 Mbit on TinyStories (1.41 times $s_{\max}$ of eq. (6)), a sensitive verdict may reflect seed noise rather than unequal convergence.

Results. The checks of Amendment A7 (section 5.8) cover the TinyStories runs only, so the FineWeb-Edu cells are compared on `val_loss_best`, as preregistered, and its comparison with TinyStories uses the TinyStories values on the same metric. At 256 Kbit the FineWeb-Edu argmin is $b = 6$, as on TinyStories (table 12): its gap to the nearest adjacent bit-width, $b = 8$, is 0.0633 nats against $s_{\max} = 0.0074$, so the null criterion does not fire, and the Welch test gives $p < 10^{-4}$. By the preregistered rule the result holds on the harder text at this budget. At 4 Mbit the argmin moves to $b = 4$: its gap to $b = 6$ is 0.0125 nats against $s_{\max} = 0.0042$ (not null, $p = 0.0019$), above the arm's power threshold of 0.0085. By the same rule this is reported as *the optimum depends on the text*. The move is towards lower precision and more parameters, the direction the objection predicts; in bits per byte the two cells are at 1.6650 ($b = 4$) and 1.6740 ($b = 6$).

Two qualifications bound this reading. First, the TinyStories argmin it is compared with is itself not well established: its gap to $b = 4$ is below the power threshold, without the convergence-pilot cell the TinyStories minimum is at $b = 4$ (section 4), and on the averaged loss val_c the TinyStories argmin at 4 Mbit is already $b = 4$ (section 5.8), so the TinyStories argmin at this budget depends on end-of-training averaging. Second, the averaging check was not run on FineWeb-Edu, so whether the FineWeb-Edu argmin is robust to averaging is not measured.

All 30 runs at 256 Kbit and 35 at 4 Mbit beat the bigram floor, so no cell is excluded and P10 holds. P8 holds: both FineWeb-Edu argmins are at $b \leq 6$, so at neither budget did the harder text move the optimum to higher precision. P9 fails: the ternary penalty in bits per byte is smaller on FineWeb-Edu at 256 Kbit (0.3036 against 0.3181) but larger at 4 Mbit (0.0770 against 0.0525), so harder text does not shrink the ternary penalty at every budget.

The residual check finds neither ranking sensitive to training length: the spread of half-budget

Table 12: FineWeb-Edu arm beside TinyStories at the two budgets it shares with the primary sweep. The ternary penalty is in nats per token unless marked; bits per byte is the only unit compared across corpora. The power row states whether the gap between the argmin and its nearest neighbour is above or below the power threshold (section 4). The residual check of this section applies to FineWeb-Edu only; the TinyStories counterpart is the half-length check of Amendment A7 on val_c (section 5.8), whose verdicts are not sensitive at 256 Kbit and not sensitive at 4 Mbit.

	256 Kbit		4 Mbit	
	TinyStories	FineWeb-Edu	TinyStories	FineWeb-Edu
argmin b^*	6	6	6	4
null criterion	not null	not null	not null	not null
nearest gap vs. power threshold	above	above	below	above
ternary penalty	0.5102	0.4253	0.0842	0.1078
ternary penalty, bits/byte	0.3181	0.3036	0.0525	0.0770
runs beating bigram floor	30/30	30/30	35/35	35/35
sensitive to training length	—	not sensitive	—	not sensitive

gains across the retrained cells is 0.0093 nats at 256 Kbit against a gap of 0.0633, and 0.0050 at 4 Mbit against 0.0125. At both budgets the gap exceeds the spread by more than the factor 1.86, so under the gain model the implied spread of residuals is also below the gap; that model is assumed on this corpus, and each gain is a single-seed difference whose noise has not been measured.

7 Limitations

L1–L10 are the numbered limitations of `PREREGISTRATION.md`, released with the code, and each is cited below by its number. L1–L8 were preregistered before any sweep run; Amendment A5 added L9 and L10 after the sweep had begun (appendix B).

Scope. Only weights are quantized; activations stay in bf16 (L1). The dequantized weights are rounded to bf16 where they enter each matrix product, which displaces a weight by less than 0.5 of a step at $b = 8$ and less than 0.125 at $b = 6$ and never changes its code; its effect on the loss was measured only on 7 checkpoints (section 3.5). The budget excludes the KV cache (L2), activation memory and optimizer state, and only the cache is quantified. Computed for one sequence at full context with a bf16 cache, it exceeds the stored weight bits in 17 of 20 cells, by up to 16.9 times (256 Kbit, $b = 1.58$). It scales with depth times width, which tends to be larger at low b but is not monotone in b (table 14). Counting it would charge ternary more than the argmin at every budget, whereas the preregistration expected it to penalize high precision. I make no claim for a budget that includes it.

All primary-sweep models share one corpus, one 512-token tokenizer and a 256-token context (L4). At 256 Kbit the embedding's share of stored bits rises from 9.8% at $b = 1.58$ to 49.5% at $b = 8$, so the cost of high precision there may depend on the vocabulary, which I did not vary. A second corpus was run, at two of the three budgets (section 6); its results are given there and in section 8, not repeated here. The FineWeb-Edu runs were not averaged, so whether the moves of the 4 Mbit argmin on the two corpora share a cause is unmeasured, and relating them is a reading, not a preregistered comparison. How far the optimum moves on text beyond these two corpora is unmeasured.

Training is quantization-aware from the first step throughout (L5). I did not run post-training

quantization (Frantar et al., 2023; Dettmers and Zettlemoyer, 2023), QAT started from a full-precision checkpoint, or QAT after a full-precision phase, which is reported to help ternary (Nielsen et al., 2025; Liu et al., 2025), so nothing here measures them. Every quantized matrix has the same b (L9), and the RMSNorm gains are counted at 16 bits in every cell (Deviation D6), so the argmin is only the best of the uniform allocations tested; allocating bits unevenly at the same average is reported to do better for 8 of 11 language models quantized after training, with savings of up to 0.25 bits per parameter (Orr et al., 2025). If that saving transferred to my models (assumed), priced at my exploratory storage slope it would be 0.015 to 0.023 nats at $b = 6$ and $b = 4$, larger than my smallest preregistered nearest-neighbour gap (0.0061 nats). The tied embedding is quantized at b like every other matrix, whereas low-bit models are commonly reported with the embedding and output head at higher precision (Ma et al., 2024; Kaushal et al., 2025; Maskey et al., 2026); the effect of that choice on the ranking is unmeasured. The largest primary model has 2 616 320 parameters and the largest model overall, an ASPECT cell, 2 621 440 (L6). TinyStories was introduced to show that language models below 10M parameters can write fluent, consistent stories (Eldan and Li, 2023); I evaluate loss, not the quality of generated text, and make no claim about generation (L10). 9 of the 20 primary cells have width 32, equal to the head dimension, and so a single attention head. The primary cells train on 1 230 to 36 543 tokens per parameter, far above the roughly 20 tokens per parameter that Kumar et al. (2025) note is often used colloquially for compute-optimal pretraining; the optimal precision is reported to depend on training length or compute: Dremov et al. (2026) find this at fixed memory as training FLOPs grow, and Kumar et al. (2025) find it for the compute-optimal precision at fixed model size and for post-training quantization. The result is therefore specific to this regime. Budgets above 4 Mbit were excluded by the specification, and whether the minimum moves at larger scale is outside the scope of this study. No study I found compares precisions at fixed weight memory with quantization-aware training between about 3M and 30M parameters (post-training quantization has been measured from 19M parameters (Dettmers and Zettlemoyer, 2023)), so where, if anywhere, my optimum turns into the optima of 1–4 bits (6 bits at low training compute) reported above that size is unknown. Three budgets support no scaling law: the local log-log slope of the ternary penalty moves from -0.761 to -0.575 against stored bits, but the two slopes against non-embedding bits (-0.594 and -0.575) are not distinguishable ($z = +0.51$; section 5.2), and the two-point extrapolation recorded as P6 missed its point estimate.

Method. At 256 Kbit, precision and shape are one axis (L3): every cell has width 32, so depth and MLP ratio vary with b, and depth falls from 17 layers at $b = 1.58$ to 2 at $b = 8$. Where shape is free, it matters (table 4). At 828 Kbit the ASPECT rule changed the $b = 4$ loss by -0.1531 nats (the mean of 3 ASPECT seeds minus the mean of 5 primary seeds, as for every shape effect here) and moved the argmin from $b = 6$ to $b = 4$. The $\pm 5\%$ tolerance lets cells at one budget differ in storage by amounts that matter at these gaps: the ASPECT $b = 6$ cell stores -3.8% relative to the ASPECT argmin. Priced at the budget-to-budget slope of the best loss (0.360 nats per e-fold of storage; exploratory), that difference moves the ASPECT gap at 828 Kbit from 0.0420 to 0.0281 nats and the primary gaps from 0.0123 to 0.0159 (828 Kbit) and from 0.0061 to 0.0070 (4 Mbit), and no argmin changes (section 5.4). At 4 Mbit the shape effect largest in magnitude, -0.0214 nats (at $b = 2$), is larger in magnitude than the 0.0061 gap between the argmin and its nearest neighbour. The preregistration expected the ASPECT rule to bound the shape effect where shape is free (L3); two rules sample only two points in the space of shapes.

The quantizer is fixed, not tuned per bit-width (section 3.5): a per-tensor absmean scale, the Gaussian MSE-optimal step of a symmetric N-level quantizer by the criterion of Max (1960) (tabulated there up to $N = 36$, minimised directly for $b = 6$ and 8), for $2 \le b \le 8$ (on the grid used,

Table 13: Share of latent weights clipped by the quantizer's step rule, averaged over the final checkpoints of the 90 quantized primary runs, beside the share the same rule clips on a unit Gaussian. Excess is the difference in percentage points and ratio is trained over Gaussian share.

	trained share	Gaussian share	excess (points)	ratio
$b = 1.58$	20.8%	23.1%	−2.32	0.9
$b = 2$	8.05%	7.4%	+0.648	1.09
$b = 3$	3.22%	2.43%	+0.784	1.32
$b = 4$	1.38%	0.816%	+0.561	1.69
$b = 6$	0.685%	0.0879%	+0.597	7.79
$b = 8$	0.271%	0.00808%	+0.263	33.6

its Gaussian mean squared error is within 0.61% of that grid's own minimum) and the BitNet b1.58 rule for ternary, with $b = 16$ the unquantized bf16 control. I did not measure the sign of the effect of its departures from optimality on the ranking. The ternary rule has a higher error on a Gaussian than the MSE-optimal ternary step; I argue that this does not change what the model can represent (section 3.5) and did not measure its effect on the loss. The 2-bit grid, $\{-2, -1, 0, 1\}\Delta$, has a level at zero and one more negative level than positive ones, where the symmetric 4-level quantizer has neither, a difference of alphabet that Liu et al. (2025) report to favour the symmetric grid at 2 bits (learned-scale QAT, mostly fine-tuned from full-precision checkpoints at 125M parameters and above); its effect on loss in my setting was not tested. On the trained latent weights, the share of weights that the Gaussian-derived step clips falls at every step up in b from $b = 2$ to $b = 8$ (table 13; averaged over the final checkpoints of the 90 quantized primary runs). The two measures of its excess over the Gaussian reference single out different bit-widths: the excess in percentage points is largest at $b = 3$, the ratio is largest at $b = 8$, and the share is below the Gaussian one for ternary. These shares count clipped weights; I did not measure the loss the clipping costs at each b, so they do not show which b the step rule handicaps more. In prior work the quantization function and its range can change scaling-law conclusions below 4 bits (Liu et al., 2025), and the quantizer fitting together with the gradient estimator moves the Pareto-optimal precision for weights and activations (Panferov et al., 2025a); per-channel or learned scales could change my ranking in either direction, and I did not test them. One untuned learning-rate rule serves every cell. A7's M2 and A8 retrained the cells around each argmin over a range of learning-rate multipliers, and the best rate lies inside the tested range in every one of them (unbracketed cells at the three budgets: 0, 0 and 0). With every such cell at its own best rate the argmin on the averaged weights is unchanged at each budget ($b = 6$, $b = 6$ and $b = 4$ at 256 Kbit, 828 Kbit and 4 Mbit; one seed per cell at 256 Kbit, three at the other budgets). At 828 Kbit it is unchanged with each cell at its own best, while on the 5 multipliers shared by all of its cells the argmin is $b = 3$ at $\times 2$ and $b = 6$ at the others (section 5.8). The best multiplier is above $\times 1$ in every one of those cells, so the rule's rate is not the best for any cell tested, and absolute losses are not those of tuned models; the cells outside those groups are not tested. Ternary is counted at 1.58 bits (L7), inside the tolerance of both $\log_2 3$ and five-trits-per-byte packing, and across all runs the counted bits miss the budget by at most 4.15% (appendix D). The one scale per quantized matrix is not counted in eq. (2); there are $4L + 1$ of them, so counting them would charge deeper cells more, which at 256 Kbit are the low-b ones. At 32 bits each they would add at most 0.84% to the counted bits (section 3.1).

The uniform residual of 0.020 nats is a prediction of the token-budget model (section 3.7), not a per-cell measurement. It cancels within a budget only if it is equal across cells, which the primary sweep assumes and does not measure: the model was fitted on 8 gains at three sizes, each trained at

a single bit-width, so size and precision are confounded in it; it has a fit error (mean absolute log residual 0.106); and most cells lie beyond its calibrated token range. A7's M3 measures half-length gains, not the residual itself, and only for the cells around each argmin, by retraining them at half length; it relates them to the residual through the gain model (section 5.8). Its preregistered verdict is "not sensitive", "not sensitive" and "not sensitive" at the three budgets, but at 828 Kbit the spread of the half-length gains, 0.0233, exceeds the 0.0059-nat gap on averaged weights between $b = 6$ and $b = 3$, so training length could reorder those two. Repetition also rises as b falls: at 4 Mbit, from 2.38 epochs at $b = 16$ to 3.99 at $b = 1.58$. So does training compute: the comparison is not compute-matched, and a 4 Mbit ternary run costs 16.8 times the compute of a bf16 run ($6\times$parameters$\times$tokens; section 3.7), a difference that favours low b. Weight storage is counted at fixed width: averaged over the runs at each bit-width, an order-0 entropy coder would admit at least 11.4% more parameters at $b \in \{2, 3, 4, 6, 8\}$, whose code entropy is below b, and 0.619% more for ternary, whose codes carry 1.570 bits (section 3.1), so the fixed-width budget favours ternary relative to an entropy-coded one (the $b = 16$ control was not measured).

Statistics. At 828 Kbit the preregistered null criterion fires: the gap from the argmin to $b = 8$, 0.0123, is below the largest seed standard deviation among the argmin and its two neighbours, 0.0201 (Welch $p = 0.212$). That comparison is unresolved at five seeds, not shown to be a tie: its minimum detectable effect (at $p \leq 0.05$ and 80% power), 0.0299, is more than twice the gap (section 5.1). On averaged latent weights the 828 Kbit argmin is again $b = 6$, but $b = 3$ is 0.0059 nats behind it ($p = 0.305$); with 10 seeds per cell (M4) $b = 3$ minus $b = 6$ is -0.0009 ($b = 3$ slightly ahead; $p = 0.832$), unresolved by A7's rule (Welch $p > 0.01$) at a minimum detectable effect of 0.0164 nats (at $p \leq 0.01$ and 80% power), so $b = 3$ and $b = 6$ are not separated and I name no best bit-width between them. The Welch 95% interval of that gap is -0.0097 to 0.0080, which excludes true differences larger than about 0.01 nats, and an exploratory two one-sided tests analysis, with a margin of 0.01 nats chosen after these data existed, gives a 90% interval of -0.0081 to 0.0063 ($p = 0.0223$), inside the margin: on the averaged loss the two are equivalent within that margin at 10 seeds per cell and not at five. Read together with the shape result (a reading, not a preregistered analysis), the data support no single optimum there. At 4 Mbit the criterion does not fire, and the Welch tests of the argmin against both neighbours, preregistered without correction, reject at the preregistered level ($p = 0.0317$ against $b = 4$). Because the claim that the argmin beats both neighbours is an intersection–union test, it needs no correction across the two, though b^* is the observed argmin, so the nominal level is not guaranteed. The gap, 0.0061, is below the power threshold of 0.0085. The Welch 95% interval on it, 0.0007 to 0.0114, reaches nearly to zero, and an equivalence test (exploratory; the margin of 0.01 nats was not preregistered) does not place it within that margin. In a sensitivity analysis that was not preregistered, a Holm correction over the family of the 6 adjacent tests across the three budgets gives $p = 0.0634$ (table 2). On the two unselected measures the gap is above the threshold: on the test split, read once on the final-step weights, it is 0.0112 ($p = 0.00393$, threshold 0.0094), and on the final validation loss it is also above (section 5.6); the preregistered verdict on `val_loss_best` is unchanged. Without the pilot cell the 4 Mbit argmin moves to $b = 4$ (table 6), and on averaged latent weights (M1) it is $b = 4$, ahead of $b = 6$ by 0.0230 nats ($p < 10^{-4}$): by A7's preregistered rule the 4 Mbit argmin depends on end-of-training averaging, and $b = 6$ there is not a robust optimum (section 5.8). The seed standard deviation measures initialization and data order within one protocol only. M1 evaluated every final checkpoint on fresh validation batches as well: the fixed validation set gives each cell a loss lower by 0.0058 to 0.0094, 0.0091 to 0.0109 and 0.0106 to 0.0119 nats at the three budgets (section 5.8), so it shifts the cells at one budget by amounts that differ by the width of each of those ranges; its

effect on each individual gap is not tested. The primary metric, the minimum validation loss, is a minimum over noisy evaluations on one fixed validation set, and is biased downward more at low b (section 5.5); under the final validation loss the argmin is unchanged (appendix E).

Provenance. Every complete run reports device `cuda` and one framework build (PyTorch 2.14.0, CUDA 13.0), and, as far as the records show, on one training host (the results table has no host field; `results/environment/pip-freeze-training-host.txt` records one environment). A repeat of the same seed gave identical logged train and validation losses at every evaluation, and identical result columns, wherever I could compare one: in 9 pairs of runs that the two shape rules make identical (3 cells, three seeds each, 1 of the pairs including a run that was stopped and resumed; appendix C), and in a 20-step repeat of one shape on random tokens on the training GPU (`results/bench_training_host_cuda.json`). I did not repeat every run, so determinism is verified for those cells and not for all, and residual nondeterminism of this device (L8) is not excluded for the others. 40 of the 142 complete runs, every 256 Kbit run and the 828 Kbit primary runs at $b \in \{1.58, 2\}$, carry a commit marked `-dirty`, whose uncommitted changes were not recorded. A line-by-line comparison of the committed training code at that commit with the later commits that produced rows shows differences only in bookkeeping. The only evidence on the effect of the uncommitted changes on training is of two kinds. The first is pairs of runs whose loss histories were identical (9 of 9; appendix C), but only the pairs of the 828 Kbit ternary cell (seeds 0 to 2) include a run from a `-dirty` commit, so they cover three of the 40 dirty runs. The second is one partial rerun, which is recorded in a commit message and a checklist item and cannot be checked from the repository. Neither covers every dirty cell, so for the other cells that effect is not bounded. A re-evaluation of two dirty checkpoints, recorded in a checklist item, concerns evaluation, not training. Some runs were trained under more than one code state. The preregistration and its amendments A1–A8 were committed to a local git repository only. The first external timestamps were made after the TinyStories sweep had finished, for A7 (2026-09-27) and A8 (2026-10-09); none is earlier (section 4 and appendix B).

8 Discussion and conclusion

Choosing a precision under a memory budget. Everything in this paragraph holds only within the measured range: weight budgets from 256 Kbit to 4 Mbit, primary-sweep models of at most 2 616 320 parameters, TinyStories at vocabulary 512 and context 256, one bit-width for every weight matrix, including the tied embedding (RMSNorm gains counted at 16 bits), per-tensor absmean scaling with a straight-through estimator, quantization-aware training from the first step and one untuned learning-rate rule. The best-supported result concerns the ends of the grid. Ternary and 2-bit weights are worse than the argmin at every budget, as is bf16 wherever it is feasible, each at $p < 10^{-4}$ in exploratory Welch tests (table 2), and on `val_loss_best` they remain the three worst-ranked cells under the ASPECT shape rule at the two budgets where it was run (table 4). Within the measured range, a budget spent on many coarse weights ($b = 1.58$ or 2) or on few bf16 weights gives a higher loss than the argmin. The argmin of the primary (TIGHTEST) sweep is $b = 6$ at every budget (fig. 1), but its standing differs. It is decisive at 256 Kbit, with a gap to $b = 8$ of 12.53 times $s_{\max}$, although there precision, depth and MLP ratio cannot be varied independently (section 7). At 4 Mbit the null criterion does not fire and the argmin survives the change of shape rule, but its gap to $b = 4$ (0.0061 nats, 95% interval 0.0007 to 0.0114) is below the power threshold of 0.0085 nats, which is twice the larger seed standard deviation of the two cells. The preregistration gives 80% as the power of a five-seed Welch test against such a gap; that figure

is stated, not measured. Without the pilot cell the argmin there is $b = 4$ (table 6). At 828 Kbit the criterion returns null between $b = 6$ and $b = 8$, and the ASPECT rule moves the argmin to $b = 4$, so no single bit-width is identified there. My reading, which is not a preregistered analysis, is a range: on `val_loss_best` the best uniform precision I measured lies between $b = 4$ and $b = 8$.

Recipe checks (Amendment A7). Amendment A7 tests these readings against the training recipe (section 5.8), on the metric val_c, the validation loss of the quantized average of the latent weights over a 400-step continuation at the final learning rate. At 256 Kbit the argmin survives every check that applies: it is $b = 6$ on val_c (not null, 0.0555 nats ahead of $b = 4$, $p < 10^{-4}$), with each argmin-group cell at its best learning-rate multiplier ($b = 6$) and at half the training length (not sensitive), the last two checks on one seed per cell. At 828 Kbit val_c separates $b = 6$ from $b = 8$ (0.0352 nats, $p = 0.00427$) but not from $b = 3$ (0.0059 nats, $p = 0.305$). With 10 seeds per cell, $b = 3$ minus $b = 6$ is -0.0009 nats ($b = 3$ slightly ahead; $p = 0.832$), unresolved by the rule of Amendment A7 (Welch $p > 0.01$ at 10 seeds per cell), at a minimum detectable effect of 0.0164 nats (at $p \leq 0.01$ and 80% power). An exploratory equivalence test, with a margin of 0.01 nats chosen after these data existed, places the difference within that margin at 10 seeds per cell (TOST $p = 0.0223$, 90% interval -0.0081 to 0.0063, at the 0.05 level; section 5.8). The half-length check, whose spread of gains (0.0233 nats) is below the gap to $b = 8$ but above the gap to $b = 3$, cannot order them either. On the averaged metric the 828 Kbit range is therefore $b = 3$ to $b = 6$, with $b = 4$ the argmin under ASPECT, and I name no single bit-width there.

At 4 Mbit the argmin is not robust to averaging: it moves from $b = 6$ on `val_loss_best` to $b = 4$ on val_c, 0.0230 nats ahead of $b = 6$ ($p < 10^{-4}$), and makes the same move under ASPECT ($b = 4$). On val_c the argmin is $b = 4$ at every learning-rate multiplier that all its group cells ran and $b = 4$ with each cell at its own best, and the ranking is not sensitive to training length. The 4 Mbit argmin therefore depends on end-of-training averaging, a property of the recipe; the preregistered primary result there remains $b = 6$.

Which comparison answers which question. Matched-parameter and iso-memory readings of the same cells rank precision differently (section 5.3). At parameter counts matched within 12%, the higher-precision member wins 4 of 4 pairs, each time with 3.22–4.97 times the storage (table 3 and fig. 2); at different precisions, matching parameters cannot hold memory fixed, so each pair combines a change of budget with a change of precision. That comparison answers how much a network of fixed size loses when its weights are stored more coarsely, the relevant question when the parameter count is fixed by something other than memory, such as an existing checkpoint or a per-token compute budget. The iso-memory comparison answers how to spend a weight-storage budget, when weight storage is the binding constraint and there is no slower tier of memory to spill into; where compute, the KV cache or memory bandwidth binds instead, other axes apply (Dremov et al., 2026; Kim et al., 2026; Alizadeh et al., 2024; Yuan et al., 2024). On my runs the iso-memory comparison places the lowest mean in the interior. Pair C shows that the two readings can reverse: $b = 16$ beats $b = 3$ at matched parameters, while $b = 3$ beats $b = 16$ at matched storage at both budgets where both exist; section 5.3 gives the earlier reports of the same difference between parameter count and storage. To them I add that a binary translation Transformer matches the loss of a float one that has fewer parameters but more bits (Zhang et al., 2023). Pair C is at a scale below 3M parameters, with QAT from step 0. Reports that ternary models match 16-bit ones (Ma et al., 2024) are made at matched parameter count and tokens and answer the first question, not the second.

Relation to fixed-bit scaling studies. The studies that compare precisions at fixed weight memory with QAT start at 30M non-embedding parameters; my models have at most about 3M parameters in total, tied embedding included. Their weight-only comparisons put the best precision at 2 bits (Panferov et al., 2025a,b), at 1 bit (Frantar et al., 2025), or at 1 to 2 bits depending on the number format and on whether embedding bits are counted (Maskey et al., 2026). The weight-and-activation Pareto frontier of QuEST lies near 4 bits (Panferov et al., 2025a). Kumar et al. (2025) report no weight-only optimum, but their fitted weights-only law ranks 3-bit weights above 4-bit and 4-bit above 6-bit at fixed weight memory inside the 3–8-bit part of its 3-to-12-bit weight sweep (my derivation from their Eq. 3 and the constants of their Table 2). Dremov et al. (2026) find the best precision falling as training compute grows, starting from 6 bits at low compute in their experiments (1, 2, 4 and 6 bits). My lowest mean, at $b = 6$ on the primary metric at every budget, with ternary and 2-bit weights worse than the argmin at every budget (table 2), lies at a higher precision than the weight-only comparisons report. Dettmers and Zettlemoyer (2023), post-training quantizing released checkpoints from 19M to 176B parameters at 3–8 bits with a 16-bit baseline, find 4 bits optimal at fixed total model bits for almost every model family and scale they test (3 bits is slightly, not significantly, better for BLOOM-176B), and describe bit-level scaling as "mostly independent of scale", with 3 bits the exception. My optimum disagrees with theirs inside the 3–8-bit range their study covers, decisively at 256 Kbit. The two results could be reconciled if the optimum depends on scale below their smallest model, whose smallest bit budget is more than an order of magnitude above my largest, or if it depends on the method: quantization-aware training from step 0 against post-training quantization, and a per-tensor absmean quantizer against their block-wise quantizers (absmax-normalized blocks with the 16-bit constants counted, with integer, float, quantile or dynamic-exponent data types). The corpus, the tokenizer and the training regime also differ. I do not measure which. The ranking of Frantar et al. (2025) follows from effective-parameter multipliers that they assume independent of model size, and that of Maskey et al. (2026) from the saturating form they fit once embedding bits are not counted; their appendix, which counts the 16-bit embedding, puts the uniform-format optimum at 2 bits for an 8 GB budget and notes that it shifts upward at very small budgets. ParetoQ's ternary and 2-bit models on the accuracy–size frontier (3-bit often surpasses 4-bit as well) come from QAT started mostly from full-precision checkpoints, and in its Figure 3 (accuracy against parameter count, not memory) such QAT beats training from scratch at every bit-width it tests (1 to 4 bits) (Liu et al., 2025). Taken at face value, my results are inconsistent with these fits holding unchanged below 3M parameters: either the effective-parameter multiplier of a precision depends on model size between my scale and theirs, or the difference lies in the quantizer, the embedding treatment, the data or the recipe. The recipe checks of Amendments A7 and A8 do not remove the conflict at 256 Kbit, where it was decisive: there $b = 6$ survives averaging, learning-rate tuning with the optimum bracketed in every argmin-group cell, and half-length training (the last two on one seed per cell), all on val_c. The remaining candidates therefore include scale and the quantizer, neither of which I vary, and the embedding treatment and the vocabulary, which I held fixed; the corpus is only partly tested, since FineWeb-Edu gives the same 256 Kbit argmin (section 6). At 4 Mbit averaging moves the optimum toward lower b, from $b = 6$ to $b = 4$, and on the averaged metric the conflict inside the shared bit range narrows there: $b = 4$ is the optimum of Dettmers and Zettlemoyer (2023) and ranks above $b = 6$ as in the fitted law of Kumar et al. (2025), but $b = 3$, which that law ranks above $b = 4$ and $b = 6$, does not rank above $b = 6$ on val_c: it is 0.0383 nats behind the argmin, against 0.0230 for $b = 6$. At 828 Kbit $b = 3$ and $b = 6$ remain unresolved by the rule of Amendment A7 (Welch $p > 0.01$), so that budget neither confirms nor removes the conflict. The disagreement with the 1–2-bit optima is unchanged by averaging: on val_c ternary and 2-bit weights trail the argmin by 0.3821 and 0.3944 nats at 256 Kbit and by 0.0487 and 0.0965 at 4 Mbit. Those cells were in no

argmin group, so their learning rate was not tuned, and that part of the recipe is untested for them.

Relation to large-scale ternary results. Ternary parity has been reported for models trained from scratch, starting at 3 billion parameters, at matched parameter count and tokens, against FP16, with the embedding at full precision (Ma et al., 2024), and ternary models are reported to beat higher-precision ones of the same bit size above one billion parameters (Kaushal et al., 2025); ternarizing pretrained models by fine-tuning still leaves a gap at that scale (Chen et al., 2024), and an independent 1B-parameter reproduction of BitNet b1.58 (a baseline in that paper), trained for one epoch on Wikipedia and, in a second setting, on a 10B-token FineWeb sample, has a higher WikiText-2 test perplexity than the full-precision model in both settings (Zhao et al., 2025). At my scale ternary is worse than a $b = 8$ model of the same size: the 828 Kbit ternary cell and the 4 Mbit $b = 8$ cell share architecture (10×64, MLP ratio 4), learning rate, seeds and training tokens, differ only in b, and reach 2.0688 and 1.3747 respectively (tables 3 and 14). At matched memory the ternary penalty relative to the argmin falls from 0.5102 to 0.2108 to 0.0842 nats between 256 Kbit and 4 Mbit, with local log–log slopes of −0.761 and −0.575 against stored bits (on TIGHTEST shapes, without an uncertainty interval; section 5.2). That is the direction expected if ternary becomes competitive at much larger scale. It is also what a scale-free effective-parameter model predicts, with no crossover (Panferov et al., 2025a; Kumar et al., 2025), as does the main-text law of Maskey et al. (2026), which does not count embedding bits: a nat penalty that falls with budget is expected even if ternary's per-weight capacity is constant. Three budgets cannot distinguish these. Reconciling my result with reports that 1–2 bits are optimal at fixed memory above 30M non-embedding parameters would require the optimum to move from $b = 6$ to 1–2 bits somewhere between my largest model and about 30M parameters, a range in which I found no study that compares precisions at fixed weight memory with QAT. In budget-equivalent terms, the loss of the ternary cell is the loss that the lowest-mean curve reaches at 0.59 of the budget at 828 Kbit and 0.78 at 4 Mbit (exploratory; section 5.2), so its relative efficiency rose between these two budgets and stayed below parity. The fall stayed inside the bound of Amendment A4, recorded after the first two budgets when a 4 Mbit ternary run already existed (P6: 0.0842 against 0.12 nats, post hoc), but the observed penalty exceeded the point prediction by +0.0210 nats. Three budgets fix neither a functional form nor a crossover, and I do not extrapolate to one (appendix B). Changing the shape rule does not remove the penalty. At the two budgets where the ASPECT rule was run (3 seeds per cell), the penalty is larger under it, 0.2533 and 0.1225 nats against 0.2108 and 0.0842 under TIGHTEST, and its local slope against stored bits between those budgets is −0.456 against −0.575, so the size of the penalty and its rate of fall depend on the shape rule. The ternary shape itself differs between the rules only at 4 Mbit, where the ASPECT 13×128 cell reaches 1.4304 against 1.4127 for the TIGHTEST 11×160 cell. At 828 Kbit both rules build it as 10×64, and the larger penalty comes from a better argmin cell (section 5.4). Two differences from the large-scale setting are unrelated to scale. I quantize and count the tied embedding at b, whereas large low-bit models keep the embedding and output head at higher precision (Ma et al., 2024; Kaushal et al., 2025; Maskey et al., 2026); and I train ternary from step 0 with one learning-rate rule, whereas ternary is reported to want a higher learning rate (Zhu et al., 2024; Kaushal et al., 2025) and to gain from a full-precision phase first (Nielsen et al., 2025). My penalties are those of my recipe. On the averaged metric val_c (M1) the penalty is smaller and still falls with budget: 0.3821, 0.1150 and 0.0487 nats, against 0.5102, 0.2108 and 0.0842 on `val_loss_best`, each measured from its own metric's argmin ($b = 4$ on val_c at 4 Mbit). Averaging gains most at low precision, an effect specific to low precision by the A7 criterion at both budgets with a $b = 16$ control (the $b = 16$ gain is 0.138 and 0.257 of the ternary gain, against a threshold of one half), so part of the penalty on `val_loss_best` is removed by end-of-training averaging;

what that part consists of I did not measure. The ternary cell was in no argmin group, so M2 and A8 did not tune its learning rate, and its penalty at its best learning rate is unmeasured. Every argmin-group cell preferred a higher learning rate than the rule gives (for $b = 6$, $\times 2$ at 4 Mbit and $\times 8$ at 828 Kbit). The rule's rate is therefore not the best one for the cells tested, and I cannot assume that it is for the ternary cells, to which the reports of a higher preferred rate remain relevant.

Shape and precision (exploratory). At matched memory, the depth–width rule can change which of two adjacent precisions wins. At 828 Kbit, switching the rule moves $b = 4$ by -0.1531 nats (the mean of the 3 ASPECT seeds minus the mean of the 5 primary seeds, as for every shape effect here) and moves the argmin from $b = 6$ to $b = 4$ (3 seeds per ASPECT cell); at 4 Mbit, shape effects exceed the gap between $b = 6$ and $b = 4$, the largest in magnitude being -0.0214 nats (at $b = 2$), but do not move the argmin (section 5.4). At the two budgets where both shape rules were run, ternary, 2-bit and bf16 weights are worse than the lowest-mean cell under both rules, and on `val_loss_best` they are also the three worst-ranked cells under both. Every cell pairs a precision with a different shape, so a precision-only comparison that fixes one shape rule can mislocate b^*. Veldanda (2026) reports that the baseline shape accounts for at least part of a published ternary margin at tiny scale, and Levine et al. (2020) predict a width-dependent transition beyond which added depth is inefficient. I measure shape effects at each precision at fixed memory, under two shape rules.

What would change the conclusion. *Larger budgets.* P2 predicted that the optimum would move to lower b as the budget grew; it was wrong (observed 6/6/6). The ternary penalty fell at every step. The gap to $b = 4$ is 0.0061 nats at 4 Mbit. The 2- and 3-bit gaps changed by less than 0.01 nats between 828 Kbit and 4 Mbit (0.1493 to 0.1419 and 0.0712 to 0.0645 nats), and the minimum on `val_loss_best` stayed where it was. On val_c it is $b = 4$ at 4 Mbit against $b = 6$ at 256 Kbit, so on that metric it did move one grid step toward lower b. Only budgets above 4 Mbit can show whether, and where, the minimum moves toward the optima of 1–4 bits (6 bits at low training compute) reported at fixed memory at 19M parameters and above. *Other corpora.* TinyStories is simple text, which may favour precision over parameter count. The FineWeb-Edu arm (section 6) tests this at 256 Kbit and 4 Mbit against predictions P8–P10, recorded before its first run: at 256 Kbit the argmin is $b = 6$ on both corpora (0.0633 nats per token ahead of $b = 8$ on FineWeb-Edu, $p < 10^{-4}$); at 4 Mbit it moves from $b = 6$ to $b = 4$, 0.0125 nats per token ahead of $b = 6$ ($p = 0.0019$, a gap above the power threshold of 0.0085 nats). P8, an argmin at $b \leq 6$, holds. P9 fails: the ternary penalty is smaller on FineWeb-Edu at 256 Kbit (0.3036 against 0.3181 bits per byte) but larger at 4 Mbit (0.0770 against 0.0525). P10 holds, with 30 of 30 and 35 of 35 runs beating the bigram floor, and the half-budget residual check finds neither ranking sensitive to training length. The 256 Kbit optimum therefore holds on both corpora, while at 4 Mbit the optimum depends on the text, moving to lower b on the harder corpus. $b = 4$ is the bit-width val_c selects on TinyStories at 4 Mbit, although the FineWeb-Edu cells were compared on `val_loss_best`; and because the corpora differ in domain and style as well as in difficulty, the arm does not show that difficulty is what moves it. A second property of the corpus, its finite size, makes lower-b cells repeat the training set more (at 256 Kbit, 2.11 epochs at $b = 1.58$ against 1.49 at $b = 8$). Up to four epochs of repetition are reported to cost little at 2.8B to 8.7B parameters, in a study whose scaling law is fitted on 7M to 9B parameters (Muennighoff et al., 2023); whether that holds at my sizes is unmeasured, and the FineWeb-Edu arm does not test it, because no cell there sees more epochs than on TinyStories. *Other quantizers.* I used one scheme. For $2 \leq b \leq 8$ it is a per-tensor absmean scale with the step-to-standard-deviation ratio that minimises the Gaussian mean squared error of a symmetric

uniform quantizer by the criterion of Max (1960); his tables stop at 36 levels, which covers $b = 2$, 3 and 4, and for $b = 6$ and 8 I minimised the same criterion directly. On the asymmetric grid used, the Gaussian mean squared error of this step is within 0.61% of that grid's own minimum (section 7). Ternary uses the BitNet rule, $b = 16$ is unquantized bf16, and every weight matrix has the same b. In prior work the quantization function (grid, range clipping, learnable or statistics-based scales) can change scaling-law conclusions below 4 bits (Liu et al., 2025), the quantizer fitting together with the gradient estimator moves the Pareto-optimal precision (Panferov et al., 2025a), and the gradient estimator changes accuracy at a fixed bit-width (Défossez et al., 2022); the granularity of the scales and learned step sizes (Esser et al., 2020) could also matter. Uneven allocation of a bit budget has been reported to beat uniform allocation at equal average bits for 8 of 11 language models (Orr et al., 2025). Any of these could help some bit-widths more than others and move the argmin, and none was tested. An iso-memory comparison of such schemes would have to count the bits their scales occupy; mine leaves its one scale per quantized matrix uncounted (section 7).

Conclusion. At a fixed weight-storage budget between 256 Kbit and 4 Mbit, with per-tensor absmean QAT from step 0 and one learning-rate rule, ternary, 2-bit and bf16 weights have a higher loss than the lowest-mean bit-width wherever they are feasible, and the argmin of the primary sweep is $b = 6$ at every budget, decisively only at 256 Kbit, where precision, depth and MLP ratio cannot be varied independently. At 828 Kbit the null criterion fires and the ASPECT rule moves the argmin to $b = 4$. At 4 Mbit the argmin is $b = 6$ with the pilot cell and $b = 4$ without it. This interior optimum below 3M parameters lies at a higher precision than the optima of 1–4 bits reported at fixed memory at 19M parameters and above; 6 bits, the largest width in their experiments (1, 2, 4 and 6 bits), is reported as optimal only at low training compute (Dremov et al., 2026).

At 256 Kbit the optimum is unchanged under the recipe checks of Amendments A7 and A8, so the disagreement there is not removed by averaging, by tuning the learning rate of the cells around the argmin, or by halving the training length (the last two on one seed per cell); the candidates that remain include scale, the quantizer and the embedding treatment. At 828 Kbit $b = 3$ and $b = 6$ remain unresolved at 10 seeds per cell (minimum detectable effect 0.0164 nats), and I name no best bit-width between them. At 4 Mbit the argmin depends on end-of-training averaging: it moves to $b = 4$ on the averaged metric, at every learning-rate multiplier that all its group cells ran ($b = 4$) and with each cell at its own best ($b = 4$). On that metric this removes the disagreement with the 4-bit optimum of Dettmers and Zettlemoyer (2023) there, though not with the 3-over-4 ordering of the fitted law of Kumar et al. (2025), and not with the 1–2-bit optima, since ternary and 2-bit weights remain worse on val_c at every budget. On FineWeb-Edu the optimum holds at 256 Kbit ($b = 6$) and moves to $b = 4$ at 4 Mbit, so by the preregistered rule it depends on the text at that budget, though on TinyStories the averaged metric also selects $b = 4$ there and the FineWeb-Edu runs were not averaged.

Of the original preregistered predictions, P2 was wrong and P5 wrong, P1 was partly correct, P4 was correct, and P3 was not testable as preregistered; P6 and P7, added in Amendment A4 after two budgets, were correct and correct with qualifications, P6 only on its bound under TIGHTEST and post hoc, and P7 only with the pilot cell (table 11). At parameter counts matched within 12%, higher precision wins 4 of 4 pairs, but it always holds more memory, so that comparison cannot separate precision from storage. The ternary penalty shrinks in nats across the measured range; budgets above 4 Mbit are outside this study. Apart from the single read of the test split, all reported losses are validation losses. On that read the final-step argmin is the `val_loss_best` argmin at every budget and the null criterion returns the validation verdicts, but at 4 Mbit the gap to $b = 4$ is above the power threshold, where on `val_loss_best` it is below; the averaged-weight argmin is the

val_c argmin at every budget, $b = 4$ at 4 Mbit, and on the averaged weights the null criterion does not fire at 828 Kbit, while $b = 3$ and $b = 6$ remain unresolved there (section 5.6).

Use of AI assistance

Claude (Anthropic), including through the Claude Code command-line tool, assisted with the analysis and experiment code, the literature search and the checking of references against their sources, and with drafting and editing the manuscript text. I set the research question and the study design, specified the content of the preregistration, decided each amendment to it and the timing of the single test-set evaluation, supervised all experiments on my own hardware, and am responsible for every result, figure and claim in this paper.

References

AmirAli Abdolrashidi, Lisa Wang, Shivani Agrawal, Jonathan Malmaud, Oleg Rybakov, Chas Leichner, and Łukasz Lew. Pareto-optimal quantized ResNet is mostly 4-bit. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 3085–3093, June 2021. doi: 10.1109/CVPRW53098.2021.00345.

Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12562–12584, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.678. URL https://aclanthology.org/2024.acl-long.678/.

Atome LM contributors. Atome LM: a tiny ternary language model for microcontroller deployment. Software, version v2.0, Zenodo, 2026. URL https://github.com/TilelliLab/atome-lm. GitHub repository TilelliLab/atome-lm, not peer-reviewed; results in FRONTIER.md, HONEST_RESULTS.md, ab_results.json and extended_results.json. Version DOI 10.5281/zenodo.21027125; concept DOI 10.5281/zenodo.20518644. Accessed 2026-09-27.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013. URL https://arxiv.org/abs/1308.3432.

Luca Bertinetto, João F. Henriques, Samuel Albanie, Michela Paganini, and Gül Varol. Preface. In *NeurIPS 2020 Workshop on Pre-registration in Machine Learning*, volume 148 of *Proceedings of Machine Learning Research*, page i. PMLR, 2021. URL https://proceedings.mlr.press/v148/bertinetto21a.html.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In A. Smola, A. Dimakis, and I. Stoica, editors, *Proceedings of Machine Learning and Systems*, volume 3, pages 747–769, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/file/0184b0cd3cfb185989f858a1d9f5c1eb-Paper.pdf.

Mengzhao Chen, Chaoyi Zhang, Jing Liu, Yutao Zeng, Zeyue Xue, Zhiheng Liu, Yunshui Li, Jin Ma, Jie Huang, Xun Zhou, and Ping Luo. Scaling law for quantization-aware training. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 18317–18336. PMLR, 2026. URL https://proceedings.mlr.press/v306/chen26gl.html.

Tianqi Chen, Zhe Li, Weixiang Xu, Zeyu Zhu, Dong Li, Lu Tian, Emad Barsoum, Peisong Wang, and Jian Cheng. TernaryLLM: Ternarized large language model. *arXiv preprint arXiv:2406.07177*, 2024.

Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments, 2018. URL https://arxiv.org/abs/1806.08295.

Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. BinaryConnect: Training deep neural networks with binary weights during propagations. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/hash/3e15cc11f979ed25912dff5b0669f2cd-Abstract.html.

Alexandre Défossez, Yossi Adi, and Gabriel Synnaeve. Differentiable model compression via pseudo quantization noise. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL https://openreview.net/forum?id=DijnKziche.

Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.10668.

Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 7750–7774. PMLR, 2023. URL https://proceedings.mlr.press/v202/dettmers23a.html.

Aleksandr Dremov, David Grangier, Angelos Katharopoulos, and Awni Hannun. Compute-optimal quantization-aware training. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2509.22935.

Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1902.08153.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Elias Frantar, Utku Evci, Wonpyo Park, Neil Houlsby, and Dan Alistarh. Compression scaling laws: Unifying sparsity and quantization, 2025. URL https://arxiv.org/abs/2502.16440.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. URL https://arxiv.org/abs/2101.00027.

Moritz Herrmann, F. Julian D. Lange, Katharina Eggensperger, Giuseppe Casalicchio, Marcel Wever, Matthias Feurer, David Rügamer, Eyke Hüllermeier, Anne-Laure Boulesteix, and Bernd Bischl. Position: Why we must rethink empirical research in machine learning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 18228–18247. PMLR, 2024. URL https://proceedings.mlr.press/v235/herrmann24b.html.

Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual Conference on Computational Learning Theory (COLT '93)*, pages 5–13, Santa Cruz, California, USA, 1993. ACM Press. doi: 10.1145/168304.168306.

Jake M. Hofman, Angelos Chatzimparmpas, Amit Sharma, Duncan J. Watts, and Jessica Hullman. Pre-registration for predictive modeling, 2023. URL https://arxiv.org/abs/2311.18807.

Hong Huang, Decheng Wu, Rui Cen, Guanghua Yu, Zonghang Li, Kai Liu, Jianchen Zhu, Peng Chen, Xue Liu, and Dapeng Wu. Tequila: Trapping-free ternary quantization for large language models. In *The Fourteenth International Conference on Learning Representations (ICLR)*, pages 55791–55810, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/5b555804d495321df2e3208cc27f4fbc-Abstract-Conference.html.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

Ayush Kaushal, Tejas Vaidhya, Arnab Kumar Mondal, Tejas Pandey, Aaryan Bhagat, and Irina Rish. Surprising effectiveness of pretraining ternary language model at scale. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=TJo6aQb7mK. arXiv title: "Spectra: Surprising Effectiveness of Pretraining Ternary Language Models at Scale".

Junhyuck Kim, Ethan Ewer, Taehong Moon, Jongho Park, and Dimitris Papailiopoulos. Not all bits are equal: Scale-dependent memory optimization strategies for reasoning models. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=b6qQmQ2F13. arXiv:2510.10964.

Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling laws for precision. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=wg1PCg3CUP.

Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017. doi: 10.1177/1948550617697177.

Yoav Levine, Noam Wies, Or Sharir, Hofit Bata, and Amnon Shashua. Limits to depth efficiencies of self-attention. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 22640–22651. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/ff4dfdf5904e920ce52b48c1cef97829-Paper.pdf. arXiv:2006.12467 ("The Depth-to-Width Interplay in Self-Attention").

Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks, 2016. URL https://arxiv.org/abs/1605.04711v2. 1st International Workshop on Efficient Methods for Deep Neural Networks (EMDNN), NIPS 2016; arXiv version v2, whose three authors differ from the five of v3 (2022).

Margaret Li, Sneha Kudugunta, and Luke Zettlemoyer. (Mis)Fitting scaling laws: A survey of scaling law fitting techniques in deep learning. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=xI71dsS3o4. arXiv version titled "(Mis)Fitting: A Survey of Scaling Laws".

Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. MCUNet: Tiny deep learning on IoT devices. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 11711–11722, 2020.

Zechun Liu, Changsheng Zhao, Hanxian Huang, Sijia Chen, Jing Zhang, Jiawei Zhao, Scott Roy, Lisa Jin, Yunyang Xiong, Yangyang Shi, Lin Xiao, Yuandong Tian, Bilge Soran, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. ParetoQ: Improving scaling laws in extremely low-bit LLM quantization. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. doi: 10.52202/085713-3055. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/83b17fb3369b1effa97ca5409526b02e-Abstract-Conference.html.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1608.03983.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1711.05101.

Nicholas Lourie, Kyunghyun Cho, Karen Ullrich, and Sanae Lotfi. Small-scale experiments: Are we there yet?, 2026. URL https://arxiv.org/abs/2608.11859.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024. URL https://arxiv.org/abs/2402.17764.

Sohir Maskey, Constantin Eichenberg, Johannes Messner, and Douglas Orr. 1-bit wonder: Improving QAT performance in the low-bit regime through K-means quantization. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 86931–86953. PMLR, 2026. URL https://proceedings.mlr.press/v306/maskey26a.html.

J. Max. Quantizing for minimum distortion. *IRE Transactions on Information Theory*, 6(1):7–12, March 1960. doi: 10.1109/TIT.1960.1057548.

Bert Moons, Koen Goetschalckx, Nick Van Berckelaer, and Marian Verhelst. Minimum energy quantized neural networks. In *2017 51st Asilomar Conference on Signals, Systems, and Computers*, pages 1921–1925. IEEE, 2017. doi: 10.1109/ACSSC.2017.8335699.

Niklas Muennighoff, Alexander Rush, Boaz Barak, Teven Le Scao, Nouamane Tazi, Aleksandra Piktus, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 50358–50376. Curran Associates, Inc., 2023. doi: 10.52202/075280-2191.

Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization, 2021. URL https://arxiv.org/abs/2106.08295.

Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16318–16330. PMLR, 2022.

Jacob Nielsen and Peter Schneider-Kamp. BitNet b1.58 reloaded: State-of-the-art performance also on smaller networks. In Ana Fred, Allel Hadjali, Oleg Gusikhin, and Carlo Sansone, editors, *Deep Learning Theory and Applications (DeLTA 2024)*, Communications in Computer and Information Science, pages 301–315. Springer Nature Switzerland, 2024. doi: 10.1007/978-3-031-66705-3_20.

Jacob Nielsen, Peter Schneider-Kamp, and Lukas Galke. Continual quantization-aware pre-training: When to transition from 16-bit to 1.58-bit pre-training for BitNet language models? In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13483–13493, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.694. URL https://aclanthology.org/2025.findings-acl.694/.

Hamish Ogilvy. Variable bit-width quantization: Learning per-group precision for "bigger-but-smaller" language models, 2026. URL https://arxiv.org/abs/2607.02893.

Douglas Orr, Luka Ribar, and Carlo Luschi. Optimal formats for weight quantisation, 2025. URL https://arxiv.org/abs/2505.12988. v3, February 2026.

Andrei Panferov, Jiale Chen, Soroush Tabesh, Mahdi Nikdan, and Dan Alistarh. QuEST: Stable training of LLMs with 1-bit weights and activations. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47820–47836. PMLR, 2025a. URL https://proceedings.mlr.press/v267/panferov25a.html.

Andrei Panferov, Alexandra Volkova, Ionut-Vlad Modoranu, Vage Egiazarian, Mher Safaryan, and Dan Alistarh. Unified scaling laws for compressed representations. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025b. doi: 10.52202/085713-5053. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/ddd7eae51dcd8bbfc936e5b19433d339-Abstract-Conference.html.

Tim Pearce and Jinyeop Song. Reconciling Kaplan and Chinchilla scaling laws. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=NLoaLyuUUF.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the

finest text data at scale. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 37, pages 30811–30849, 2024. doi: 10.52202/079017-0970.

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 100535–100570. Curran Associates, Inc., 2024. doi: 10.52202/079017-3189.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, pages 525–542. Springer, 2016. doi: 10.1007/978-3-319-46493-0_32.

Egor Romanyukov, Timofey Novikov, Timur Shokarov, Elizaveta Zorkina, Anastasia Palienko, and Stepan Dergachev. Does step law transfer to small-scale language models? an empirical recalibration below 59M parameters, 2026. URL https://arxiv.org/abs/2609.27581.

Manuele Rusci, Alessandro Capotondi, and Luca Benini. Memory-driven mixed low precision quantization for enabling deep network inference on microcontrollers. In I. Dhillon, D. Papailiopoulos, and V. Sze, editors, *Proceedings of Machine Learning and Systems*, volume 2, pages 326–335, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/file/0c8abcf158ed12d0dd94480681186fda-Paper.pdf.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL https://aclanthology.org/P16-1162/.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.

Christian Brandt Thomassen. Mapping the schedule × bit-width boundary in sub-100M quantisation-aware training, 2026. URL https://arxiv.org/abs/2605.25966. Submitted for review at TMLR.

Tristan Thrush, Christopher Potts, and Tatsunori Hashimoto. Improving pretraining data using perplexity correlations. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=huuKoVQnB0. Preregistration in arXiv v1 (9 Sep 2024); preregistered results in v2 (10 Mar 2025).

Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine*

Learning Research, pages 48630–48656. PMLR, 2024. URL https://proceedings.mlr.press/v235/tseng24a.html.

Gautam Veldanda. Baseline shape decides the verdict: A controlled re-examination of ternary language models at 60K parameters, 2026. URL https://arxiv.org/abs/2609.29397.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023. doi: 10.48550/arXiv.2310.11453.

Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. HAQ: Hardware-aware automated quantization with mixed precision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8612–8620, 2019. URL https://openaccess.thecvf.com/content_CVPR_2019/html/Wang_HAQ_Hardware-Aware_Automated_Quantization_With_Mixed_Precision_CVPR_2019_paper.html.

B. L. Welch. The generalization of 'Student's' problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947. doi: 10.1093/biomet/34.1-2.28.

Linjie Yang and Qing Jin. FracBits: Mixed precision quantization via fractional bit-widths. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 10612–10620, 2021. doi: 10.1609/aaai.v35i12.17269. Issue 12.

Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. LLM inference unveiled: Survey and roofline model insights, 2024. URL https://arxiv.org/abs/2402.16363.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/1e8a19426224ca89e83cef47f1e7f53b-Paper.pdf.

Yichi Zhang, Ankush Garg, Yuan Cao, Łukasz Lew, Behrooz Ghorbani, Zhiru Zhang, and Orhan Firat. Binarized neural machine translation. In *Advances in Neural Information Processing Systems*, volume 36, pages 60099–60116, 2023. doi: 10.52202/075280-2627.

Kaiyan Zhao, Tsuguchika Tabaru, Kenichi Kobayashi, Takumi Honda, Masafumi Yamazaki, and Yoshimasa Tsuruoka. Direct quantized training of language models with stochastic rounding. In Hung-yi Lee and Tongliang Liu, editors, *Proceedings of the 17th Asian Conference on Machine Learning*, volume 304 of *Proceedings of Machine Learning Research*, pages 1150–1165. PMLR, 2025. URL https://proceedings.mlr.press/v304/zhao26b.html.

Rui-Jie Zhu, Yu Zhang, Steven Abreu, Ethan Sifferman, Tyler Sheaves, Yiqiao Wang, Dustin Richmond, Sumit Bam Shrestha, Peng Zhou, and Jason K. Eshraghian. Scalable matmul-free language modeling, 2024. URL https://arxiv.org/abs/2406.02528. v7, 25 Jul 2025.

A Per-Cell Results

Table 14 lists every cell of the experiment. The upper block is the primary sweep: the TIGHTEST shape rule and 5 seeds per cell. Every headline number of the primary sweep (the means and standard deviations of `val_loss_best`) comes from this block. The lower block is the secondary shape-robustness sweep at 828 Kbit and 4 Mbit: the ASPECT shape rule and 3 seeds per cell. It was declared secondary before any run, and it cannot override the primary sweep (section 4).

The columns are defined as follows. $L{\times}d$ is the number of layers by the residual width d, with $d/32$ attention heads. The MLP expansion ratio is chosen by the search from $\{2, 2.5, 3, 3.5, 4\}$. The parameter count covers the quantized weights: the attention and MLP projections of every layer and the tied embedding, counted once. *Weight bits* is the storage counted by the accounting code, which is b bits per quantized weight plus 16 bits for each of the $(2L+1)d$ RMSNorm gains. It is always the *actual* bit count, never the target. *Err* is the signed relative error (weight bits $- B)/B$ against the budget B. The search rejects any shape outside $\pm 5\%$, and training raises an error before its first step if the model it builds misses that bound. *Emb* is the tied embedding's share of the stored bits. *Tokens* is the length of each run, set per cell by the uniform-residual rule of Amendment A3 (appendix B). It depends only on the parameter count, so every seed of a cell trains on the same number of tokens. *Ep.* is the number of passes over the 805 873 616 unique training tokens. *Mean* and *sd* are the mean and seed standard deviation, over the n seeds, of `val_loss_best` in nats per token (section 3.9). The test losses of the primary cells, read once, are in table 5.

Two features of the table need a note. First, the cell at 256 Kbit and $b = 16$ does not exist: at the minimum width the tied embedding alone costs 100.0% of the budget (appendix F, D2). Second, the two shape rules sometimes select the same architecture: at 828 Kbit for $b = 1.58$ and $b = 16$, and at 4 Mbit for $b = 8$. In those cells the secondary runs use seeds 0–2 and repeat the primary runs with the same seeds, and they reproduce them exactly (appendix C). Their secondary means differ from the primary means only because they average 3 seeds instead of 5.

Table 14: Every cell. Shape is layers × width; mlp is the MLP expansion ratio; weight bits are the stored bits actually counted (never the target); err is the signed error against the budget; emb is the tied embedding's share of the stored bits; tokens are training tokens per run (identical across seeds); ep. is epochs over the unique training tokens; FLOPs is the training compute of one run, 6×params×tokens, which is not matched across cells. Mean and sd are `val_loss_best` over n seeds.

Budget	b	$L{\times}d$	mlp	params	weight bits	err	emb	tokens	ep.	FLOPs	mean	sd	n
Primary sweep, TIGHTEST shape rule													
256 Kbit	1.58	17×32	2	155 648	263 844	+0.65%	9.8%	1 702 166 528	2.11	$1.6 \times 10^{+15}$	2.8416	0.0127	5
	2	9×32	4	126 976	263 680	+0.59%	12.4%	1 625 718 784	2.02	$1.2 \times 10^{+15}$	2.8467	0.0135	5
	3	6×32	3.5	83 968	258 560	−1.37%	19.0%	1 480 818 688	1.84	$7.5 \times 10^{+14}$	2.6175	0.0101	5
	4	4×32	4	65 536	266 752	+1.76%	24.6%	1 400 274 944	1.74	$5.5 \times 10^{+14}$	2.4489	0.0076	5
	6	3×32	2.5	44 032	267 776	+2.15%	36.7%	1 280 049 152	1.59	$3.4 \times 10^{+14}$	2.3315	0.0048	5
	8	2×32	2	32 768	264 704	+0.98%	49.5%	1 197 441 024	1.49	$2.4 \times 10^{+14}$	2.4267	0.0055	5
	16	infeasible: the tied embedding alone is 100.0% of the budget											
828 Kbit	1.58	10×64	4	524 288	849 880	+0.24%	6.1%	2 239 037 440	2.78	$7.0 \times 10^{+15}$	2.0688	0.0051	5
	2	4×96	3	417 792	849 408	+0.18%	11.6%	2 127 167 488	2.64	$5.3 \times 10^{+15}$	2.0073	0.0062	5
	3	5×64	4	278 528	846 848	−0.12%	11.6%	1 941 110 784	2.41	$3.2 \times 10^{+15}$	1.9292	0.0050	5
	4	17×32	3.5	207 872	849 408	+0.18%	7.7%	1 817 051 136	2.25	$2.3 \times 10^{+15}$	1.9680	0.0201	5
	6	12×32	3	139 264	848 384	+0.06%	11.6%	1 659 994 112	2.06	$1.4 \times 10^{+15}$	1.8580	0.0122	5
	8	2×64	2.5	106 496	857 088	+1.09%	30.6%	1 562 443 776	1.94	$1.0 \times 10^{+15}$	1.8702	0.0160	5
	16	3×32	4	53 248	855 552	+0.91%	30.6%	1 336 147 968	1.66	$4.3 \times 10^{+14}$	2.1294	0.0081	5

Budget	b	$L \times d$	mlp	params	weight bits	err	emb	tokens	ep.	FLOPs	mean	sd	n
4 Mbit	1.58	11×160	2.5	2 616 320	4 192 666	−0.04%	3.1%	3 218 440 192	3.99	$5.1 \times 10^{+16}$	1.4127	0.0034	5
	2	20×96	3.5	2 076 672	4 216 320	+0.52%	2.3%	3 054 927 872	3.79	$3.8 \times 10^{+16}$	1.4705	0.0047	5
	3	18×96	2	1 376 256	4 185 600	−0.21%	3.5%	2 784 002 048	3.45	$2.3 \times 10^{+16}$	1.3931	0.0041	5
	4	9×96	4	1 044 480	4 207 104	+0.31%	4.7%	2 615 934 976	3.25	$1.6 \times 10^{+16}$	1.3346	0.0027	5
	6	3×160	2	696 320	4 195 840	+0.04%	11.7%	2 387 148 800	2.96	$1.0 \times 10^{+16}$	1.3286	0.0043	5
	8	10×64	4	524 288	4 215 808	+0.51%	6.2%	2 239 037 440	2.78	$7.0 \times 10^{+15}$	1.3747	0.0043	5
	16	7×64	2	262 144	4 209 664	+0.37%	12.5%	1 914 732 544	2.38	$3.0 \times 10^{+15}$	1.5250	0.0060	5
Secondary sweep, ASPECT shape rule													
828 Kbit	1.58	10×64	4	524 288	849 880	+0.24%	6.1%	2 239 037 440	2.78	$7.0 \times 10^{+15}$	2.0683	0.0057	3
	2	8×64	4	425 984	869 376	+2.54%	7.5%	2 136 506 368	2.65	$5.5 \times 10^{+15}$	2.0629	0.0075	3
	3	6×64	3	278 528	848 896	+0.12%	11.6%	1 941 110 784	2.41	$3.2 \times 10^{+15}$	1.9258	0.0017	3
	4	5×64	2.5	217 088	879 616	+3.74%	14.9%	1 834 942 464	2.28	$2.4 \times 10^{+15}$	1.8150	0.0032	3
	6	10×32	4	139 264	846 336	−0.18%	11.6%	1 659 994 112	2.06	$1.4 \times 10^{+15}$	1.8570	0.0102	3
	8	7×32	4	102 400	826 880	−2.48%	15.9%	1 548 681 216	1.92	$9.5 \times 10^{+14}$	1.8948	0.0027	3
	16	3×32	4	53 248	855 552	+0.91%	30.6%	1 336 147 968	1.66	$4.3 \times 10^{+14}$	2.1289	0.0111	3
4 Mbit	1.58	13×128	4	2 621 440	4 197 172	+0.07%	2.5%	3 219 849 216	4.00	$5.1 \times 10^{+16}$	1.4304	0.0048	3
	2	13×128	2.5	1 982 464	4 020 224	−4.15%	3.3%	3 023 077 376	3.75	$3.6 \times 10^{+16}$	1.4491	0.0014	3
	3	12×96	4	1 376 256	4 167 168	−0.65%	3.5%	2 784 002 048	3.45	$2.3 \times 10^{+16}$	1.3906	0.0027	3
	4	10×96	3.5	1 062 912	4 283 904	+2.14%	4.6%	2 626 289 664	3.26	$1.7 \times 10^{+16}$	1.3258	0.0021	3
	6	9×96	2	712 704	4 305 408	+2.65%	6.8%	2 399 698 944	2.98	$1.0 \times 10^{+16}$	1.3078	0.0042	3
	8	10×64	4	524 288	4 215 808	+0.51%	6.2%	2 239 037 440	2.78	$7.0 \times 10^{+15}$	1.3758	0.0057	3
	16	6×64	2.5	253 952	4 076 544	−2.81%	12.9%	1 901 068 288	2.36	$2.9 \times 10^{+15}$	1.5304	0.0020	3

B Amendment Trail

The preregistration was committed as `de30cb9` at 2026-09-13 15:06 UTC, before any run of the experiment. The first sweep run started at 2026-09-14 06:37 UTC, according to the training host's sweep log. After that commit the preregistration was amended eight times. Each amendment was committed to a local git repository, and its commit timestamp is the evidence for when it was made; git timestamps are set by the committer's machine. The history of that repository is not released. The commit identifiers in this paper, and in the `git_sha` column of the results file, name commits of the unreleased original and do not resolve in the released repository. What can be checked there is in the folder `timestamps/`: for each externally timestamped commit it holds a manifest of SHA-256 hashes, the OpenTimestamps proof of that manifest, and, under `timestamps/files/`, the file versions the hashes cover (I checked that every hash matches its file). The text of `de30cb9` is also there, marked as not timestamped. Statements that compare commits (the training-code comparison across the `-dirty` runs, appendix C) cannot be reproduced from the released repository. The first external timestamps (OpenTimestamps) cover commits of three dates, all after the TinyStories sweep had finished: 2026-09-27 (A7 with the correction to A4, and the §17 note to A7), 2026-10-02 (the §18 note to A7, at 21:52 UTC; the heading of §18 and its timestamp file carry the local date 2026-10-03) and 2026-10-09 (A8 and its §20 note). Nothing earlier was externally timestamped. Table 15 gives each commit and the data that existed when it was made. The paragraphs below give the trigger and the change. All times are UTC.

Amendments A4 to A8 were written after results existed. No amendment changes the primary metric or the null criterion of section 4. Apart from the edits made by A1 and A2 to the paragraphs on compute and on the convergence pilot, the committed text of the original document is unchanged. A3 removed early stopping, so that the preregistered definition of an undertrained run (item 6 of Section 6) is met by every run; the code's replacement flag came after the sweep began (appendix F).

Table 15: Amendments to the preregistration, with the data that existed when each was committed. A1 and A2 share one commit. A6 was corrected two minutes after its first commit, and the second hash is that correction.

ID	Commit	Committed (UTC)	Data in existence at the commit
A1, A2	`d957083`	2026-09-13 15:13	Smoke tests only; no grid configuration trained past 60 steps.
A3	`5cfc7ab`	2026-09-14 06:35	Pilot runs (seed 0, old caps) of three grid cells, including 4 Mbit $b = 6$; off-grid calibration and seed-noise runs. No sweep run: the sweep started at 06:37.
A4	`b008193`	2026-09-16 07:46	Every 256 Kbit and 828 Kbit primary run. On the training host the first 4 Mbit run ($b = 1.58$, seed 0) had also finished; see the text.
A5	`e4e462c`	2026-09-16 09:30	As for A4, plus the second 4 Mbit ternary run ($b = 1.58$, seed 1).
A6	`d5a2d55`, `26539a9`	2026-09-26 17:35	Both TinyStories sweeps complete and analysed (finalised 2026-09-25 12:03). No FineWeb-Edu run.
A7	`f00fcd5`	2026-09-27 04:54	As for A6, with the FineWeb-Edu arm in progress, plus averaging measurements on seven local seed-0 checkpoints, which motivated A7 and are not among its results. No A7 run. The preregistration notes §17 (`81820ff`, 2026-09-27 06:24) and §18 (`6539d58`, 2026-10-02 21:52) precede the A7 runs they concern.
A8	`14debeb`	2026-10-09 15:54	Both TinyStories sweeps and the FineWeb-Edu arm complete; every M1 continuation, 69 of the 72 M2 runs and all 24 M3 runs. No A8 run. The preregistration note §20 (`7acde5a`, 17:01) precedes every A8 run.

A1: compute ceiling removed. The preregistration capped projected GPU-hours, and the grid validator failed above that cap. A1 removed the cap. Wall-clock time is still projected, but it is never enforced and no run is refused on cost. The reason given was that compute was not the binding constraint, so a cap could only have forced a smaller experiment. Removing a cost cap can only allow more runs or more tokens. A1 left the one protocol limit in place: the token caps could be adjusted once, before the main sweep, and never after.

A2: convergence pilot widened. The pilot that was to test early stopping moved from one held-out cell to one held-out cell per budget. The three budgets had different token caps, so a calibration at one budget said little about the others. A2 kept the rule of one adjustment made before the sweep, and it stated that pilot cells stay excluded from the headline curve. The pilot runs themselves contribute to no table or figure, but the three pilot cells, which are grid cells, were retrained and are included in every table and figure (appendix F).

A3: convergence protocol replaced. The specification's protocol was early stopping on validation loss with a fixed patience. Each budget had a fixed token cap, and any run that reached its cap was marked undertrained. The pilot ran one seed of each pilot cell under these caps, on device `mps` (appendix C), and showed that this protocol could not work as written. No pilot run stopped early. In all three, validation loss was still improving close to the end: the best evaluation came at 98%, 96% and 100% of training at 256 Kbit, 828 Kbit and 4 Mbit. A3 attributes this to the schedule, a cosine decay to the minimum learning rate that keeps lowering the loss until late in

training. It also argues, without measuring it, that stopping earlier would have cut the schedule while the learning rate was still high and left a worse model. The uniform caps had a second defect. At a fixed budget, low b buys more parameters, so under uniform caps the low-precision cells received the fewest tokens per parameter, a confound that handicaps them.

A3 replaced the protocol. Early stopping was removed and every run now completes its cosine schedule. Each cell's token budget is solved so that every cell ends at the same predicted convergence residual. The token budgets come from a gain model fitted on off-grid calibration runs, with a residual target of 0.020 nats and a cap of 4 epochs (section 3.7). The target is the tightest one that every cell can reach within the cap, to a resolution of 10^{-4} nats (computed from the gain model; no run was trained to test it). The largest primary cell (4 Mbit, $b = 1.58$) needs 3.99 epochs, and the larger ASPECT ternary cell at 4 Mbit also stays within the cap, at an epoch count just below 4 that table 14 prints as 4.00.

I rejected two alternatives. Token budgets proportional to parameter count left the residual dependent on model scale: the residuals predicted from the measured gain between the last two doublings of the token budget are 0.547, 0.105 and 0.045 nats at the three off-grid scales, at equal tokens per parameter (predicted, not measured directly). The preregistration's A3 record gives the third value as 0.046; the values here are recomputed unrounded from `results/scale_free.json` and `results/calibration.json`. The residual is largest for the smallest model, and at a fixed budget the smaller models are the high-precision cells, so this rule would have biased the comparison against high precision. A tail-drift convergence test was also rejected as a convergence criterion before any sweep run, because under a cosine schedule it measures the shape of the schedule rather than convergence. The seed standard deviation used to plan the resolution of the null criterion was measured at the same time, on an off-grid configuration: 0.0075 nats over 5 seeds. The minimum resolvable gap of 0.0150 nats is twice that standard deviation, the approximate gap at which a two-sample Welch test with 5 seeds per cell has 80% power as stated in the preregistration; a simulation gives 77% (table 2).

Section 5.4 of the preregistration permitted one adjustment to the token caps before the main sweep. A3 was made before the main sweep, but it changed more than the caps. It also removed early stopping. It did not redefine an undertrained run; the code kept the rejected tail-drift test as the flag until after the sweep had begun (appendix F). Under the token budgets of A3, every reported loss is predicted by the fitted model to sit about 0.020 nats above its fully converged value. The residual is not a per-cell measurement. Because the fit predicts the same residual for every cell, it is *assumed* to cancel in comparisons at equal budget.

A4: predictions P6 and P7. Two budgets had been scored when A4 was written. The ternary penalty (section 5.2) had fallen from 0.5102 nats at 256 Kbit to 0.2108 nats at 828 Kbit. Those two points fit penalty $\propto B^{-0.753}$ exactly, and two points cannot test a power law. A4 therefore added two predictions for 4 Mbit. They are labelled as post hoc with respect to the first two budgets and are never presented as part of the original preregistration. P6 predicts, as its point estimate, a ternary penalty of about 0.0632 nats (A4 records it to three decimal places; the value here is recomputed by the analysis), and in any case below 0.12 nats, which is the claim under test. P7 predicts that the argmin stays at $b = 6$, contradicting P2. At 828 Kbit the preregistered null criterion returns null between $b = 6$ and $b = 8$, and the argmin moves under the ASPECT rule (section 5). A4 also recorded an extrapolation of the exponent to a break-even budget far outside the measured range, and stated that it would not be relied on.

A4 says that no 4 Mbit run had started and that the results file held no 4 Mbit row. The copy of the results file committed with A4 held every 256 Kbit primary run, 27 of the 35 828 Kbit primary

runs ($b = 8$ had 2 seeds and $b = 16$ had 0), and no 4 Mbit row. On the training host the remaining 828 Kbit runs had also finished, and the host's copy of the results file, and its sweep log, held a 4 Mbit row. The log records the start time of each training session and the wall-clock time of each run, and from these the end time of every run can be reconstructed. The reconstruction omits the time between runs, so it gives lower bounds: the first 4 Mbit run ($b = 1.58$, seed 0) finished at or after 03:28 UTC, and the second ($b = 1.58$, seed 1) at or after 07:47 UTC, against the commit at 07:46 UTC. The reconstruction is consistent with the interruption step logged for the third run at that session's deadline, so the omitted time is short: the first run finished several hours before the commit and the second about a minute after it. I do not treat that margin as settling the order of the second run and the commit, because the clocks of the two machines are not known to agree to that precision. No 4 Mbit sweep run at any other bit-width had started. Neither prediction can be scored from ternary runs alone: P6 needs the loss of the 4 Mbit argmin and P7 needs its location. Still, one input to P6 existed when P6 was recorded, and the repository does not show whether it had been seen. A 4 Mbit result at $b = 6$ also existed: the convergence pilot's single run of that cell under the superseded protocol, with a best validation loss of 1.3670 nats. P7 predicts that this cell is the argmin. The outcomes are in table 11. P6 is correct: the observed penalty of 0.0842 nats is below the bound, and the point estimate is off by +0.0210 nats. Under the alternative ASPECT shape rule (3 seeds per cell) the 4 Mbit penalty is 0.1225 nats, above the bound. P7 is correct with the retrained pilot cell at $b = 6$ included; the argmin's gap to $b = 4$ is below the power threshold. Without that cell, as the preregistration's pilot clause reads, the lowest 4 Mbit mean is at $b = 4$ and P7 is wrong (section 4 and appendix F).

A5: limitations L9 and L10. A5 added two limitations after the sweep had begun, and they are presented as such (section 7). A5 gives their source as a first pass at the literature and a comparison of generated text with the dataset's own paper. It states the observation on the text only as a summary: correct local grammar and drifting meaning. I did not archive a sample, so that observation cannot be checked from the repository. L9 records that every weight in a model carries the same b. A5 cites only Ogilvy (2026) for the gain of non-uniform allocation: a 131M-parameter model at 1.82 mean bits that beats a 55M FP16 model with 3.8 times less storage, which is a comparison with FP16, not with a uniform allocation at the same average bits. I add Orr et al. (2025), who report that tensor-wise non-uniform allocation at the same average bits improves the KL divergence after post-training quantization for 8 of 11 language models. These reported gains mean that any best b named here is to be read as the best *uniform* allocation, not the best allocation. L10 calls 10M parameters the dataset's "coherence threshold" and says that its authors place fluent and consistent story generation below it (Eldan and Li, 2023). I checked this against that paper. The phrase "below 10 million parameters" is theirs, but "coherence threshold" and the reading of 10M as a size below which generation fails are not. Their claim is that models below 10M parameters, or with a single transformer block, trained on TinyStories still produce fluent and consistent stories; 10M is the upper end of the parameter count in the first of those two conditions, and the single-block models they show reach 21M. Within that range they report that quality grows with size. Their 1M-parameter, 8-layer model answers none of their factual completion tests correctly and often produces ungrammatical or senseless sentences. Of their example completions from models of about 2.5M and 21M parameters they write that models of this size sometimes produce less coherent completions but can also often match or surpass the ones shown, and that a model larger by an order of magnitude consistently produces coherent and relevant ones. My models have 32 768 to 2 621 440 parameters, from well below the smallest model they study (1M parameters) to about their 2.5M-parameter model. L10's practical consequence is unchanged: I evaluate loss, not generated

text, and make no claim about generation. L10 names the primary 4 Mbit ternary cell (2 616 320 parameters) as the largest model of the grid. The ASPECT ternary cell at 4 Mbit, trained after L10 was written, is larger (2 621 440 parameters) and is the largest. The off-grid calibration model that anchors the gain model has 3 047 424 parameters and is not a grid cell.

A6: a second corpus. A6 was motivated by what it calls the strongest objection to the result: that TinyStories is simple text. Simple text might reward precision over parameter count, and that alone could produce the optimum observed here. The specification excluded every dataset other than TinyStories. I lifted that exclusion for this one arm after the primary sweep had finished.

A6 adds a FineWeb-Edu arm built from `HuggingFaceFW/fineweb-edu` at revision `87f09149ef47`, using the first 359 595 documents of `sample/10BT/000_00000.parquet` (ODC-By). The arm trains its own 512-token tokenizer with the same recipe and uses the same split rule. It has 830 708 553 unique training tokens, 3.1% more than TinyStories. The architectures and per-cell token budgets are identical to the primary sweep. The arm covers 256 Kbit and 4 Mbit with 5 seeds and the TIGHTEST rule. It omits 828 Kbit because the TinyStories argmin there is not established: the null criterion returns null, and under ASPECT the argmin moves to $b = 4$. The arm is secondary and cannot override the TinyStories result.

A6 records predictions P8 to P10 and a check of the residual assumption. For that check, seed 0 of the three cells around each argmin is retrained at half its token budget. The correction `26539a9` changed this check from twice the token budget to half. Doubling would have exceeded the epoch cap at 4 Mbit, so repetition would have confounded the check. According to the amendment, the arm's result directory held only its bigram floor at that time. A further correction, dated 2026-09-27, was committed as `d74768d`. It leaves the check unchanged but withdraws the claim that switching from doubling to halving leaves the comparison unaffected. Under the gain model, the check from below is stricter than the doubling check by a factor that the token exponent fixes. A verdict of "not sensitive" therefore bounds the spread of the *residuals* at a multiple of the gap that the decay per doubling fixes, not at the gap itself. Each gain is a single-seed difference, and the noise of the check has not been measured. The design and predictions are in section 6.

A7: checks of the training recipe. A7 was written after the TinyStories sweep had finished, in response to a literature review that found three recipe choices whose effect on the comparison between bit-widths was large enough, or unmeasured and plausibly large enough, to move the argmin at 828 Kbit and 4 Mbit. The first is end-of-training weight noise at low b, where latent weights collect at quantization decision boundaries and oscillate between levels. This is shown for convolutional networks, most pronounced at 4 bits or fewer in efficient networks with depth-wise separable layers (Nagel et al., 2022), and, for ternary weights, a related failure, weights trapped at the boundary of the zero zone, is reported as deadzone trapping (Huang et al., 2026); I expect the same in my transformers, most at low b. The second is the single untuned learning-rate rule, which I apply at a scale below the 5M–221M models over which Porian et al. (2024) swept the learning rate; they report that hyperparameter sensitivity decreases with scale. The third is token budgets extrapolated beyond the gain model's calibrated range. A7 records that single-trajectory averaging measurements on seven local seed-0 checkpoints had been seen when it was written; they motivated it and are not among its results, which were recomputed on the training host. It specifies four checks, M1–M4, described in section 5.8, with their falsifiers. It changes no prediction, criterion or primary verdict. Its notes fix, before the runs concerned, where M1's averaged weights are stored and which are read at the test read (§17), and the open choices of M2–M4 (§18): grid neighbours, the seed rule, the choice of the best learning-rate multiplier, the continuation learning rate, the

exact half budget, the M4 pairs and the M4 power target.

A8: the learning rate bracketed. A8 was written after 69 of the 72 M2 runs had been read, before any run it specifies. M2 had found the best tested multiplier at the upper edge of its range in most argmin-group cells, so the learning-rate optimum was not bracketed, and §16 allowed only one extension step. A8 extends the search for those cells to $\times 8$, then to $\times 16$ where $\times 8$ is better than $\times 4$, with a cap at $\times 16$, and re-applies the M2 falsifier unchanged over the full range (section 5.8). It changes no earlier verdict: M2 is reported as recorded, and A8 beside it. Its note (§20) fixes, before any A8 run, the tie-break, the candidates for the best multiplier, the handling of failed runs and when each step is decided.

C Reproducibility

Environment. The code requires Python 3.12 and the dependencies declared in `pyproject.toml`. Every row of the results files of the sweeps, the second corpus and the A7 and A8 follow-up runs (`results/a7/m2.csv` to `m5.csv`) records PyTorch `2.14.0+cu130` and CUDA 13.0; the file of the M1 continuations has no such column. Every training run, in the primary and secondary sweeps, the second corpus and A7/A8, used one interpreter: the virtual environment of the training host's main checkout. Its exact package versions are in `results/environment/pip-freeze-training-host.txt`, read from that environment after the last run. All 75 package directories carry the modification time 2026-09-13, the day before the first sweep run, so I take its Python packages to be those of every run, not only of the last (the NVIDIA driver libraries changed during the runs; see below). It holds Python `3.12.14`, `torch 2.14.0` with the CUDA 13.0 runtime wheels (`nvidia-cuda-runtime 13.0.96`, cuDNN `9.24.0.43`), `numpy 2.5.3`, `tokenizers 0.23.2`, `datasets 5.0.1`, `pandas 3.0.5` and `pyyaml 6.0.3`. The file `config/fixed.yaml` declares `norm_dtype: bfloat16`. That entry is not what training did: the RMSNorm gains are held and applied in fp32 (Deviation D6; section 3.1). The file is hashed into every results row, so it is left unchanged; a reproduction should follow the code, not that entry. Training uses bf16 activations and fp32 latent weights; the dequantized weights are rounded to bf16 in each matrix product (section 3.5), and the RMSNorm reductions and the cross-entropy run in fp32. The micro-batch is 128 sequences of 256 tokens and there is no gradient accumulation, which gives the fixed effective batch of 32 768 tokens. The device is resolved at run time and logged in every complete row.

Pipeline. Each stage is one command, run from the repository root. Here `python` is the project environment's interpreter, and `ISOMEM_DATASET` must be unset because it selects the second corpus.

```
python3.12 -m venv .venv && . .venv/bin/activate && pip install -e .   # environment
python scripts/prepare_data.py --force   # splits, tokenizer, streams; rewrites hashes
python scripts/make_baseline.py          # bigram floor
python scripts/dry_run.py                # search and validate every cell; trains nothing
python scripts/pilot.py                  # convergence pilot (A2, A3)
python scripts/calibrate_convergence.py  # token-budget fit, off-grid (A3)
python scripts/test_scale_free.py        # decay per doubling across scales (A3)
python scripts/measure_seed_noise.py     # seed standard deviation, off-grid
python -m isomem.sweep                   # primary and secondary sweeps; resumable
python scripts/build_histories.py        # evaluation histories from the checkpoints
python scripts/gain_precision_check.py   # final checkpoints with bf16 norm gains (D6)
python scripts/analyze.py --json results/analysis.json \
       --histories results/histories.json         # analysis
python scripts/plot.py --histories results/histories.json   # figures
```

Table 16: Committed SHA-256 hashes (`config/fixed.yaml`). The token hashes are asserted on every load of the stream, the tokenizer hash on every load of the tokenizer file, and the document hashes are not asserted on load. Token hashes cover the `uint16` streams read in training. Document hashes cover the raw documents in split order, so they also fix the permutation.

Artifact	SHA-256
Tokenizer	`01424aed96fd2dab1f242607075a29175eb5980ed6c16c78fe9f82506c49c1e9`
Train tokens	`9ee4150f3f785f04d09729868213c5dcb96e8e999e71a55b727a60fc9ba2576e`
Validation tokens	`b2a57c48812f118594f8db522f7fce078dd531a546f9b73eb1727a774a3aabb6`
Test tokens	`efe89eca286f838a9a066decaa7bda58e14222033752512ebdbc122f0b7030f6`
Train documents	`9cfe8e928cb69ba367690d7285d48a041a4c9710ab9c864e42bf63ddf1fdf867`
Validation documents	`7ebd592b8b93912f3a810b979305691257beb847643cac923d00b62f9de59e92`
Test documents	`1243aab56f764cd0f2e31a8e3c7913c0f759e23f3634bdb07d7da238ce7fb8e4`

The file `results/histories.json` holds the evaluation history of every complete run, which is stored in that run's checkpoint; `scripts/build_histories.py` collects them, and with `--check` compares the checkpoints present with the file. Only 7 final checkpoints are held off the training host (seed 0 of 828 Kbit $b \in \{1.58, 2, 3, 4, 6, 8\}$ and of 4 Mbit $b = 1.58$), and their entries match the file exactly; the other entries cannot be rebuilt off the training host.

Frozen data. The document pool is the train split of `roneneldan/TinyStories`. The dataset's own validation split is not used. My splits come from one permutation of the pool, seeded with 1 337, in proportions 0.98, 0.01 and 0.01. The tokenizer is a byte-level BPE with vocabulary 512, trained on my train split only. Table 16 lists the committed hashes. The code asserts the hash of a token stream at every load of that stream, the test stream included, and a mismatch raises an error; this check is present, unchanged, at each of the four commits at which sweep runs were trained (`7694177`, `5b75060`, `d40b754` and `fcdc692`); `isomem/data.py` is byte-identical at all four commits. For the runs recorded with the suffix `-dirty` (below), whose uncommitted changes were not recorded, its identity is not demonstrated. It asserts the tokenizer hash whenever the tokenizer file is loaded; training reads the pre-tokenized streams and does not load that file. The document hashes are recorded when the splits are built and are not asserted on load; a slow test recomputes the validation one. I re-hashed the tokenizer and the train and validation token streams at write-up, and they match the committed values; I did not store the output of this re-hash. The test stream was opened once, by the single test read (section 5.6), whose loader verified its hash against the committed value; the 100 complete primary rows carry its test loss and no other row does (test split read: yes). The read ran on the training host on 2026-10-13 (13:38 to 13:45 UTC), and its output was committed in `5ad131e` at 13:45 UTC. Its file `results/test_read.json` records the commit as `unknown`, which is what the code writes when it finds neither git metadata nor a commit stamp, so the code state of that read is not recorded. The dataset revision is not pinned in the configuration, and the split manifest records only the dataset name. Pinning it in `config/fixed.yaml` would change the configuration hash that every results row carries, so I record it here instead. The documents come from revision `f54c09fd23315a6f9c86f9dc80f725de7d8f9c64` of `roneneldan/TinyStories`, the only revision in the data cache of the machine that built the splits; the token streams were built there and copied to the training host. At write-up I recomputed the train and validation document hashes from that cached revision, offline, and both match table 16; I did not store the output of this recomputation either. The test documents were not hashed, because that would read the test split. The same revision was still the dataset's head on the Hugging Face Hub at write-up.

Code provenance. The results file holds 147 run identifiers: 142 complete and 5 infeasible. The rows the analysis keeps were produced by 3 commits, `7694177-dirty`, `d40b754`, `fcdc692`, each of which descends from the last pre-sweep amendment (A3, 2026-09-14). The value ending in `-dirty`, carried by 40 complete runs, marks a working tree with uncommitted changes whose content was not recorded. The first of these produced every 256 Kbit primary run and the first 828 Kbit primary runs. The second produced the remaining primary runs and the first secondary runs. The third produced the remaining secondary runs. A superseded error row carries a further commit, `5b75060` (below). Every row carries the same configuration hash, `b2100eb1a1f320b6`, which is the truncated SHA-256 of the committed `config/fixed.yaml`.

I checked in two ways that the training computation is the same across the 3 commits. The first check compares source files. Every file under `isomem/` that exists at all of them, and both configuration files, are byte-identical across them, with two exceptions: `train.py` and `sweep.py` changed between `7694177` and `d40b754`, through the intermediate commit `5b75060`. Four modules under `isomem/` exist only at `fcdc692`; no module that training uses imports any of them. Read line by line, the changes to `train.py` and `sweep.py` only affect how results are recorded and how interrupted runs are resumed. They rename a diagnostic field in the code (the column of `results/results.csv` keeps its original header `converged`), redefine the `undertrained` flag, let a checkpoint written before the rename still load, and stop re-recording infeasible cells. None touches the model, the quantizer, the data pipeline, the optimizer, the learning rate, the token budget or the loss evaluation. However, the suffix `-dirty`, which the deploy script wrote on the development machine, means the working tree deployed to the training host at `7694177` held uncommitted or untracked files whose content was not recorded, and comparing source files cannot rule out that tracked sources differed.

The second check compares training runs. Where the two shape rules select the same architecture, a secondary run and the primary run with the same seed are the same computation (appendix A). These pairs span both changes of commit. The 828 Kbit ternary primaries were trained at `7694177-dirty` and their twins at `d40b754`. The primaries of 828 Kbit, $b = 16$ and 4 Mbit, $b = 8$ were trained at `d40b754` and their twins at `fcdc692`. In every pair, the training and validation losses match bit for bit at every evaluation. There are nine pairs (three cells, seeds 0 to 2), and the comparison is of the evaluation histories in `results/histories.json`. Within the cells that can be checked this way, neither the uncommitted changes nor the later commits altered training.

A separate run is reported as further evidence, but it cannot be checked from the repository. After a driver update, the 256 Kbit, $b = 8$, seed-0 run, first trained at `7694177-dirty`, was retrained with later code on the new driver libraries. The message of commit `bd62af7` and an item of my development checklist (item 16; neither is released) state that the retrained validation curve matched the recorded one bit for bit at every evaluation of the retraining, which covered only the first part of the run's token budget, not all of it. The retraining's output was never stored: it is in neither the repository nor the training host's checkout, run directory or temporary directories, which I searched at write-up. The statement therefore rests on the commit message alone. Settling it would take a fresh retraining of that run, compared against its history in `results/histories.json`.

Runs interrupted by the sweep's nightly stops or by stop signals resumed from checkpoints. Resuming reproduced an uninterrupted run exactly in the one case that can be compared: the 828 Kbit ternary primary of seed 2 was stopped at step 30 634 of 68 330 and resumed, and its uninterrupted twin under the other shape rule matches it at all 274 evaluations. The unit test `test_resume_bitwise` (the tests are not released) checks agreement to 10^{-5} on a short 2-bit run on CPU, and the follow-up runner of Amendment A7 has a resume test with exact equality of model and optimizer state, also on CPU (`tests/test_a7_followup.py`, not released). A resumed

run records the commit of the process that finished it. According to the sweep log, 14 complete runs were resumed, and 4 of them were trained under more than one code state. Segments of runs were trained at states that no kept row names: `5b75060`, `5b75060-dirty`, `7694177`, `unknown`. The first steps of the first run were trained before the code on the host carried a commit stamp, and the sweep log records them as `git unknown`. The stamping commit, `7694177`, changed only provenance code. Seeds 0 and 1 of the 828 Kbit, $b = 3$ primary cell were partly trained at `5b75060`: seed 0 at `5b75060-dirty`, whose uncommitted changes were not recorded, and seed 1 at `5b75060`. Both were finished at `d40b754`, and the superseded error row of seed 0 carries `5b75060`. No twin covers this cell, because the two shape rules select different shapes for it. A row records only the commit that finished its run. At 828 Kbit the primary $b = 1.58$ and $b = 2$ cells came from the first commit and the rest from the second, and the secondary $b = 2$ cell mixes the second and third. The rerun of 256 Kbit, $b = 8$ described above does not exercise the ternary path. To reproduce the TinyStories runs exactly, check out `fcdc692`. Later commits add second-corpus support (a new corpus module and changes to the data, tokenizer, evaluation, analysis and sweep modules, selected by the environment variable above) and an optional learning-rate multiplier in `train.py` for the A7 arm, whose default of 1.0 leaves the learning rate unchanged.

Results file. `results/results.csv` was append-only during the sweep and is published complete. The single test read afterwards filled `test_loss` and `test_ppl` in the 100 primary rows and changed no other cell. In total, 6 run identifiers appear more than once. Each of the 5 seeds of 256 Kbit, $b = 16$ was recorded twice before infeasibility became a terminal status. The other duplicate is 828 Kbit, $b = 3$, seed 0: an error row, from a checkpoint written before the column rename that could not be loaded, followed by its completed retry. The analysis keeps the last row for each identifier. In rows from `7694177-dirty`, the `undertrained` column holds the retired tail-drift test. The analysis therefore recomputes the flag from `tokens_seen` and the cell's token budget, and does not read the column. A run counts as undertrained if it received fewer than 99.9% of its token budget; the budget is rounded down to whole optimizer steps, so a run that finishes its schedule falls short of it by less than one step. Under that definition, 0 of the 142 complete runs are undertrained. Checkpoints hold the final-step weights, not the weights at `val_loss_best`. Any later evaluation of a checkpoint, including the test evaluation, therefore measures the final step.

Compute. Every complete run reports device `cuda`, PyTorch `2.14.0+cu130` and CUDA 13.0 in its row, and the sweep log shows a single working directory. The GPU of that host is an NVIDIA GeForce RTX 3060 Laptop GPU (`results/bench_training_host_cuda.json`) under Linux; the rows themselves do not record the GPU name. The convergence pilot ran on a different machine, on device `mps` (`results/pilot.json`), so the pilot evidence behind A3 comes from another device than the sweep. The calibration, scale-free and seed-noise runs do not record their device; their files (`results/calibration.json`, `results/scale_free.json`, `results/seed_noise.json`) have no device field, and their console output, which printed it, was not kept. I infer, but did not record, that they ran on the training host's GPU, on two grounds. First, the calibration file records a throughput (`tokens_per_s`) that is between 0.77 and 1.18 times the sweep's CUDA throughput on the nearest uninterrupted cells (the 84 k-parameter cell at 256 Kbit and the 2.6 M-parameter ternary cell at 4 Mbit) and, for its small model, is about twice (1.9 to 2.2 times) the convergence pilot's sustained rate on the other machine's device at nearly the same size. Second, the planning code, written before the sweep, takes its throughput anchors from these runs and labels them RTX 3060 throughput (`scripts/dry_run.py`). The driver is not recorded per row. The driver libraries were upgraded from 595.84 to 595.91 at 2026-09-25 06:34 UTC (my development checklist, which is not

released). The last sweep process started at 2026-09-24 20:00 UTC, before the upgrade, and the recorded wall-clock times of its eight runs (7.55 h in total) put its end at about 03:35 UTC, before the upgrade as well. Every row of `results/results.csv` is therefore inferred, not recorded, to have used the 595.84 libraries. The second-corpus, A7 and A8 runs started after the upgrade, and I infer, without a record, that they used the 595.91 libraries. The sweep ran from 2026-09-14 to 2026-09-25. Complete runs trained on 290 586 558 464 tokens in total: 197 918 228 480 in the primary sweep and 92 668 329 984 in the secondary sweep. These totals exclude the pilot, calibration and seed-noise runs. The column `wall_clock_s` cannot give the total training time on its own: the timer starts after a checkpoint is loaded, so for each of the 14 resumed runs it covers only the final segment. I therefore estimate each resumed run's time from the per-step rate of its final segment, with the step it resumed from taken from `results/sweep.log`. Where a cell has both resumed and uninterrupted seeds, the estimate for each of the 14 resumed runs is within 2.9% of the uninterrupted seeds' mean. The two sweeps took an estimated 174 GPU-hours, excluding the pilot, calibration and seed-noise runs and the time between runs.

Code and data. The code, configuration, tokenizer, split manifest, results file, evaluation histories and analysis are in one repository (`https://github.com/milar111/iso-memory-scaling`). The tests, the paper's sources and the script that writes the numbers into the paper are not part of the release. The token streams are not committed. `prepare_data.py --force` regenerates them and rewrites the hashes in `config/fixed.yaml` and in the split manifest without comparing them with the old ones; a reproduction must confirm with `git diff` that the rewritten values equal table 16. Checkpoints are not in the repository.

D Iso-Memory Audit

The experiment compares bit-widths at equal storage, so each cell must actually store its budget. Figure 4 checks this. It plots each primary cell's mean validation loss, ±1 seed standard deviation, against the weight bits the cell actually stores. The horizontal axis is logarithmic, and dotted lines mark the three budgets. If the search or the tolerance check had failed, cells would sit away from their budget line. Every cell sits on its line. The figure cannot detect an error in the accounting itself, because it plots the accounting's own count. That count is checked separately (below). Over all 142 complete runs, the largest absolute relative budget error is 4.15% (below budget), from the secondary sweep (4 Mbit, $b = 2$, ASPECT), inside the ±5% tolerance. In the primary sweep the largest is +2.15% (256 Kbit, $b = 6$).

The tolerance is enforced before training, not checked afterwards. The search refuses shapes outside it. Training builds the model, asserts that its parameters match the accounting, and raises an error before the first step if the counted bits miss the budget. `test_bit_accounting_exact` compares the count with hand-computed configurations, with no tolerance. The errors are small but nonzero, and they have both signs. As preregistered, the headline figure plots loss against actual rather than target bits per weight (fig. 1). The largest embedding share of any trained cell is 49.5%. It belongs to the 256 Kbit, $b = 8$ cell. No cell exceeds the preregistered embedding-bound threshold of one half, so no marker is drawn hollow.

E Selection Gain

This analysis is exploratory (section 4). The primary metric, `val_loss_best`, is the minimum of a run's validation curve over its evaluations, all scored on the same fixed validation batches. Table 17

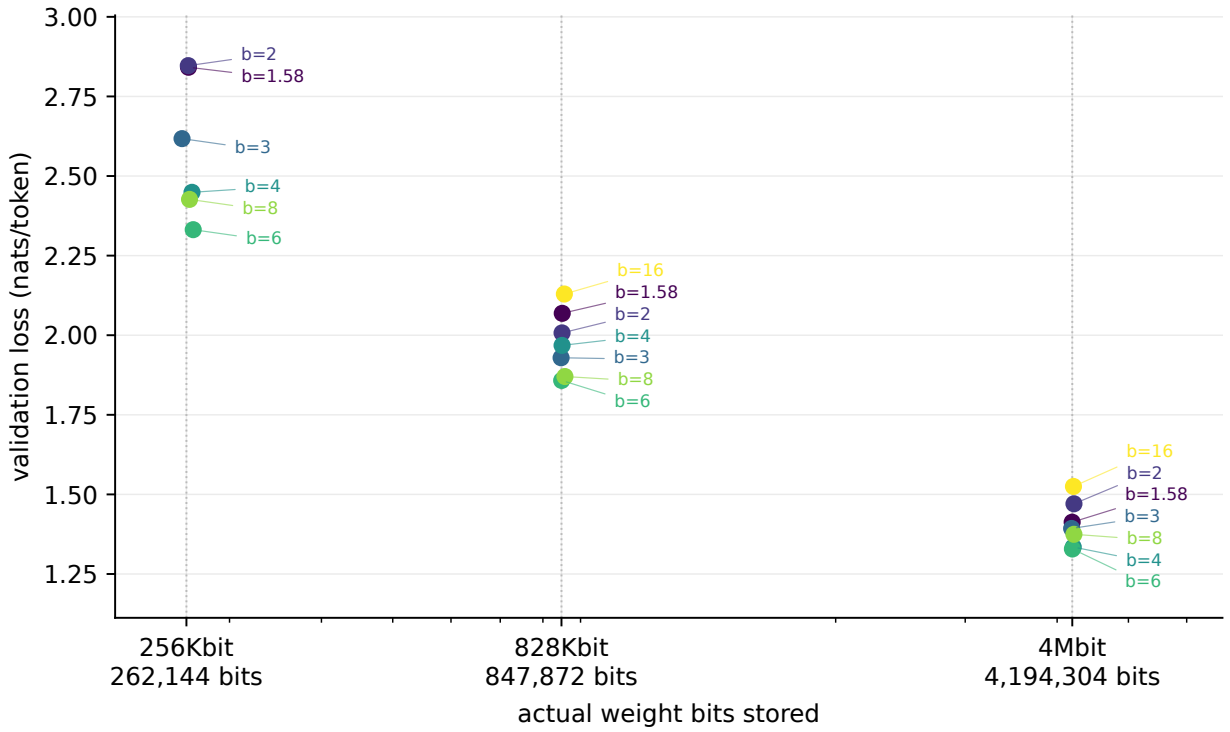


Figure 4: Audit of the iso-memory constraint. Each point is a primary cell's mean validation loss (± 1 seed sd, 5 seeds), plotted against the weight bits it actually stores. Labels give b. Dotted lines mark the three budgets. The largest budget error over all complete runs is 4.15% (tolerance $\pm 5\%$). Embedding-bound cells would be drawn hollow; there are none, and the largest embedding share among trained cells is that of 256 Kbit, $b = 8$ (49.5% of its stored bits).

reports the selection gain, the final evaluation minus that minimum. It is largest at the lowest b at every budget. At 256 Kbit it falls at every step up in b; the only step up in b at which the gain rises is $b = 3 \to 4$ (+0.0006, within seed sd 0.0064) at 828 Kbit and $b = 6 \to 8$ (+0.00003, within seed sd 0.0008) at 4 Mbit. Low-b cells train on more tokens (table 14), so they are also evaluated more often.

A gain of this kind arises whenever a minimum is taken over noisy evaluations, even if the underlying curve never rises. A no-rise null, built from each run's own non-increasing trend and its measured evaluation-to-evaluation fluctuation, accounts for the observed gains (section 5.5), and no cell shows a late rise at Holm-adjusted $p < 0.05$. (The alternative estimator of the fluctuation, from residuals about the trend, rejects one cell, 4 Mbit, $b = 4$, but it rejects 14.2% of 400 synthetic no-rise curves (real trends plus independent Gaussian noise at each run's own difference-based standard deviation, which favours the difference-based estimator by construction), against 2.8% for the estimator used here. These rates are per curve at unadjusted $p < 0.05$, whereas the cell test above is Holm-adjusted. The calibration assumes independent noise and does not measure power against a real late rise.) I therefore treat the gain as selection bias of the metric: the data give no evidence of a late rise, although the power of this test against one is not measured (fig. 3). It lowers `val_loss_best` most at low b. That favours the lower neighbours of b^*, against which the argmin holds despite the bias in their favour, and favours b^* relative to higher bit-widths, including $b = 8$, which sets the gap at 256 Kbit and 828 Kbit (section 5.5).

If each run is instead scored by its final evaluation, the argmin is $b = 6$ at 256 Kbit, $b = 6$ at

Table 17: (a) Selection gain: final validation loss minus the reported minimum (`val_loss_best`), mean ± seed sd per cell, in nats. It is the distance from the minimum over the run's noisy evaluations to its last evaluation. It is not a measure of a late rise: a flat or still-falling curve also produces it, and whether any curve rose after its minimum is tested in section 5.5. Monotone: whether the gain falls at every step up in b. Inversions: 828 Kbit, $b = 3 \to 4$: +0.0006, within seed sd 0.0064; 4 Mbit, $b = 6 \to 8$: +0.00003, within seed sd 0.0008. At every budget the gain is larger at the lowest b than at the highest, so taking the minimum lowers low-b losses more than high-b losses. It works against b^* relative to lower bit-widths and in its favour relative to higher ones (section 5.5). (b) Shift of the minimum: each run's minimum over its odd-numbered (even-numbered) evaluations is compared with the loss at the next evaluation of the other parity, and the two directions are averaged (mean ± seed sd per cell, nats). A minimum over half of the evaluations carries less selection than the minimum over all of them, so this is an order-of-magnitude check on the downward bias of `val_loss_best`, not a measurement of it. Exploratory.

Budget	1.58	2	3	4	6	8	16	monotone
(a) Selection gain: final minus `val_loss_best`								
256 Kbit	0.0474 ±0.0245	0.0273 ±0.0147	0.0219 ±0.0131	0.0121 ±0.0064	0.0061 ±0.0045	0.0018 ±0.0017	—	yes
828 Kbit	0.0197 ±0.0095	0.0129 ±0.0038	0.0094 ±0.0063	0.0100 ±0.0064	0.0026 ±0.0009	0.0012 ±0.0010	0.0008 ±0.0006	no
4 Mbit	0.0075 ±0.0031	0.0061 ±0.0049	0.0053 ±0.0024	0.0049 ±0.0030	0.0008 ±0.0005	0.0008 ±0.0008	0.0005 ±0.0004	no
(b) Shift of `val_loss_best` from alternate evaluations								
256 Kbit	0.0354 ±0.0109	0.0486 ±0.0237	0.0270 ±0.0105	0.0124 ±0.0039	0.0049 ±0.0030	0.0020 ±0.0012	—	
828 Kbit	0.0201 ±0.0053	0.0150 ±0.0102	0.0115 ±0.0055	0.0142 ±0.0048	0.0030 ±0.0003	0.0007 ±0.0005	0.0004 ±0.0004	
4 Mbit	0.0060 ±0.0025	0.0105 ±0.0038	0.0050 ±0.0021	0.0024 ±0.0004	0.0010 ±0.0005	0.0007 ±0.0004	0.0007 ±0.0004	

828 Kbit and $b = 6$ at 4 Mbit. The best final losses are 2.3376, 1.8606 and 1.3294 nats. The argmin is therefore the same under both metrics. Applied to final losses as an exploratory check, the null criterion returns not null, null and not null (section 5.5); the preregistered verdicts are those on `val_loss_best`. The qualification of the 828 Kbit result (a null verdict and an argmin that depends on the shape rule, section 5) stands.

F Deviations from the Specification

The specification was written before the code. Implementing it produced eight deviations, and all eight were declared in Section 9 of the preregistration in its first commit (`de30cb9`), before any sweep run. That table has not changed since. Two later changes are amendments rather than deviations and are described in appendix B: the convergence protocol (A3), which replaced the specification's early stopping, and the second corpus (A6). Further departures from the preregistration were made or found after the sweep began and were declared nowhere in advance. They are listed at the end of this section.

D1: the 828 Kbit budget is read in binary units. The budget is $828 \times 1024 = 847\,872$ bits, consistent with the binary reading of 256 Kbit and 4 Mbit. The specification's worked example implies the decimal 828 000. The two readings differ by less than the ±5% search tolerance. That does not make the choice irrelevant to shape selection. I repeated the search at 828 000 bits: TIGHTEST selects a different shape for 7 of the 7 828 Kbit cells, and ASPECT for 3. The budget was fixed before any run and was not revisited. The sensitivity is one more instance of the dependence on shape at 828 Kbit that section 5 reports.

D2: 256 Kbit at $b = 16$ is infeasible. The grid has 20 feasible cells, not 21. The vocabulary (512) and the minimum width (32, set by the head dimension of 32) are held fixed. With these, the tied embedding alone costs $512 \times 32 \times 16 = 262\,144$ bits, which is 100.0% of the 256 Kbit budget. Any transformer block exceeds the budget. The cell is reported as the limit of the embedding-bound regime, not dropped silently. Its 5 planned runs are recorded with status `infeasible`.

D3: ternary weights are counted at 1.58 bits. The accounting uses the grid's declared value of 1.58 rather than $\log_2 3$, and not the 1.6 bits per weight of packing five trits per byte. Every trained ternary shape would remain inside the ±5% tolerance under either alternative cost. As with D1, however, the search minimises budget error. Under $\log_2 3$ it would select a different ternary shape at 256 Kbit and at 4 Mbit, and under 1.6 it would select a different ternary shape at all three budgets.

D4: absmean scaling at every bit-width. The specification requires absmean scaling for ternary weights, as in BitNet b1.58 (Ma et al., 2024), and leaves the other widths open. There, ranges set from the extreme values of the tensor (min–max, or its symmetric form absmax) are a common choice (Jacob et al., 2018; Nagel et al., 2021). With a single outlier 100 times the largest of the other weights of a Gaussian test tensor, absmax zeroes more than 99% of the weights at $b = 1.58$, 2 and 4 (`test_absmax_collapses_on_outlier`), and absmean does not. I expect absmax to degrade differently at different b, which is the axis under study; I did not train with it, so this expectation is untested beyond that one tensor. I use absmean at every width. For $2 \le b \le 8$ the step is the MSE-optimal step of a symmetric quantizer with that number of levels on a Gaussian source by the criterion of Max (1960) (his tables stop at $N = 36$ levels; for $N = 64$ and 256 I minimise the same criterion); on the integer grid actually used it is within 0.61% of that grid's own optimum (section 3.5). It is applied to $\sigma = \text{mean}|w|\sqrt{\pi/2}$. This is a fixed rule, not tuned on my data. The ternary weight quantizer follows BitNet b1.58 (absmean scale, rounding and clipping to $\{-1, 0, 1\}$).

D5: idempotence is tested on the grid and, on a synthetic tensor, on integer codes. The specification asks for $q(q(x)) = q(x)$. Under absmean scaling this cannot hold for the values themselves. Quantization zeroes some weights, which lowers the absmean and so shrinks the recomputed step. I test the properties that do hold. First, projection onto the grid at a fixed step is exactly idempotent. Second, on a Gaussian test tensor the integer codes stay the same when the step is recomputed (`test_integer_codes_are_idempotent`). That does not hold for trained weights. I re-quantized each two-dimensional tensor of the seed-0 828 Kbit checkpoints with its recomputed step: no code changed at $b = 1.58$ and $b = 2$, and the share of codes that changed is 0.57% at $b = 3$ and 14.9% at $b = 8$. Training always quantizes the fp32 latent weights, never an already-quantized tensor, so the training computation is unaffected.

D6: RMSNorm gains are counted at 16 bits. The specification keeps the gains in bf16 and counts them. The accounting counts them at 16 bits, as specified. The deviation is that their latent parameters are fp32, like every other latent weight, rather than bf16, and they are applied in fp32. Rounding them to bf16 changes the final-step validation loss by at most 4.1×10^{-4} nats on the checkpoints re-evaluated; counted at 32 bits, 2 cells would leave the tolerance (section 3.1).

D7: weight decay excludes the norm gains. The specification gives a weight decay of 0.1 without qualification. Following common practice, I exclude the one-dimensional RMSNorm gains from decay, identically in every run (`test_weight_decay_excludes_norm_gains`).

D8: the search tie-break is a named rule. The specification selects the shape that minimises $|\text{actual bits} - B|$. That does not identify a unique shape: several shapes with very different aspect ratios can lie within a fraction of a percent of the best error. Two deterministic rules are implemented. TIGHTEST is the primary rule and follows the specification literally: it minimises the error and breaks ties lexicographically on $(L, d, \text{mlp ratio})$. ASPECT picks, within the $\pm 5\%$ band, the shape whose d/L is closest to a fixed target of 10, and then minimises the error (section 3.3). The two rules select different shapes for most cells at 828 Kbit and 4 Mbit (table 14). This difference is why the secondary sweep exists.

Departures made after the sweep began, not declared in advance. None of the following is recorded in the preregistration. Each was made, or found, after the first sweep run at 2026-09-14 06:37 UTC.

The undertrained flag was redefined. Item 6 of Section 6 of the preregistration excludes undertrained runs from the headline curve, and defines them as runs that reach their token cap without stopping early. A3 removed early stopping, so every run meets that definition. The code run under A3 (`7694177`) set the flag instead from the tail-drift test that A3 had rejected. Commit `5b75060`, at 2026-09-15 10:52 UTC, replaced it: a run is undertrained if it completed fewer optimizer steps than its schedule. By then 40 runs were complete, and the old flag marked 32 of them, which item 6 of Section 6 would have excluded. The analysis does not read this flag: it counts a run as undertrained if it received fewer than 99.9% of its token budget, and under that definition 0 of the 142 complete runs are undertrained (appendix C).

The pilot cells are in the headline curve. Section 5.4 of the preregistration excludes the pilot cell from the headline curve, and A2 kept that exclusion for all three pilot cells, while the run plan of §5.1 trains every feasible cell. The pilot cells are grid cells: $b = 3$ at 256 Kbit and at 828 Kbit, and $b = 6$ at 4 Mbit. They were retrained under the final protocol with all 5 seeds and are included in every table and figure; the analysis without them is reported beside it (section 4 and table 6). At 256 Kbit and 828 Kbit the argmin is the same without them ($b = 6$ and $b = 6$). At 4 Mbit it is not: without the $b = 6$ cell, the lowest mean is at $b = 4$ (1.3346, against 1.3286 at $b = 6$), and the null criterion returns not null (sections 4 and 5.1).

The power threshold's standard deviation was chosen after results existed. The preregistration does not say which seed standard deviation the power threshold uses. The larger of the two cells' values first appears in the analysis code of commit `6c2fd3a` (2026-09-16). The results file committed at the same time held every 256 Kbit primary run and 27 of the 35 828 Kbit primary runs (section 4).

The matched-parameter pairing and the scoring of P3 were set after results existed. The preregistration plans the matched-parameter reading (item 2 of its Section 6) but gives neither a rule for pairing cells nor a scoring rule for P3. The pairing rule first appears in commit `6c2fd3a` (2026-09-16): cells are paired when their parameter counts agree within 12%. The report of that

commit printed the P3 verdict unconditionally. The scoring rule, that P3 holds when higher precision wins every such pair while the matched-memory argmin is interior, was first written into the paper pipeline in commit `357ecf6` (2026-09-27), after the sweep and its analysis (section 5.3). The test that item 2 of Section 6 names, whether the argmin moves between the matched-parameter and matched-bits panels, cannot be computed on this grid, so P3 is reported as not testable as preregistered.

The 4 Mbit pilot record was dropped and restored. Commit `483f44c` (2026-09-26) removed the 4 Mbit pilot run from `results/pilot.json`. The record was restored from commit `5df1129`, where it was first written, in commit `31cebe9`.